

电子科技大学
UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS



论文题目 基于 RISC-V 的微内核虚拟化方案研究

学科专业	<u>计算机科学与技术</u>
学 号	<u>202321081001</u>
作者姓名	<u>李金哲</u>
指导教师	<u>罗 蕾 教 授</u>
学 院	<u>信息与软件工程学院（示范性软件学院）</u>

分类号 _____ 密级 _____

UDC^{注 1} _____

学 位 论 文

(题名和副题名)

(作者姓名)

指导教师 _____

(姓名、职称、单位名称)

申请学位级别 _____ 学科专业 _____

提交论文日期 _____ 论文答辩日期 _____

学位授予单位和日期 _____

答辩委员会主席 _____

评阅人 _____

注 1：注明《国际十进分类法 UDC》的类号。

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Discipline _____

Student ID _____

Author _____

Supervisor _____

School _____

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知,除了文中特别加以标注和致谢的地方外,论文中不包含其他人已经发表或撰写过的研究成果,也不包含为获得电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

作者签名: 作者签名

日期: 2024 年 08 月 31 日

论文使用授权

本学位论文作者完全了解电子科技大学有关保留、使用学位论文的规定,同意学校有权保留并向国家有关部门或机构送交论文的复印件和数字文档,允许论文被查阅。本人授权电子科技大学可以将学位论文的全部或部分内容编入有关数据库进行检索及下载,可以采用影印、扫描等复制手段保存、汇编学位论文。

(涉密的学位论文须按照国家及学校相关规定管理,在解密后适用于本授权。)

作者签名: 作者签名

导师签名: 导师签名

日期: 2024 年 08 月 31 日

摘 要

关键词：

ABSTRACT

Keywords:

目 录

第一章 绪论	1
1.1 研究工作的背景与意义	1
1.2 国内外研究背景与现状	3
1.2.1 虚拟化技术研究现状	3
1.2.2 基于硬件虚拟化扩展的 TEE 构建机制研究现状	4
1.2.3 基于指令集安全特性的架构级 TEE 研究现状	5
1.2.4 异构计算环境下 GPU-TEE 安全隔离与防护技术研究	9
1.3 本文主要贡献和创新	10
1.4 论文结构	10
第二章 相关技术理论概论	13
2.1 RISC-V 背景知识	13
2.1.1 RISC-V 特权级介绍	13
2.1.2 RISC-V 中断与异常处理机制介绍	14
2.1.3 RISC-V 内存保护机制介绍	15
2.1.4 RISC-V 硬件虚拟化拓展 (H-extension) 介绍	16
2.2 TEE 相关技术介绍	18
2.2.1 基于指令集的 TEE 分类	18
2.2.2 基于可信域抽象的 TEE 分类	18
2.2.3 面向异构计算环境的 TEE	18
2.3 微内核相关知识介绍	19
2.3.1 微内核操作系统 EasyAda 介绍	19
2.3.2 微内核虚拟化方案 EasyAda-hypervisor	20
2.4 Rust 相关知识介绍	22
第三章 基于 RISC-V 的微内核虚拟化方案设计	23
3.1 设计目标和需求分析	23
3.2 EasyAda-trust 工作流	23
3.3 威胁模型	24
3.4 EasyAda-trust 详细设计	25
3.4.1 总体设计	25
3.4.2 安全监视器 EasyAda-SM 设计	27

3.4.3 安全特性设计	30
3.5 本章小结	41
第四章 基于 Rust 的内存安全固件实现	42
4.1 内存安全固件	42
4.2 安全区域和内存管理层 SRMM 实现	42
4.2.1 内存管理原语	43
4.2.2 内存保护原语	46
4.2.3 安全内存管理原语	48
4.3 面向 CVM 的 enclave 管理模块实现	51
4.3.1 Enclave 间内存安全	52
4.3.2 安全启动	52
4.3.3 中断和异常安全	54
4.3.4 IOPMP 管理	55
4.4 本章小结	57
第五章 基于微内核的可信虚拟化方案实现	58
5.1 安全 CPU 虚拟化实现	58
5.1.1 安全上下文管理	58
5.1.2 安全异常处理	60
5.2 安全内存虚拟化实现	60
5.2.1 安全区域生命周期管理	61
5.2.2 不可信共享内存映射	62
5.3 安全 I/O 虚拟化实现	63
5.3.1 安全设备管理	64
5.3.2 数据使用环境保护	65
5.3.3 CPU 侧 I/O 数据安全	65
5.3.4 设备侧 I/O 数据安全	66
5.4 机密异构计算环境实现	66
5.4.1 硬件加速器的保护	66
5.4.2 加速器管理 CVM 初始化	67
5.4.3 建立信道	68
5.5 本章小结	69
第六章 系统测试与分析	70
第七章 总结和展望	71

参考文献	72
攻读硕士学位期间取得的成果	76

图目录

图 1-1 虚拟化技术分类	4
图 1-2 章节结构	11
图 2-1 未引入硬件虚拟化拓展时 RISC-V 系统架构	13
图 2-2 引入硬件虚拟化拓展后的 RISC-V 系统架构	17
图 2-3 微内核操作系统 EasyAda 总体架构	19
图 2-4 EasyAda-hypervisor 总体架构	21
图 3-1 基于 RISC-V 的微内核虚拟化方案总体架构	25
图 3-2 EasyAda-SM 总体架构	28
图 3-3 EasyAda-SM 的内存管理模式	29
图 3-4 EasyAda-trust 可信启动流程	31
图 3-5 CVM 二阶段页表初始化	34
图 3-6 CVM 的 I/O 数据保护机制	38
图 3-7 硬件加速器管理 CVM 设计	40
图 4-1 SRMM 管理单元实现	42
图 4-2 PageAllocator 结构	45
图 4-3 PMP 槽结构	47
图 4-4 PMP 运行时配置	49
图 4-5 PageAllocator 结构	50
图 4-6 不同类型 Enclave 与区域类型和分配器的关联关系	51
图 4-7 CVM 管理器结构	51
图 4-8 EasyAda -SM 中非安全/安全设备的 IOPMP 管理	56
图 5-1 EasyAda-trust 下 hart 分区结构图	59
图 5-2 EasyAda-trust 共享内存创建流程	63
图 5-3 EasyAda-trust 下 CVM 的 I/O 密钥管理	65
图 5-4 引入安全设备后的 CVM 安全内存视图	67

第一章 绪论

1.1 研究工作的背景与意义

近年来，物联网（Internet of Things, IoT）和嵌入式技术的迅速发展正在推动各行业向智能化方向升级。物联网通过将大量边缘设备纳入到网络管理中，实现对边缘数据的收集，传输和处理，并且已经在智能家居、工业自动化、智慧城市和医疗健康等领域得到了广泛应用。作为物联网的核心，嵌入式系统在设备控制、传感器数据监测和实时数据处理等方面发挥着关键作用，为各种智能设备提供技术支撑。

然而，随着物联网设备功能的不断扩展，嵌入式技术正面临更加复杂的应用需求和安全问题：（1）在场景方面，多样化的嵌入式应用场景对嵌入式系统的实时性、可靠性和资源利用效率提出了差异化要求，对于关键安全场景，需要确保高实时性，可靠性和安全性；而消费电子更关注低成本和高能效。传统嵌入式系统通常只运行单一操作系统，在需要兼顾多种功能需求时，往往依赖于增加硬件资源，从而导致系统成本、复杂性和功耗的显著提升；而基于 OpenAMP^[1] 的多操作系统部署方案虽然可以一定程度上兼顾多种功能需求，但是无法很好的确保操作系统间的资源隔离和安全。（2）在安全方面，传统嵌入式系统通常针对特定场景高度定制化，其软硬件高度固化因此攻击面较小。但是随着边缘计算和 IoT 的发展，嵌入式场景需要承担更复杂的计算任务，并且依赖互联网实现设备控制和数据处理。系统的代码量膨胀和开放的网络环境导致嵌入式系统攻击面急剧增大，对嵌入式系统的安全性提出了更高的要求。

为了应对嵌入式硬件资源丰富化和应用场景复杂化的趋势，现代嵌入式系统正在从单一操作系统演变为混合关键系统（Mixed Critical System, MCS）以提升系统资源利用率和安全性，降低重复开发成本。现代 MCS 通过引入虚拟化技术，对物理硬件进行抽象，实现在同一硬件环境上部署多操作系统以应对不同应用场景的差异化需求，并且通过隔离不同运行环境间的内存，CPU 和 I/O 等资源，增强了系统的稳定性和可靠性。但是基于虚拟化的隔离方案以虚拟机监视器（hypervisor）作为可信计算基（Trust Computing Base, TCB），可信基较大易被攻破，难以有效保护用户的隐私数据。为了确保数据得到有效保护，用于保护敏感代码与数据的可信执行环境（Trust Execution Environment, TEE）在过去十年间已经得到广泛普及。目前所有主流 CPU 指令集均已推出各自的 TEE 解决方案（如 ARM 的 TrustZone^[2]、Intel 的 SGX^[3]，AMD 的 SEV^[4]，以及 RISC-V 的 Penglai^[5] 和 Keystone^[6] 等）以

提供一个安全的执行环境，该环境通常被称为飞地（enclave）或安全世界（secure world）。TEE 的应用场景广泛，可部署在云服务器、移动设备、IoT 设备、传感器以及硬件令牌等环境中。

目前嵌入式场景下大部分 TEE 方案基于 ARM TrustZone 拓展和 GP-TEE^[7] 规范实现，但是这种实现方式存在部分缺陷：（1）闭源性：ARM 指令集的闭源特性导致其难以根据用户需求进行精细化定制。（2）拓展性不足：ARM TrustZone 将运行环境分为安全世界（Secure World, SW）和普通世界（Normal World, NW），所有的 TA 均使用同一个 SW，TA 间资源隔离主要依赖 TEE-OS，TCB 较大，容易引入代码漏洞或被攻破。（3）虚拟化支持不足：ARM TrustZone 并未在硬件层面支持运行机密虚拟化，导致其需要额外工作以支持运行机密虚拟机（Confidential Virtual Machine, CVM），否则需要针对依赖 TEE 的应用开发专用的可信操作系统（TEE-OS）和可信应用（Trust Application, TA），增加开发难度和成本。

而随着边缘计算的发展，大量的 AI 模型被部署到端侧执行，这些模型需要处理大量的用户隐私数据因此应当得到足够的保护。而将这些模型移植到 TEE-OS 中执行需要花费大量的工作，或者仅使用 TrustZone 保护关键计算路径，但是仍需要付出额外性能开销和侵入式修改模型执行逻辑的代价。为了降低厂商的移植难度，降低 TEE 的准入门槛，主要 CPU 厂商均提出了基于 CVM 抽象的 TEE 框架，如 ARM CCA^[8]。CCA 的设计目标是在保留现有系统软件对 CVM 所需硬件资源进行管理的同时，防止各类软件及其他硬件原语对 CVM 的内容进行窥探或篡改。虽然 CCA 在增强安全性的同时降低了软件开发成本，但是 CCA 主要是面向服务器或高性能嵌入式场景，难以兼容基于 TrustZone 的 TEE 方案，因此 CCA 难以满足中低端嵌入式设备对功耗和实时性的要求，升级现存大量嵌入式设备的成本极大。

虽然 TEE 能够防御不可信组件对 TEE 的篡改或非法访问，但是 TEE 本身仍然可能存在代码漏洞，并且因为 TEE 通常具备更高的权限，其中的代码漏洞会造成更大的破坏，严重影响 TEE 的安全性和系统稳定性。以 ARM TrustZone 为例，披露的代码漏洞中多数为不安全内存访问带来引发内存破坏导致的^[9]。而 Rust^[10] 通过编译时的内存安全检查，可以实现可靠性与高效性的兼得，在尽可能消除代码中不安全内存操作的同时确保性能，非常适合开发内存安全的嵌入式应用。目前已经有部分研究^[11, 12] 使用 Rust 实现了 TA 以消除内存漏洞，但是目前缺少使用 Rust 实现软件 TCB 的相关研究工作。

RISC-V 是一种基于既定精简指令集计算机（RISC）原理的开放标准指令集架构（ISA）。得益于其新颖的开放指令集和指令集模块化的特点，RISC-V 可以在保持简洁性和可靠性的同时支持多种平台，硬件系统可以根据场景需要包含可选的 RISC-V 标准拓展，甚至实现用户自定义的拓展指令，这使得 RISC-V 可以支持从

嵌入式微控制器到高性能计算的丰富场景。过去几年中 RISC-V 已经在嵌入式领域得到了广泛应用，主流工具链和操作系统均添加了对 RISC-V 的支持。虽然目前 RISC-V 已经完善了硬件辅助的虚拟化拓展（H 拓展）并且提供了一些硬件以支持在 RISC-V 上实现 TEE 方案，但是相关研究仍在起步阶段。

综上所述，结合开源的 RISC-V 指令集，实现支持运行机密虚拟机的 TEE 方案，并且通过引入内存安全语言 Rust 消除软件 TCB 中存在的内存漏洞，在确保嵌入式系统实时性和可靠性的同时，提高嵌入式系统的安全性和资源利用率，降低 TA 开发成本和 TEE 准入门槛这一课题具有重大研究意义。

1.2 国内外研究背景与现状

1.2.1 虚拟化技术研究现状

虚拟化技术的逻辑起源可追溯至 20 世纪 70 年代，其核心本质在于通过构建硬件抽象层实现物理资源与执行环境的解耦。1974 年，Popek 与 Goldberg 提出了虚拟机监视器（VMM）的经典理论框架，界定了合格 VMM 须具备的等效性（Equivalence）、高效性（Efficiency）及资源控制（Resource Control）三大关键属性^[13]。

虚拟化的实现基石在于“陷入-模拟”（Trap-and-Emulate）机制，该机制深度依赖于指令集中特权指令与敏感指令的正交性逻辑：当硬件架构满足敏感指令集为特权指令集的子集时，即具备原生虚拟化能力。在此架构下，客户机操作系统（Guest OS）执行敏感指令将触发特权级陷阱（Trap），由处于最高特权级的 Hypervisor 捕获异常并实施语义模拟，从而在保障物理资源受控的前提下，实现多虚拟机间的强隔离与资源复用^[14]。根据实现路径及 Guest OS 的感知程度，虚拟化技术演进为两种主流范式：全虚拟化通过完全的硬件模拟支持未经修改的 Guest OS，具备极佳的兼容性；而半虚拟化则通过超级调用（Hypercalls）接口建立 Guest 与 Hypervisor 的协同演进机制，以适度的非侵入式修改换取更高的交互效率。

根据 Hypervisor 与宿主环境的依赖关系，其部署架构可分为 Type-1（裸金属型）与 Type-2（宿主型）两类，如 1-1 所示。Type-1 架构直接植根于物理硬件，拥有对底层资源的绝对控制权，因其访存路径极短且指令处理链路精简，在性能表现与硬件利用率上占据显著优势。相比之下，Type-2 架构寄生于宿主操作系统（Host OS），需跨越 Host OS 的抽象层间接访问硬件。尽管这种间接调用机制引入了额外的性能损耗，但其能够充分复用 Host OS 成熟的驱动体系与管理工具，展现出更强的环境适配性与部署灵活性。

面向传统云服务场景的虚拟机监视器（如 KVM^[15]、Xen^[16] 等）侧重于大规模租

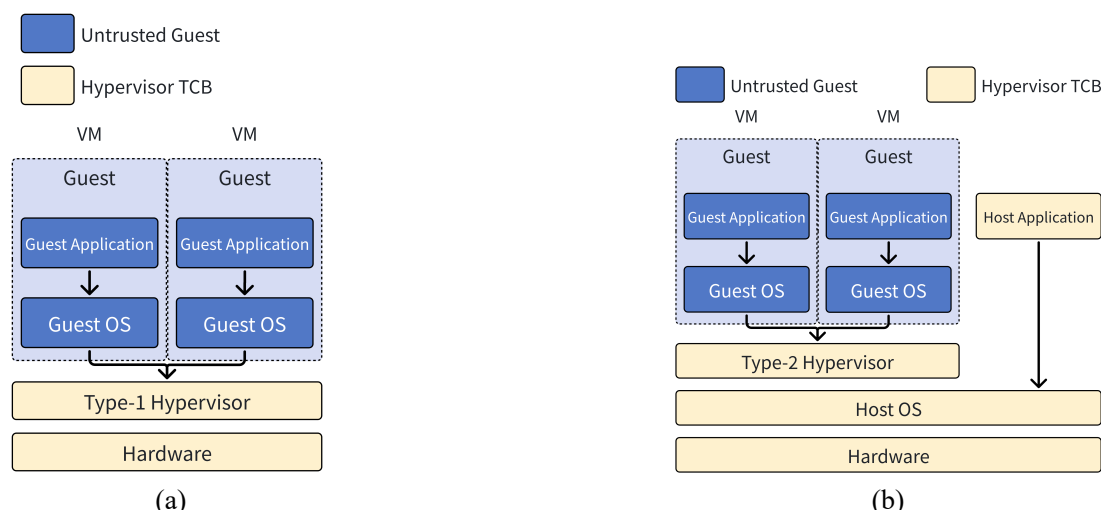


图 1-1 虚拟化技术分类

户的资源隔离。尽管已有研究尝试对其进行轻量化改造以向嵌入式场景迁移^[17, 18], 但受限于其宏内核架构下庞大的可信计算基 (TCB), 仍难以满足嵌入式环境对计算资源受限及严苛实时性的双重要求。为此, 学术界与工业界相继提出了 Bao、ACRN、XtratuM 等针对高实时性与低功耗定制优化的轻量级方案^[19-21]。然而, 此类方案在实际部署中仍面临资源动态调配率偏低及系统架构可拓展性受限等瓶颈。

在车载与工业控制等高实时性关键场景中, 基于微内核的虚拟化技术凭借其极简的特权层级与卓越的系统响应性能, 成为突破上述瓶颈的优选路径。依托 seL4 微内核构建的 camkes-vmm 框架, 通过极致的架构精简与低延迟设计, 确立了高实时与低开销需求的理想范式^[22]。同时, 以 QNX 为代表的商用微内核方案, 通过跨架构的强兼容性与关键任务 (Mission-critical) 调度策略, 在自动驾驶等领域印证了微内核架构在兼顾强实时性与高安全性方面的固有优势^[23]。

综上所述, 虚拟化技术已从早期的资源模拟演变为支撑现代复杂系统的核心基底。微内核虚拟化技术通过架构级的解耦与精简, 在系统灵活性与执行效率之间取得了优异平衡, 高度契合现代嵌入式系统的复杂虚拟化需求, 并已在严苛的实时性场景中确立了显著的技术优势地位。

1.2.2 基于硬件虚拟化扩展的 TEE 构建机制研究现状

主流指令集架构 (ISA) 均已引入成熟的硬件辅助虚拟化技术, 通过提供标准化的特权级扩展, 使 Hypervisor 能够有效屏蔽底层异构硬件的差异, 实现统一的虚拟机管理与资源调度。相比于逻辑复杂的通用操作系统, Hypervisor 具备更精简的架构设计, 能够显著压缩可信计算基 (TCB) 的攻击面。因此, 利用 Hypervisor 构建安全隔离域以实现可信执行环境 (TEE), 已成为机密计算领域的重要研究方

向。

在学术界，针对传统硬件动态信任根（DRTM）机制因频繁触发硬件重置导致的巨额性能开销，以及宏内核 TCB 过于庞大难以验证等挑战，McCune 等人^[24]构建了专用轻量化 Hypervisor——TrustVisor。该方案通过软件模拟实现了度量信任根与微型软件 TPM，在仅约 6KLoC 的极小特权内核保障下，利用二级页表（NPT）将安全应用（PAL）的内存空间与不可信组件强制隔离，在不依赖特定硬件微码的前提下，以极低的性能开销为应用程序提供了隔离执行环境。

为进一步优化 Hypervisor 自身的稳健性，Steinberg 等人^[25]借鉴微内核设计理念，提出了安全虚拟化架构 NOVA。该架构将资源管理、设备驱动及虚拟机监控逻辑移出特权内核，使内核态核心代码缩减至约 9KLoC；同时利用硬件虚拟化扩展，为每个虚拟机（VM）分配独立且运行于用户态的 VMM 实例，确保了单一组件的失效不会威胁全局安全。在兼容性研究方面，Jia 等人^[26]针对 Intel SGX 等专有硬件 TEE 的闭源依赖问题，提出了基于通用虚拟化技术的 HyperEnclave 架构。其核心贡献在于通过软件定义的方式实现了与 SGX 兼容的 Enclave 抽象，成功将 TEE 保护能力与特定硬件微码解耦，并采用内存安全语言 Rust 构建轻量级监视器 RustMonitor，从语言特性层面进一步消减了 TCB 的攻击面。

针对 Arm TrustZone 架构因软硬件耦合度过高导致的潜在攻击风险，Pereira 等人^[27]利用静态分区 Hypervisor 构建了 Bao-Enclave。该方案通过硬件辅助虚拟化技术将安全功能从庞大的特权操作系统中剥离，利用空间分区机制为可信应用（TA）提供强隔离运行环境，有效防御了针对常规 TEE 实现的架构性攻击，并在保持极小 TCB 的同时实现了对硬件资源的精细化访问控制。

综上所述，基于虚拟化的 TEE 方案凭借硬件平台的广泛兼容性，能够在不依赖特定安全微码的前提下，实现 TA 与不可信组件间的物理级隔离，并显著降低安全保护带来的性能损耗。然而，尽管 Hypervisor 架构已极度精简，其作为特权层软件仍面临复杂的侧信道分析及部分针对性硬件攻击的威胁，这促使研究者进一步探索软硬协同的深度防护机制。

1.2.3 基于指令集安全特性的架构级 TEE 研究现状

纯虚拟化 TEE 方案虽能提供基础的内存隔离，但仍面临软件 TCB 冗余及难以防御物理层硬件攻击等局限。针对高安全需求场景，硬件厂商引入了具备内存加密与硬核隔离能力的专有扩展。

1.2.3.1 面向 x86 架构的硬件机密计算关键技术研究

Intel SGX (Software Guard Extensions) [3] 作为一种具有代表性的硬件扩展技术,其核心贡献在于定义了极为严苛的威胁模型:该模型假定包括操作系统与超监视器在内的所有系统软件栈均不可信,仅将硬件 CPU 与受保护的可信应用 (Trust Application, TA) 纳入可信计算基 (Trust Computing Base, TCB)。SGX 通过指令集扩展实现了名为飞地 (Enclave) 的硬件隔离执行环境,支持在开放的云环境下进行远程机密计算。其交互机制依赖于特定的硬件指令:利用 ECALL 发起入界请求以进入受保护空间,并经由 OCALL 返回执行结果。

然而,SGX 在提供高强度隔离的同时,亦面临显著的技术局限性。首先,由于 SGX 将 TA 视作 TCB 的关键组成部分,而传统 TA 多采用 C/C++ 等非内存安全语言构建,导致其易受到缓冲区溢出、释放后使用 (Use-after-Free) 等内存破坏攻击的威胁。针对此问题,Wang 等人[11] 提出了 Rust-SGX 框架,通过将内存安全的 Rust 语言引入 SGX SDK,利用其所有权机制在编译期消除内存隐患,有效规避了高昂的运行时安全检查开销。其次,SGX 采用基于进程的隔离范式,受限于受限的受保护内存 (EPC) 容量及复杂的编程模型,现有应用往往难以平滑迁移。特别是对于计算资源需求极高的深度学习模型,SGX 在可扩展性与开发成本方面面临巨大挑战。

为了降低开发难度,CPU 厂商尝试引入基于 CVM 抽象的 enclave,降低用户开发成本。AMD SEV[4] 是基于 AMD 硬件虚拟化拓展的拓展,通过为每个 VM 指定唯一的运行时标识符 ASID,该 ID 在 VM 整个生命周期中均存储在内存中不发生变化,确保每个虚拟机仅能访问当前 ASID 关联的系统资源。同时借助 AES 加密引擎,使用当前 ASID 关联密钥对所有读写内存操作进行加解密,其他 VM 或 hypervisor 仅能访问加密后的数据,确保了被保护数据的机密性。

为破解进程级 TEE 的灵活性瓶颈,硬件厂商相继推出了基于机密虚拟机 (Confidential Virtual Machine, CVM) 抽象的硬件方案,旨在降低开发门槛并实现对既有软件生态的兼容。AMD SEV[4] (Secure Encrypted Virtualization) 利用虚拟化扩展技术,为每个虚拟机分配唯一的地址空间标识符 (ASID),并结合硬件级 AES 加密引擎,确保 Guest VM 的内存数据在整个生命周期中均以密文形式存储。此举实现了 Guest VM 与 Hypervisor 之间的语义脱钩,有效保障了租户数据的机密性。

随后,针对 SEV 难以抵御的内存重放及映射欺诈等攻击,AMD 推出了增强版 SEV-SNP[28] (Secure Nested Paging) 协议。该方案引入了反向映射表 (Reverse Map Table, RMP) 机制,通过建立物理页面与归属 VM 之间的强绑定关系,强制性防止 Hypervisor 篡改内存页面的访问权限。AMD

与此同时, Intel 提出了 TDX^[29] (Trust Domain Extensions) 架构, 将 CVM 定义为独立的可信域 (Trust Domain, TD)。TDX 引入了全新的特权层级 SEAM (Secure Arbitration Mode) 及专用的 TDX Module, 从而在架构层面剥离了不可信 Hypervisor 对 TD 生命周期的控制权。与 SEV-SNP 不同, TDX 在内存访问控制上采用了更为直接的硬件标记位 (TD bit) 机制, 任何未经授权的特权级对机密内存的非法访问均会触发硬件级阻断。

综上所述, x86 架构下的 TEE 研究已从早期的进程级细粒度隔离转向兼顾性能与生态的虚拟机级保护, 通过软硬协同的深度演进, 逐步确立了现代云计算环境下机密计算的技术底座。

1.2.3.2 基于 ARM 架构的安全扩展与 TEE 隔离技术研究

与 x86 架构主导服务器及高性能计算领域的格局不同, ARM 架构凭借其卓越的能效比在嵌入式及移动终端市场占据绝对主导地位。因此, 该场景下的机密计算研究主要围绕 ARM TrustZone 技术及 GlobalPlatform (GP) 标准展开。

针对信息物理系统 (CPS) 在富执行环境 (Rich Execution Environment, REE) 受损时面临的实时性挑战, Wang 等人^[30] 提出了 RT-TEE 架构。该研究利用 TrustZone 强制保障系统的可用性, 通过引入策略驱动的事件分层调度器与硬件安全定时器, 并结合驱动精简技术极大地压缩了可信计算基 (TCB) 的规模, 从而确保关键控制逻辑在遭遇拒绝服务 (Deny of Service, DoS) 攻击时仍能保持时限确定性。

为突破 TrustZone 原生架构中安全世界 (Secure World) 资源独占且难以动态扩展的局限, Brasser 等人^[31] 构建了 SANCTUARY 架构。该方案通过巧妙利用地址空间硬件隔离原语, 在不引入额外虚拟化开销的前提下实现了用户态 Enclave 范式, 支持在非安全侧动态划分出受硬件保护的执行环境, 有效抵御了针对敏感数据的侧信道分析与内存探测攻击。与此同时, Zhao 等人^[32] 提出了 SecTEE 方案, 旨在 ARM 平台上模拟类 Intel SGX 的 Enclave 抽象。通过将细粒度软件监测与硬件隔离机制相结合, SecTEE 成功降低了 TEE 开发的准入门槛, 提升了移动平台机密计算的可扩展性。

在面向云端多租户的复杂场景中, 针对 TrustZone 缺乏多安全世界隔离能力的架构缺陷, Hua 等人^[33] 研发了 vTZ 系统。该方案在最高特权级引入受信任管理模块, 利用硬件虚拟化扩展 (VE) 与 TrustZone 隔离原语, 实现了安全世界的逻辑隔离与时分复用, 确保了不同租户的机密虚拟机 (CVM) 在共享硬件资源时具备互不干扰的隔离强度。

此外, 针对 TEE 内部 TA 因使用 C/C++ 等非内存安全语言而引入的内存漏洞风险, RusTEE^[12] 提出了一套基于 Rust 语言的框架。该方案通过扩展 Rust 编译

器以适配 TrustZone 特殊环境，并设计了受信任的系统调用与跨世界通信（World Switching）接口，利用 Rust 的所有权与借用检查机制在编译阶段静态消除内存破坏隐患，显著增强了 TEE 软件栈的稳健性。

1.2.3.3 基于 RISC-V 开放架构的可定制 TEE 研究现状

作为新兴的开源指令集架构（ISA），RISC-V 凭借其高度的可扩展性为学术界探索面向服务器及嵌入式场景的定制化 TEE 方案提供了理想平台。

Dong 等人^[5, 34, 35]针对现代机密计算在无服务（Serverless）及大规模云场景下遭遇的扩展性与性能瓶颈，提出了一套从内存隔离到 I/O 安全的 RISC-V 软硬协同优化方案。为解决安全内存初始化滞后及容量受限问题，Penglai-sPMP 通过引入受护页表（Guarded Page Table, GPT）与挂载式默克尔树（Mountable Merkle Tree, MMT），实现了高达 512GB 的动态隔离空间支持；在此基础上，Penglai-HPMP 采用混合物理内存保护策略，利用段寄存器加速页表遍历（Page Walk），显著降低了多维权限验证带来的性能损耗；而 Penglai-sIOPMP 则通过多级树状检查器构建了高可扩展的 I/O 隔离机制，在有效防御恶意 DMA 攻击的同时提升了异构计算下的网络吞吐量。

针对现有商业 CPU 厂商 TEE 方案缺乏灵活性且难以跨硬件定制的局限，Dayeol 等人^[6]提出了首个基于 RISC-V 的开源机密计算框架 Keystone。该框架在最高特权级引入可编程的安全监视器（Security Monitor），通过对 TEE 依赖的硬件原语进行抽象封装，实现了核心功能的模块化与可重用性。Keystone 允许开发者根据具体应用场景动态权衡 TCB 规模与功能集合，在不修改底层硬件的前提下，为从低功耗传感器到云服务器的多样化设备提供可定制的受信任执行环境。此外，针对商用 TEE 难以通过高等级安全认证的挑战，Ozga 等人^[36]提出了一种基于 RISC-V 的形式化验证框架，通过对安全监视器逻辑进行数学证明，确保了 TEE 可信根的确定性，填补了机密计算在形式化验证领域的通用性空白。

综合上述研究可知，传统的 x86 架构 TEE 方案受限于指令集的复杂性，其开发成本高昂且难以适配下游厂商的灵活定制需求；ARM 架构虽具备一定灵活性，但在支持多安全世界等复杂场景时仍存在架构级缺陷。更为关键的是，x86 与 ARM 作为闭源指令集，其 TEE 实现方案严格受制于特定的 CPU 供应商。相比之下，RISC-V 作为可扩展的开源指令集，不仅能有效规避闭源架构导致的准入门槛过高问题，更赋予了用户根据特定安全需求与硬件约束进行深度定制的能力，为构建下一代高性能、高可信的计算体系奠定了基础。

1.2.4 异构计算环境下 GPU-TEE 安全隔离与防护技术研究

随着端侧人工智能与云端大模型业务的爆发式增长，保障异构计算平台（如 CPU+GPU/NPU）的数据隐私与模型完整性已成为系统安全领域的研究热点。在架构实现上，高性能服务器场景通常采用独立显存的离散式 GPU，其受信任执行环境（TEE）仅需隔离 GPU 的内存映射 I/O（MMIO）区域；而在端侧场景中，集成式硬件加速器通常与 CPU 共享物理内存，因此不仅需隔离 MMIO，更要求对 GPU 缓冲区实施动态物理隔离。为在不改变底层硬件设计的前提下构建 GPU TEE，学术界主要探索了基于 CPU 安全特性监控与软硬协同原语防护两条技术路径。

在基于 CPU 安全特性的监控机制研究中，Wu 等人^[37]提出的 GEVisor 利用现有 Intel SGX 构建了经过形式化验证的安全参考监视器（SRM），在 Enclave 与 GPU 间建立可信 I/O 路径，有效消除了系统对不可信驱动的依赖。面向端侧统一内存架构，Deng 等人^[38]在 StrongBox 中利用 ARM TrustZone 技术，通过动态拦截 MMIO 与二阶段页表控制实现了高效隔离，其可信计算基（TCB）仅约 1.4K LoC。为突破传统 TEE 因频繁数据加解密导致的性能瓶颈，Fan 等人^[39]提出的 XpuTEE 框架在 VMX Root 模式下引入微型监控器 XM，通过联合抽象与直接物理映射实现了“去加密化”监控，将大模型推理开销显著压缩至 2.5% 以下。

为应对新一代指令集架构与存量基础设施的兼容性挑战，机密计算研究逐渐转向软硬协同的深度解耦范式。针对 Arm CCA 架构缺乏加速器支持的局限，Wang 等人^[40]提出的 CAGE 框架在根世界（Root World）部署安全监控模块，借助粒度保护检查（GPC）原语率先实现了 Armv9 架构下的 GPU 硬件级隔离；其首创的影子任务（Shadow Task）机制在维持不可信宿主驱动逻辑的同时，将 TCB 增量严格控制在 1.3K LoC 以内。此外，针对现有机密加速器高度依赖特定硬件更迭的问题，Wang 等人^[41]设计了具备强兼容性的 ccAI 系统。该方案通过在物理层部署轻量级监视器动态截获并重定向 GPU 任务提交路径，从架构层面完成了 TEE 与不可信驱动的深度解耦，能以极低性能损耗平滑支持 DeepSeek、Llama 等大模型的机密推理，为异构 AI 算力的泛化安全部署提供了高效路径。

尽管异构隔离技术不断演进，Lu 等人^[42]在 MOLE 研究中揭示了新型硬件侧攻击面：GPU 内部集成的微控制器（MCU，如 Falcon/GSP）拥有超越系统 IOMMU 限制的特权级访问能力。攻击者可通过劫持 MCU 固件绕过 CPU 侧所有监控机制，直接窃取 TEE 显存数据。该发现打破了“直通即安全”的传统假设，表明未来的异构机密计算不仅需要系统级外部隔离，更亟须建立涵盖外设内部组件完整性度量的全路径防护体系。

1.3 本文主要贡献和创新

本文以基于 RISC-V 硬件虚拟化扩展及 CVM 安全生命周期管理为核心研究对象。依托既有的基于微内核的 EasyAda-Hypervisor，本文重点开展了底层安全固件设计、虚拟化组件安全扩展及端侧异构机密计算环境的构建，其主要研究内容与创新工作如下：

1. **提出并实现了一种基于 Rust 内存安全特性的模块化底层安全监视器 (EasyAda-SM)：**针对传统 C 语言构建的安全固件易受内存破坏漏洞威胁的局限性，本文采用 Rust 语言从底层独立设计并实现了 EasyAda-SM。在架构抽象中，构建了安全区域与内存管理层，深度结合 Rust 的所有权模型实现了一套基于页分配器 (Page Allocator) 的安全内存调度机制，从编译期静态消除了二次释放等内存隐患。同时，将物理内存保护 (PMP) 机制抽象为统一的内存保护原语，实现了不可信组件与 CVM 安全内存间的物理级强隔离，为整个系统确立了坚实的信任根基。

2. **对基于微内核的 EasyAda-Hypervisor 引入安全扩展以支持 CVM 全生命周期管理：**为适应机密计算的安全需求，本文对现有的 EasyAda-hypervisor 进行了深度的安全扩展与架构增强。通过引入职责解耦机制，将常规的物理内存分配与设备发现保留在 Hypervisor 层，而将二阶段页表隔离映射、安全区域动态构建及可信上下文切换等核心敏感操作委托给 EasyAda-SM 处理。结合设计的基于本地证明机制的安全启动链与机密执行流，确保了 CVM 的 CPU 上下文及内存视图对不可信 Hypervisor 的绝对隐匿，从而在不破坏原有架构的前提下，实现了不可信虚拟机与机密虚拟机的安全共存与统一调度。

3. **拓展 EasyAda-trust 架构并构建端侧机密异构计算环境：**针对端侧异构硬件中支持直接内存访问 (DMA) 的外设可能绕过 CPU 隔离机制发起的恶意攻击，本文进一步拓展了 EasyAda-trust 的系统设计。通过引入 RISC-V I/O 物理内存保护 (IOPMP) 扩展，实现了基于设备唯一标识 (SID) 的外设动态鉴权与细粒度内存访问控制。在此基础上，创新性地提出并构建了专用的“加速器管理 CVM”架构，利用 I/O 半虚拟化技术与安全共享内存映射机制，在严格保障用户 AI 模型权重及隐私数据机密性与完整性的前提下，成功实现了底层硬件加速器在不可信执行环境与机密执行环境间的时空复用，为端侧异构算力的泛化安全部署确立了完备的技术范式。

1.4 论文结构

本文的内容组织与结构安排如下：

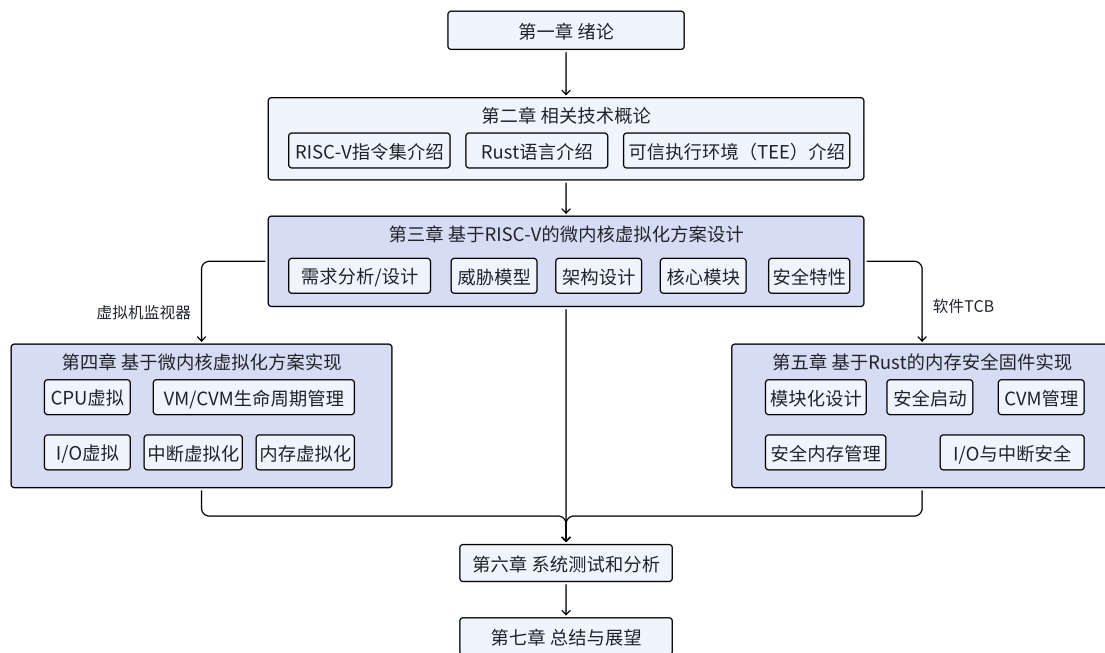


图 1-2 章节结构

第一章：绪论。本章聚焦智能物联网与端侧异构计算环境下的安全与实时性挑战，详细阐述了研究基于 RISC-V 架构的可信执行环境（TEE）与微内核虚拟化技术的背景、动机与核心目标。通过对现有微内核 Hypervisor 架构及主流 TEE 方案进行深度调研与对比分析，明确了本文的研究切入点，并系统梳理了本文的主要研究工作与组织结构。

第二章：相关技术理论概述。本章奠定了本文的理论与技术基础。首先从 RISC-V 指令集架构出发，深入剖析了硬件虚拟化扩展（H-extension）以及物理内存保护（PMP）、I/O 物理内存保护（IOPMP）等底层硬件安全原语。随后，系统梳理了机密计算与 TEE 的核心概念，对比了不同级别 Enclave 抽象的架构优劣势，并全面介绍了自研微内核操作系统 EasyAda 的设计哲学及虚拟化核心技术。

第三章：基于 RISC-V 的微内核虚拟化方案设计：本章系统阐述了支持 CVM 生命周期管理的微内核虚拟化架构 EasyAda-trust 的总体设计。首先，在深度剖析端侧高安全需求与威胁模型的基础上，确立了系统的设计目标与多方信任协作模型。其次，自顶向下地构建了清晰界定不可信组件与 TCB 的系统级隔离边界，并围绕安全启动及 CPU、内存、中断与 I/O 等多维虚拟化层面，深入论述了贯穿 CVM 全生命周期的底层防护机制。最后，针对端侧异构算力的安全保护需求，创新性地提出了专用加速器管理 CVM 架构，构建了基于 IOPMP 的安全外设隔离与基于共享内存的硬件加速器时分复用机制。

第四章：基于 Rust 的内存安全固件实现：本章详细论述了运行于最高特权级

的安全监视器 EasyAda-SM 的工程实现。首先，依托 Rust 语言的所有权机制在编译阶段静态消除了传统固件中典型的内存破坏漏洞。其次，抽象并构建了安全区域与内存管理层，设计了基于页分配器的精细化安全内存调度机制。随后，深入剖析了面向 CVM 的 Enclave 管理模块，系统阐述了基于 PMP 的强隔离原语、基于本地证明的安全启动链，以及机密执行流间的安全上下文切换路径。最后，针对端侧异构计算场景，实现了基于 IOPMP 的动态外设授权机制，构建了有效抵御恶意直接内存访问（DMA）攻击的系统安全底座。

第五章：基于微内核的可信虚拟化方案实现：

第六章：系统测试与分析：

第七章：总结与展望：

第二章 相关技术理论概论

2.1 RISC-V 背景知识

x86 架构因其漫长的发展历史与向后兼容性负担，指令集日趋复杂庞大。尽管在服务器等高性能计算领域表现优异，却难以契合嵌入式场景对严苛功耗与成本的约束。

ARM 架构作为精简指令集（RISC）的代表，凭借 IP 授权模式赋予下游厂商一定的定制自由度，在嵌入式市场占据主导地位。然而，受限于其封闭的商业壁垒，硬件开发面临高昂的 IP 授权成本；同时，ARM 架构跨代演进（如 ARMv7 至 ARMv8）差异显著，导致了严重的指令集碎片化问题，系统高度依赖特定版本的编译工具链以维持跨平台兼容性与性能优化。

相比之下，新兴的 RISC-V 架构凭借开源开放与高度模块化的设计理念，与系统软件的解耦演进需求高度契合。硬件厂商可根据具体场景按需组合指令集扩展，软件生态亦能据此实现灵活伸缩，有效规避了跨代架构更迭带来的高昂代码移植成本。在云边端协同计算趋势下，资源受限的边缘节点可仅实现基础指令子集以满足极低功耗约束，而云端高性能设备则可集成复杂扩展以跃升算力。RISC-V 这种高度弹性的架构特性，为构建跨越异构硬件的通用、可扩展可信执行环境（TEE）提供了理想的底层硬件支撑

2.1.1 RISC-V 特权级介绍

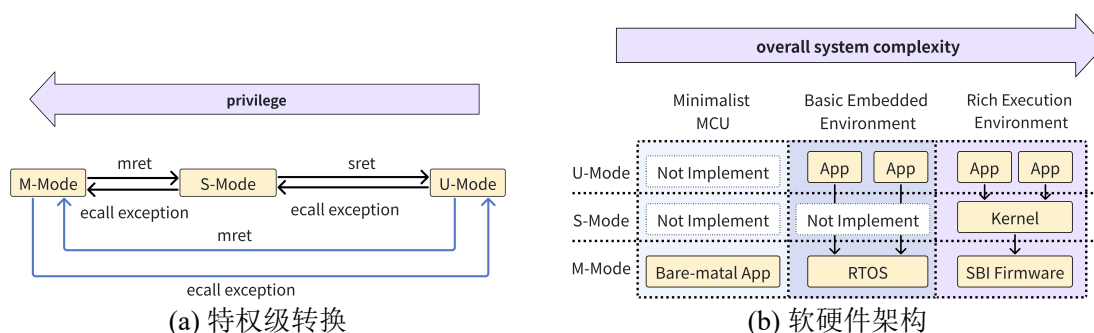


图 2-1 未引入硬件虚拟化拓展时 RISC-V 系统架构

RISC-V 架构将独立的 CPU 核心抽象为硬件线程（Hard Thread, hart）。在任何运行时态下，hart 均处于特定的特权级中。RISC-V 规范定义了由低至高的三种主要特权级：用户模式（U-Mode）、监管者模式（S-Mode）与机器模式（M-Mode）。如图2-1(b)所示，根据指令集标准，M 态为强制实现的基础特权级，而 U 态与 S 态

为可选扩展。实际商用硬件通常基于特定应用场景与功耗考量进行架构裁剪，如运行 RTOS 或裸机应用的 MCU 场景可以实现 U 和 M 态甚至仅实现 M 态。

各特权级之间具备严格的权限隔离边界^[43]：U-Mode 作为最低特权级，仅获准访问通用寄存器并执行基础指令，其访存操作严格受限于页表映射机制；S-Mode 在此基础上，进一步获准访问特定的控制与状态寄存器（Control Status Register, CSR）并执行相关特权指令，其访存操作同样受限于 MMU 页表机制；M 态作为系统最高特权级，拥有对所有物理硬件、CSR 及特权指令的无限制访问权，并可绕过页表直接寻址物理内存。任何低特权级尝试执行越权指令的行为均会触发硬件异常拦截。

在特权级状态转换机制上，系统上电重置时 hart 默认初始化为 M 态。控制流从高特权级向低特权级转移需显式执行 mret 或 sret 指令完成状态降级；反之，当系统发生外部中断或执行异常时，硬件将根据 CSR 中的委派（Delegation）配置，自动陷入（Trap）至 S 态或 M 态的异常向量入口。此外，低特权级软件可主动触发 ecall 或 ebreak 指令发起系统调用，以同步陷入的方式将控制权安全移交至高特权级执行环境。各特权级间的状态转换逻辑如 2-1(a) 所示。

2.1.2 RISC-V 中断与异常处理机制介绍

RISC-V 架构将所有打破正常程序控制流的突发事件统称为陷入 (Trap)。在微架构层面，陷入被严格划分为两类：同步的异常 (Exception) 与异步的中断 (Interrupt)。异常通常由当前执行的指令直接引发 (如非法指令解码、缺页异常或主动发起的 ecall 系统调用)，而中断则由独立于当前执行流的外部异步事件触发 (如时钟中断、软件中断或外部设备 I/O 请求)。RISC-V 手册规定中断按类型分为外部中断 (External Interrupt)、软件中断 (Software Interrupt)、时钟中断 (Timer Interrupt) 三种。

根据 RISC-V 基础特权级规范，系统发生的所有陷入默认均强制路由至最高特权级 M 态处理。然而，在部署了复杂操作系统或 Hypervisor 的场景中，频繁的 M 态陷入与上下文切换将引入不可接受的性能开销。为此，RISC-V 创新性地设计了陷入委派机制。通过精细化配置 M 态异常委派寄存器 (medeleg) 与中断委派寄存器 (mideleg)，底层安全固件可将特定类型的陷入事件 (如 S 态下的缺页异常或时钟中断) 显式旁路，使其直接触发 S 态的陷入响应而无需 M 态介入。这一机制极大地缩短了关键中断的响应延迟，是实现高效软硬件协同与机密虚拟化资源调度的核心架构支撑。

2.1.3 RISC-V 内存保护机制介绍

2.1.3.1 物理内存保护（PMP）机制

物理内存保护（Physical Memory Protection, PMP）机制是 RISC-V 架构标准提供的核心安全原语，也是在该架构上构建 TEE 的必要底层硬件支撑。在微架构层面，每个 hart 均集成了一组专用的 PMP 地址寄存器（`pmpaddrx`）与配置寄存器（`pmpcfgx`），且这些寄存器的读写权限被严格限制在 M 态。

在使能了 PMP 机制的硬件平台中，系统默认采用严格的隔离访问策略：在未显式授权的情况下，运行在更低特权级的软件被默认限制对任何物理地址的访问。通过在 M 态下对 PMP 寄存器组进行精细化配置，系统能够以 hart 为粒度，为其划分特定的连续物理地址空间，并显式授予相应的读、写及执行权限。简言之，PMP 机制从硬件底层提供了一种强隔离的访问控制手段，能够精准限制特定 hart 对任意物理内存区域的访存边界与操作能力。

2.1.3.2 增强的物理内存保护（Smepmp）机制

Smepmp（Supervisor Mode Execution Prevention and Machine-mode Physical Memory Protection）扩展是对 RISC-V 基础 PMP 机制的关键安全增强。在原生 PMP 架构中，锁定位（L 位）的语义存在天然的耦合局限：当 L=0 时，M 态默认拥有对全量物理内存的无限制访问权；当 L=1 时，M 态与低特权级将被强制绑定相同的 R/W/X 权限。这种粗粒度的设计导致系统无法在赋予 S 态合法访存权限的同时有效剥夺 M 态的对应权限，难以防御针对高特权级固件的混淆代理（Confused Deputy）攻击，亦无法从架构层面彻底杜绝 M 态意外执行驻留于低特权级内存中的恶意载荷。

为突破上述架构瓶颈，Smepmp 扩展创新性地引入了机器安全配置寄存器（`mseccfg`）。该寄存器通过三个核心控制位重塑了底层的物理访存控制逻辑：首先是机器模式锁定（MML）位，当 MML 被置位时，系统将彻底重载传统 PMP 配置中 L 位与 R/W/X 位的组合语义，实现了 M 态与更低特权级内存权限的深度解耦。例如系统可配置仅低特权级可读写、M 态不可访问的隔离区域，从而在物理硬件层面严格贯彻了最小特权原则。其次是机器模式白名单策略（MMWP）位，使能该位将逆转 M 态的默认访存行为，从原有的默认放行切换为严苛的默认拒绝（Default-Deny），强制要求 M 态的所有访存操作必须命中显式授权的 PMP 条目。最后是规则锁定旁路（RLB）位，它在维持整体隔离强度的前提下，允许授权的安全固件动态修改已锁定的 PMP 条目，极大提升了 TEE 运行时的内存管理弹性。

综上所述，Smepmp 机制在微架构层面弥补了原生 PMP 权限颗粒度粗糙的设

计缺陷，而原生 PMP 机制以最小的硬件开销构筑了坚实的宏观物理隔离边界，为嵌入式环境提供了兼具高实时性与高性价比的底层安全原语。

2.1.3.3 I/O 物理内存保护 (IOPMP) 机制

输入输出物理内存保护 (Input/Output Physical Memory Protection, IOPMP) 是 RISC-V 架构中用于保障系统级外设访存安全的关键硬件扩展。尽管原生的 PMP 机制能够有效限制 hart 的越权访存行为，但它仅作用于 CPU 侧，无法约束具备 DMA 能力的外设。恶意或受控的外设可轻易绕过 CPU 的 PMP 隔离边界，直接对受保护的物理内存发起读取或篡改操作 (即 DMA 攻击)，这构成了机密计算环境下的重大安全隐患。

为弥补这一架构级安全短板，RISC-V 引入了 IOPMP 扩展。在微架构实现上，IOPMP 通常部署于系统互连总线与外设控制器之间，充当所有 I/O 访存事务的硬件级安全网关。其核心访问控制逻辑基于设备唯一标识 (Source ID, SID) 展开：当外设发起总线级读写请求时，IOPMP 会首先捕获该请求携带的 SID，并通过内部的路由表将其映射至特定的内存域 (Memory Domain, MD)。每个内存域内配置了一组类似于 PMP 的规则条目，用于精确界定该 SID 对应的设备所能访问的连续物理地址范围及其读写权限。任何未命中授权条目的 DMA 请求均会被 IOPMP 硬件直接拦截并上报异常。

2.1.4 RISC-V 硬件虚拟化拓展 (H-extension) 介绍

RISC-V 硬件虚拟化扩展 (H-extension) 是 RISC-V 指令集架构中专为提升 Hypervisor 运行效率而设计的关键硬件扩展。传统软件模拟的虚拟化方案面临高昂的陷入-模拟开销，而 H 拓展通过在处理器微架构层面引入新的特权级、寄存器副本以及二阶段地址翻译机制，从硬件底层为高效构建 Type-1 或 Type-2 型 Hypervisor 提供了系统级支撑。

如图2-2所示，使能 H 拓展后，原有的 S 态被拓展为 Hypervisor 扩展监管者模式 (HS-Mode)，同时新增了虚拟监管者模式 (VS-Mode) 与虚拟用户模式 (VU-Mode)。在 HS 态下运行的 Hypervisor 拥有对物理硬件与虚拟机的完全控制权；而 Guest OS 及 Guest App 则分别运行在 VS 态与 VU 态下。这种设计确保了 Guest OS 无需修改核心代码即可无感运行，并在试图执行影响全局的敏感特权指令时安全地触发硬件异常并陷入 HS 态处理。

频繁的虚拟机退出 (VM-Exit) 是制约虚拟化性能的核心瓶颈之一。为此，H 拓展为 VS 态引入了一套完整的 S 态寄存器影子副本 (如 vsstatus、vstvec、vsepc、vsscratch 等)。当处理器运行在虚拟化环境 (V=1) 时，Guest OS 对常规 S 态寄存器

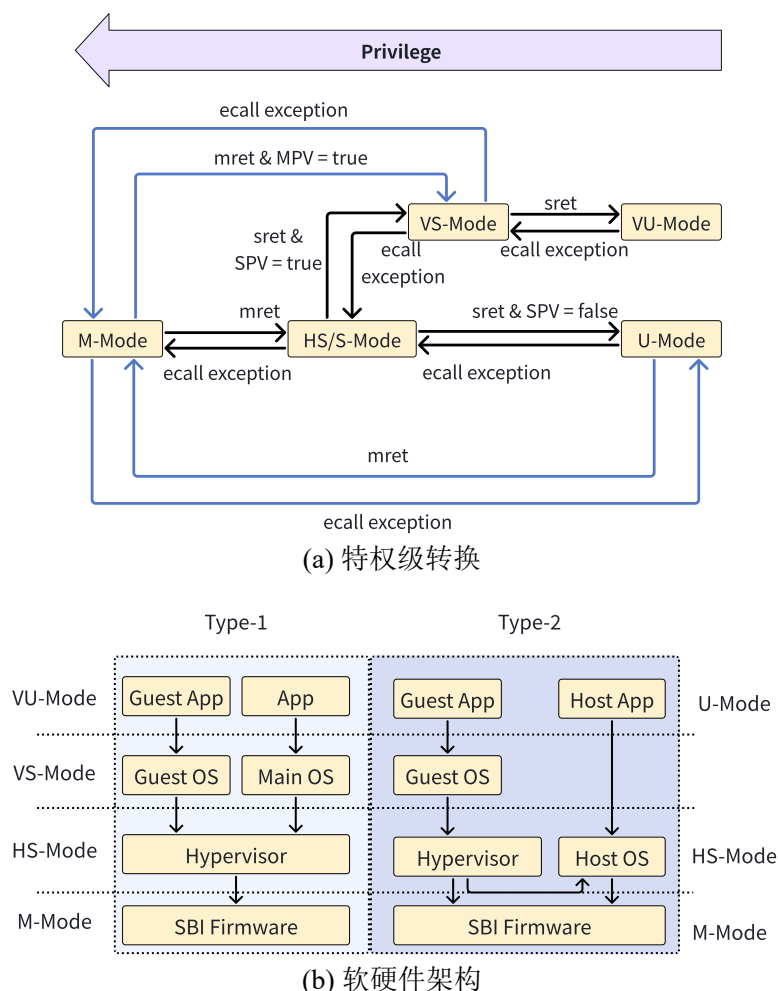


图 2-2 引入硬件虚拟化拓展后的 RISC-V 系统架构

的读写操作会被硬件自动重定向至对应的虚拟控制状态寄存器中。这一机制使得客户系统在配置自身上下文时无需陷入 HS 态，极大地降低了寄存器访问带来的上下文切换开销。

为实现虚拟机间的物理内存强隔离，H 拓展引入了二阶段页表机制。第一阶段 (Stage-1) 完全由 Guest OS 控制，负责将虚拟地址 (VA) 翻译为客户物理地址 (GPA)，其页表基址由虚拟监管者地址转换与保护寄存器 (vsatp) 管理；第二阶段 (Stage-2) 由 Hypervisor 严密管控，负责将 GPA 强制翻译为真实的宿主物理地址 (HPA)，其页表基址由 Hypervisor 客户地址转换与保护寄存器 (hvatp) 管理。MMU 在处理 VS/VU-Mode 的访存请求时，会自动串行执行这两阶翻译，彻底阻断了虚拟机越权访问宿主物理内存或其他虚拟机地址空间的可能。

同时，H 拓展增强了 RISC-V 原有的委派机制。通过配置 Hypervisor 级的中断与异常委派寄存器 (hideleg 与 hedeleg)，系统可将原本默认陷入 HS 态的特定异常 (如 VS 态系统调用或缺页异常) 直接旁路委派至 VS 态，交由 Guest OS 内部的处理。

理程序直接响应。此外，H 拓展设计了完善的虚拟中断注入机制 (如通过 hvip 寄存器配置 VSSIP、VSTIP、VSEIP)，允许 Hypervisor 在软件层面灵活模拟虚拟硬件外设，并高效地向目标 VS 态挂起并注入虚拟中断信号。

2.2 TEE 相关技术介绍

2.2.1 基于指令集的 TEE 分类

2.2.2 基于可信域抽象的 TEE 分类

2.2.3 面向异构计算环境的 TEE

- (1) **指令集拓展**: 该范式将 AI 运算逻辑直接嵌入 CPU 的执行流水线中，通过 RISC-V 预留的编码空间实现自定义指令扩展（如矩阵运算或向量增强）。加速单元共享处理器的通用寄存器组或向量寄存器，加速指令由 CPU 译码后直接分发至执行单元。此模式消除了数据在异构组件间的移动开销，具有极高的指令响应速度和极低的上下文切换延迟。它适用于细粒度、高频次的张量运算，是追求极致单核计算密度及实时性场景的首选理论模型。
- (2) **协处理器**: 此范式下，加速器作为 SoC 内部独立的硬件 IP 存在，通过片上总线与 CPU 交互。加速器拥有独立的控制逻辑与局部存储空间，由 CPU 通过 MMIO 进行任务配置和启动，并通过 DMA 自主完成大规模数据的搬运。该架构实现了计算压力从通用核心向专用硬件的有效卸载。通过异步工作机制，CPU 仅负责高层调度，而加速器专注于流水化处理大体量数据。这种模式在能效比与系统灵活性之间进行了平衡，是当前主流嵌入式 AI SoC 范式。
- (3) **片外加速器**: 加速器位于独立的物理芯片或插卡上，通过高速串行总线与主机互联。此类加速器通常具备独立的全局显存和复杂的层次化缓存结构。系统通过驱动程序管理设备内存空间，并采用命令队列模式进行大规模任务的批处理提交。其逻辑设计的复杂度不再受限于主机 SoC 的热设计功耗与封装面积，能够提供极高的峰值算力。然而，由于跨物理总线通信的存在，该范式对主机侧的安全隔离机制提出了更高的时延容忍度与带宽管理要求。

2.3 微内核相关知识介绍

2.3.1 微内核操作系统 EasyAda 介绍

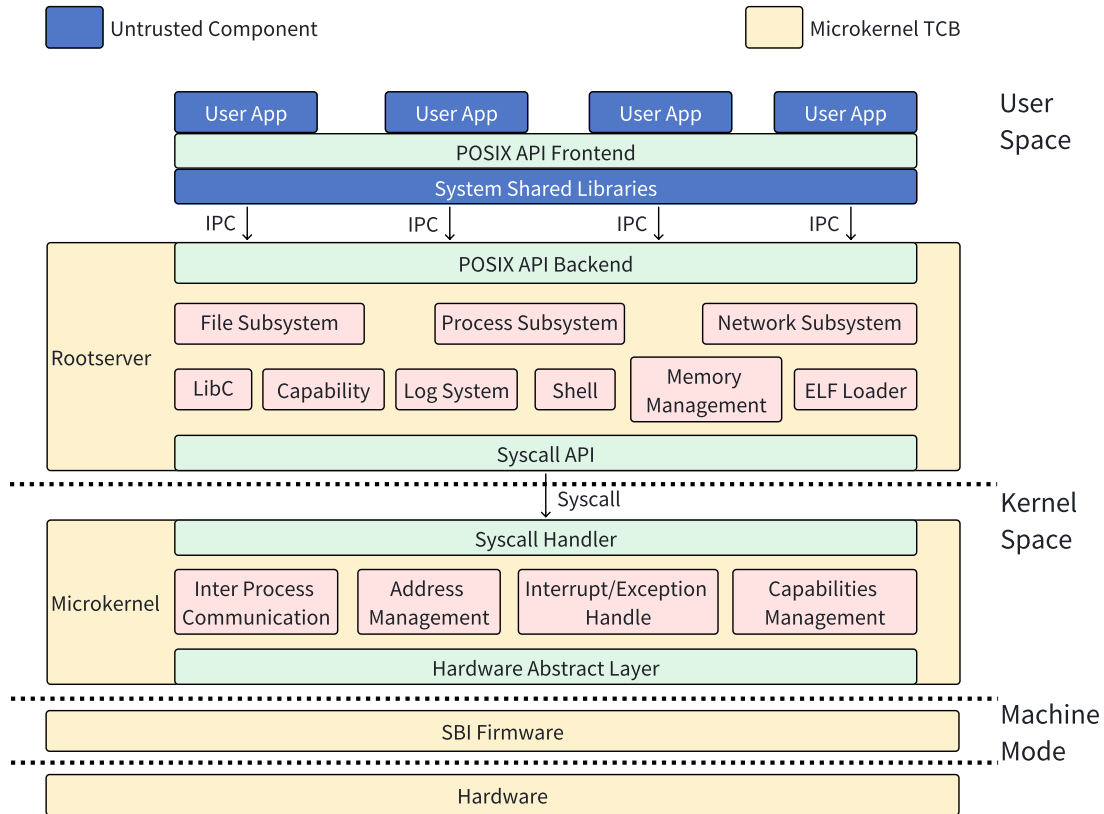


图 2-3 微内核操作系统 EasyAda 总体架构

EasyAda 是一款自研微内核操作系统，如图2-3所示，其设计理念遵循最小特权原则，旨在构建高安全性、高可靠性的操作系统内核，适用于对稳定性和安全性要求较高的嵌入式应用场景。

EasyAda 将内核态精简为仅保留必要的特权功能模块，如核间通信、权能管理、中断与异常处理、时钟管理、地址空间管理和内核对象管理等核心组件，内核将对外提供的服务封装为系统调用接口，仅暴露较小的攻击面。同时，通过硬件兼容层（Hardware Abstract Layer, HAL）抽象了操作系统依赖的硬件原语，如启动/暂停 CPU 核心，中断配置和处理，MMU 配置等，实现了统一的硬件操作接口，简化了操作系统在不同指令集和架构间移植的难度并且有效降低了内核代码的复杂度，确保 EasyAda 可以在异构环境上对外提供统一的软件平台。

EasyAda 将核心功能模块外的其余大部分功能模块迁移至用户态，在地址空间隔离的上下文中运行，并由根服务（Rootserver）统一管理。根服务作为一个运行在用户态的多线程进程，集成了多个功能子系统。这些子系统既包括独立的模

块化组件，如文件系统、进程管理、网络协议栈等，也包括执行特定任务的独立线程，如 Shell 交互模块等。根服务将对外提供的服务封装为 RPC 接口，为操作系统提供统一、透明的服务访问接口，并且对 RPC 参数进行严格的检查以防范外部攻击。其他服务或用户应用可以通过系统共享库提供的 POSIX API 前端，将函数调用转换为 RPC 调用。同时，所有应用和系统服务的拥有独立的权能空间，仅能访问当前权能空间支持的 RPC 接口和系统调用，进一步缩小系统攻击面。

基于微内核的设计思想，EasyAda 相比 Linux 等宏内核操作系统具备更小的软件 TCB，而相比传统的 RTOS 如 FreeRTOS 等，EasyAda 又具备强大的可拓展性，可以通过拓展系统服务实现与宏内核对等的功能和性能。因此 EasyAda 通过灵活调整用户态系统服务配置，可以适应边缘到中高端嵌入式等多样化的场景和需求，为云边端场景提供通用的 OS 底座。

因为 EasyAda 被设计为面向多样化场景的通用操作系统，所以其应用和系统服务运行在 U 态，内核运行在 S 态（未开启虚拟化）/HS 态（开启虚拟化支持），并且通过 SBI 请求与 M 态 SBI 固件进行交互以实现硬件管理。

2.3.2 微内核虚拟化方案 EasyAda-hypervisor

图2-4展示了支持管理 CVM 生命周期的基于 RISC-V 的 EasyAda-hypervisor 设计。EasyAda-hypervisor 仍然遵守了 EasyAda 微内核的设计哲学和思路：

- （1）**统一资源管理**：主要系统资源由 EasyAda 负责管理和分配。充分复用微内核操作系统现有代码和能力，实现统一的资源管理并减少 EasyAda-hypervisor 的代码量，缩小其 TCB。
- （2）**hypervisor 解耦**：hypervisor 被划分为运行在 EasyAda 用户态下的 VMM 和内核态下的 hypervisor 驱动，以尽可能复用 EasyAda 现有代码和功能。前者负责主要的 VM 管理工作，包括管理 VM 生命周期和软硬件资源，模拟虚拟化设备后端，以及管理 VM 服务线程等；后者作为屏蔽不同架构硬件虚拟化拓展实现的抽象层，对外提供基于虚拟化依赖的硬件原语抽象的统一接口，完成对硬件虚拟化拓展相关寄存器进行配置，以及接收和处理 VM 中断和异常等。
- （3）**静态分区**：hypervisor 采用静态分区策略，虽然该策略缺乏灵活性，但是因为嵌入式场景应用高度定制化的特点，静态分区策略足够满足其功能需求。并且静态分区策略可以简化代码逻辑，缩小 hypervisor 的攻击面，这一点在引入 CVM 支持之后更加重要，因为标准 RISC-V CoVE 拓展要求 SM 实现数十个 SBI 接口以实现 CVM 资源的细粒度控制，显著增大了 TCB 的攻击面，并且导致 hyperivisor 需要数个 SBI 调用才能完成一个

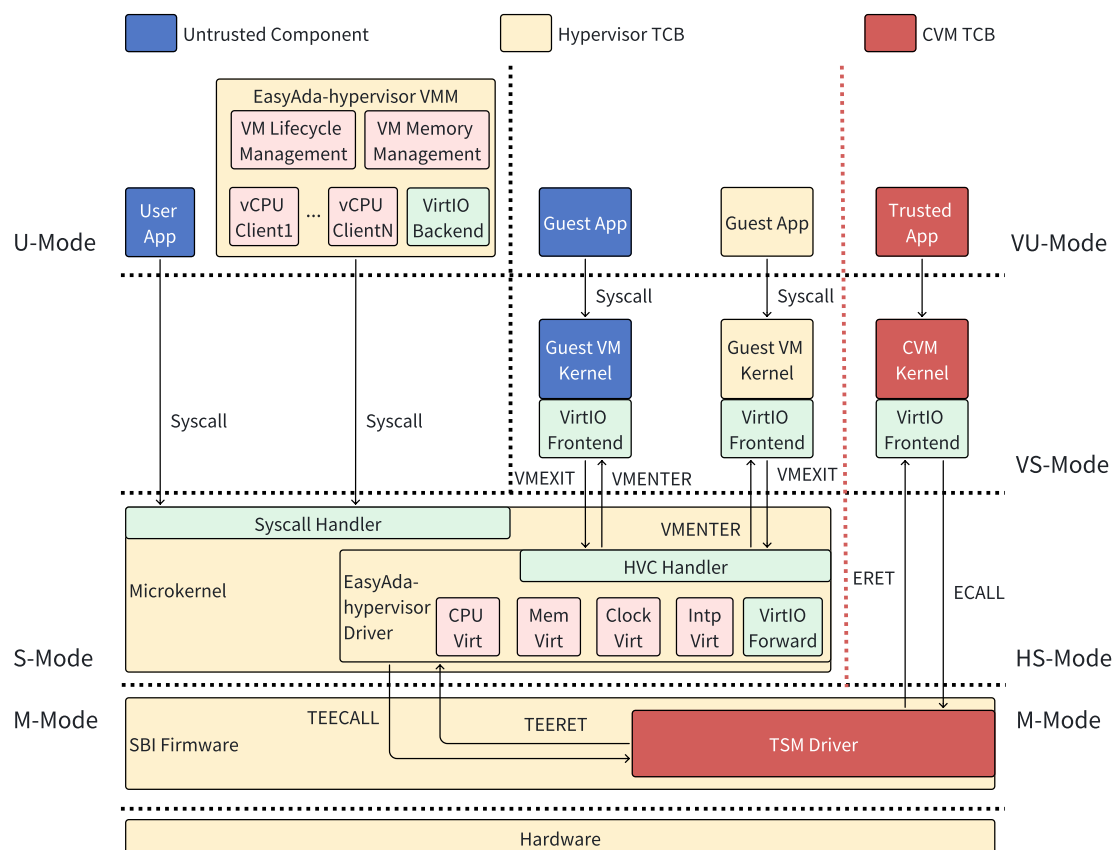


图 2-4 EasyAda-hypervisor 总体架构

功能，严重影响了系统的实时性和性能。

EasyAda-hypervisor 通过在 EasyAda 中引入两个核心组件实现基于硬件虚拟化拓展的 VM 生命周期管理：（1）运行在内核态（HS 态）的 hypervisor 驱动，负责抽象虚拟化硬件原语并对上提供虚拟化能力接口。其虚拟化能力主要可以分为以下五类：（1）**CPU 虚拟化**：目标是基于虚拟化专用特权级（RISC-V 的 VS/VU/HS 态），为 VM 提供符合其运行要求数量的 vhart 并在隔离的硬件环境中运行 VM 上下文，在实现操作系统无感迁移到虚拟化环境中运行的同时，避免 VM 访问和篡改 hypervisor 或其他 VM 的上下文信息；（2）**内存虚拟化**：目标是基于硬件虚拟化提供的二阶段地址翻译能力，通过动态配置和管理 VM 的二阶段页表，实现细粒度的，VM 无感的强制内存隔离；（3）**中断虚拟化**：目标是基于硬件提供的中断控制器（包括本地中断控制器和平台中断控制器等），通过中断注入或中断直通等手段，实现 VM 中断管理的同时，避免 VM 接收和配置不属于当前 VM 的中断；（4）**时钟虚拟化**：目标是基于虚拟硬件时钟，通过虚拟化计时器和虚拟时钟补偿计时器，实现 VM 无感退出/恢复；（5）**I/O 虚拟化**：目标是基于共享物理内存和虚拟中断，通过为不同 VM 模拟虚拟设备，在保证 VM 无感迁移的同时，提升 VM 的

I/O 性能和安全性。

用户态 VMM 主要包括两个部分：VM 资源和生命周期管理模块和 vhart 服务线程，负责提供完整的虚拟机管理与运行支持。VM 资源和生命周期管理模块负责虚拟机的生命周期管理，包括创建、运行、暂停、恢复及删除虚拟机等功能，并且通过 Shell 和 RPC 接口对外提供交互式接口，简化用户对虚拟机实例的管理。

vhart 服务线程可以视为 vhart 的调度实体和 VM 在 host 侧的用户接口，作为 VirtIO 后端负责响应 VM 的 I/O 请求。以实现 I/O 虚拟化和中断处理的高效执行。vhart 服务线程运行于 VMM 进程上下文中，与 VMM 及内核态虚拟化组件协同工作，优化虚拟机的计算与 I/O 任务调度。

2.4 Rust 相关知识介绍

第三章 基于 RISC-V 的微内核虚拟化方案设计

3.1 设计目标和需求分析

根据前文的介绍，基于 RISC-V 的微内核虚拟化方案——EasyAda-trust 主要面向支持硬件虚拟化的中高端嵌入式场景，设计目标为在提供可信计算能力，在 CPU 侧和 I/O 侧保护用户隐私代码和数据的同时，尽可能降低因为内存隔离等保护机制引入的额外性能开销，以及降低 TEE 的开发难度和准入成本，降低端侧 AI 模型等安全应用供应商的开发成本和难度。

基于以上设计目标，本文认为 EasyAda-trust 应当满足以下需求。

1. **安全性：**安全性包括功能安全和数据安全两个方面，从功能上来说，整个虚拟化方案中上层代码中出现的漏洞或不安全行为不应影响更高特权级软硬件的状态，同时单一应用或 VM 的崩溃不应扩散影响到其它同层应用或 VM；从数据保护的角度，每一个软件实体应当遵循最小权限原则，能且仅能访问当前实体所依赖的资源和服务等，即使是更高特权级的软件也不应当拥有无限访问上层数据和代码的权力。

2. **可拓展性：**可拓展性同样分为硬件和软件两个方面。从硬件角度，虚拟化方案应当支持用户根据需求引入和整合定制化的硬件，或者引入更高级的指令集拓展（如向量计算拓展等）；从软件角度，虚拟化方案应当支持灵活的部署方案，并且允许用户开发定制化应用。

3. **硬件兼容性：**虚拟化方案应当实现对现有商用硬件的广泛兼容性，并且能够向前兼容更先进的硬件平台。

4. **易用性：**虚拟化方案应当兼容主流基础软件以及相关工具链，包括普通 VM 和 CVM，降低开发人员开发难度。

3.2 EasyAda-trust workflow

在进一步讨论 EasyAda-trust 的设计之前，方便说明其威胁模型，需要先定义 EasyAda-trust 的开发流程和其中涉及的角色。EasyAda-trust 根据嵌入式场景的需求简化了生命周期中包含的角色并将其分为四大类：

- (1) **芯片 IP 供应商：**负责芯片 IP 设计。
- (2) **硬件供应商：**完成硬件平台设计并根据芯片 IP 和硬件平台设计完成硬件制造，采购以及最终的组装，并对上提供 RoT，硬件密钥对和 SM 配置和实现等。

(3) **平台供应商**: 负责所有基础软件如操作系统, hypervisor, 的研发和配置。

(4) **平台用户**: 基于平台供应商提供的软硬件生态和相关 SDK, 根据业务场景开发部署定制化的 VM/CVM 和应用。

以典型的智能驾驶场景为例, ARM 等公司负责设计芯片 IP, 高通/联发科等公司基于芯片 IP 完成硬件平台的设计, 制造和组装并提供 RoT 和 SM。智驾供应商则负责批量采购硬件平台, 并基于硬件平台完成开发 EasyAda-trust 的基础组件, 如微内核操作系统, hypervisor 和 SDK 等, 对上层提供统一的开发平台; 上层业务部门则可以根据不同汽车厂商的需求选择不同的 EasyAda-trust 配置和实例化方案, 并基于 SDK 二次开发 VM/CVM 以及其上的应用 (这一阶段中可能还有第三方开发人员开发的侧载应用)。最终作为一个统一的智能驾驶系统对驾驶人员提供服务。在实际场景中, 多个角色可能由同一实体或组织承担。

3.3 威胁模型

根据上一节对 EasyAda-trust 全生命周期中的角色的分析, 其威胁模型如下所示。硬件层面, 不考虑供应链攻击, 本文假定方案运行的硬件环境实现符合标准且无缺陷; 系统上下文切换时, 所有微架构层的运行轨迹均被正确清除。

软件层面, 本文假定软件层攻击者会尝试攻陷受保护的 CVM, 其攻击方式可以分为 (1) 篡改 CVM 的控制流或数据流执行逻辑, 破坏其运行完整性; (2) 窃取 CVM 中受保护的数据; (3) 冒充被攻击的 CVM 向其所有者发起交互。

因为嵌入式系统的特性, 基础软件通常是可信的, 不会主动尝试攻击其他软件实体以及窃取和篡改用户隐私数据, 但是可能存在会被恶意应用的代码漏洞; 普通 VM 上可能运行不可信的第三方侧载应用, 并且普通 VM, 微内核操作系统和 hypervisor 的根权限可能因为代码漏洞被利用或受到网络攻击而被完全控制; 同时, 本文认为运行在最高特权级的 SM 可信但是同样可能存在代码漏洞的。

同时, 攻击者可能尝试运行恶意的 CVM, 还可通过 hypervisor 管控的寄存器、共享内存缓冲区、虚拟输入输出设备, 或 CVM 初始镜像状态、扁平设备树 (Flattened Device Tree, FDT) 等, 对被攻击 CVM 的全生命周期及输入数据实现完全控制。

本文的威胁模型排除针对内存、总线及处理器的物理攻击。因为已有成熟的防护方案可应对此类攻击, 例如锁步故障检测机制、基于纠错码的内存保护、总线加密等。针对面临物理攻击风险的目标系统, 可单独部署上述技术而无需对 EasyAda-trust 的实现进行修改。

3.4 EasyAda-trust 详细设计

3.4.1 总体设计

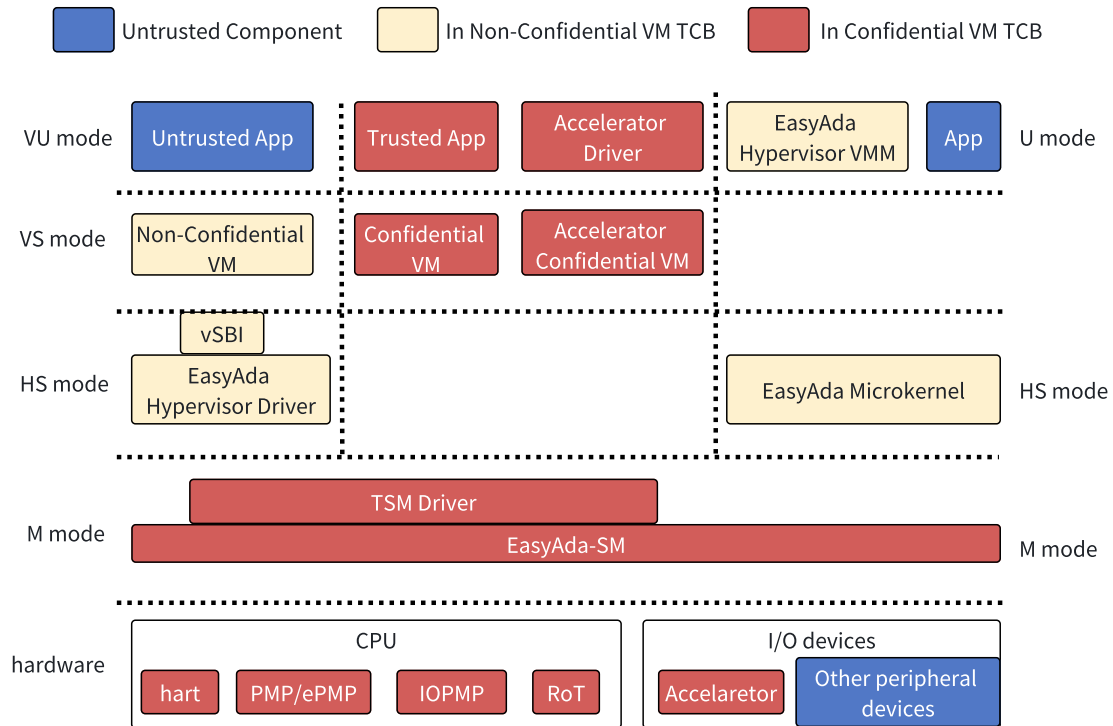


图 3-1 基于 RISC-V 的微内核虚拟化方案总体架构

如图3-1所示，如果将整个 EasyAda-trust 视为两层 TEE 的组合，则可以将其划分为以下三个层级：

1. **不可信应用（蓝色部分）：**不可信应用包括第三方开发者开发的侧载应用，以及由供应商直接开发，但是存在安全隐患，可能被利用的应用。是终端用户主要使用的软件实体。

2. **非机密虚拟机 TCB（黄色部分）：**黄色部分主要是不存在恶意但是可能被攻破的软件组件（为了方便叙述，后文和蓝色部分统一称为不可信组件），包括不可信的 VM，微内核操作系统 EasyAda 以及微内核 hypervisor——EasyAda-hypervisor。不可信 VM 负责为上层应用提供统一的基础软件，降低应用开发成本和难度；EasyAda-hypervisor 负责管理所有 VM 的资源和生命周期，对不可信 VM 提供虚拟 SBI 固件实现，并复用 EasyAda 现有功能以实现统一的资源管理并减少代码量；EasyAda 是整个系统的核心，负责管理和调度所有的硬件资源，并且允许对性能和安全性要求更高的应用直接部署在 EasyAda 上以避免虚拟化对性能和实时性的负面影响。

3. 机密虚拟机 TCB（红色部分）：红色部分是可信且不可被攻破的组件，包括运行在最高特权级的安全监视器 EasyAda-SM，实现在 SM 中的 TEE 安全管理驱动（TEE Security Manager Driver, TSM Driver），运行在相互隔离的运行环境（EasyAda-trust 称之为 enclave）中的 CVM 以及其上运行的各类 TA。EasyAda-SM（如无特殊说明，后文以 SM 代指）是隔离不可信组件和 CVM 的核心，EasyAda-hypervisor 只能通过 TSM Driver 提供的接口实现对 CVM 生命周期和资源的管理。CVM 对其上的 TA 提供了统一的开发平台，尤其是可以复用现有 AI 模型的开发框架和运行时依赖，开发者无需针对 TEE 环境开发专有的安全 AI 模型。

EasyAda-trust 为了兼顾的兼容性和安全性，要求硬件环境至少应当实现 **RISC-V 硬件虚拟化拓展——H 拓展和物理内存保护机制（PMP）**。边缘用户可以在一个相对轻量化和精简的硬件环境上运行 EasyAda-trust，而需要更高性能或依赖定制化硬件的场景则可以引入增强计算能力的指令拓展和定制化硬件。

第一层 TEE 以 EasyAda 和 EasyAda-hypervisor 作为核心软件 TCB，目标是通过 hypervisor 在不可信 VM 间提供强制内存和 I/O 资源隔离，确保 VM 内代码和数据的安全，并且借助微内核高安全的特性，尽可能缩小 TCB 的攻击面，避免不可信应用通过攻击获取 EasyAda-hypervisor 的根权限实现窃取和篡改用户隐私数据。按照前文对 EasyAda-trust 全生命周期的分析，实际部署流程中黄色部分通常是基础软件的一部分，由统一的供应商负责开发部署，确保这些组件安全可信符合供应商的利益，因此可以将其视为无恶意的软件，但是 EasyAda-hypervisor 仍然具备较大攻击面，难以满足更高等级的安全需求。

因此，EasyAda-trust 在第一层 TEE 的基础上引入了第二层 TEE（如无特殊说明，后文 TEE 相关描述均默认描述第二层 TEE），以运行在最高特权级的 SM 作为核心软件 TCB，目标是借助 RISC-V PMP 等机制提供的强制内存隔离能力实现 CVM 与不可信组件的内存隔离，保护 CVM 中数据和代码的不被窃取和篡改，并且借助一个更加精简的 SM，在实现隔离不可信组件和 CVM 执行流的同时缩小软件 TCB 的攻击面。

EasyAda-SM 的设计目标是在不同硬件平台上对通用的安全固件，通过模块化和分层设计，允许用户根据使用场景引入或移除模块。EasyAda-trust 拓展了 EasyAda-SM，引入了 CVM 管理模块等组件并设计实现了基于 EasyAda-SM 的 TSM 驱动。

同时，EasyAda-trust 拓展了 EasyAda-hypervisor 以实现对 CVM 生命周期的管理。为了简化 CVM 创建流程，避免不稳定的中间状态增加 TEE 攻击面并影响其安全性，EasyAda-trust 仅在 EasyAda-hypervisor 中针对 CVM 定义了三种无中间状态的动作：**转换（promote）**，**执行（run）**和**销毁（destory）**。转换负责将一个已

创建但未运行的 VM 实例通过 TSM 驱动转换为一个 CVM 实例；执行负责运行一个 CVM 实例；销毁则负责停止一个 CVM 实例并回收其资源。

进一步的，为了在端侧实现安全的外设管理和机密异构计算环境，EasyAda-trust 在设计并实现了专用的加速器管理 CVM，并针对无 IOMMU 的场景进行了优化。

虽然 EasyAda-trust 将 EasyAda-SM 和 CVM 视为可信的软件 TCB，但是可信并不意味着绝对安全，用户在开发过程中可能因为缺乏测试等原因引入破坏 TEE 安全性的代码漏洞，尤其是不安全的内存访问导致的内存泄漏等问题。针对用户可能引入的代码漏洞，EasyAda-trust 对 EasyAda-SM 和 CVM 分别采取了不同的方案以尽可能减少代码漏洞或隔离并限制局部代码漏洞，避免单一漏洞导致整个 TEE 被攻破。EasyAda-SM 使用内存安全的编程语言 Rust 实现以尽可能消除其中因为不安全的内存访问产生的代码漏洞，并通过分层设计，将不安全的内存操作进行封装和限制以降低测试和形式化验证的难度。同时，EasyAda-trust 为每个 CVM 启用了硬件虚拟化拓展，通过二阶段地址翻译实现不同可信组件间的强制内存隔离，避免 CVM 的安全漏洞影响 TEE 的稳定性和安全性。具体设计和实现将在后文详细介绍。

3.4.2 安全监视器 EasyAda-SM 设计

3.4.2.1 设计目标

EasyAda-SM 并非专为 EasyAda-trust 设计，而是被设计为支持部署从简单裸机应用到复杂 CVM 级别 enclave 抽象的通用的安全固件。任何运行在 HS/S 态的不可信组件都可以通过 SBI 接口获取 EasyAda-SM 提供的 TEE 等能力。因此本文对 EasyAda-SM 提出了以下设计目标。

- (1) **兼容性**：EasyAda-SM 应当兼容现有 RISC-V 主流 TEE 方案，包括不可信组件的 host，enclave 中运行的 TA 和 TEE-OS，以及开发工具链等。同时 EasyAda-SM 应当尽可能兼容 RISC-V 商用硬件环境，支持在不同级别的硬件环境上提供统一的 TEE 安全能力。
- (2) **安全性**：不安全的内存访问是 TEE 中可信组件需要面临的主要代码漏洞。EasyAda-SM 应当尽可能消除内部不安全的内存操作，增强 TEE 的安全性。
- (3) **可拓展性**：EasyAda-SM 的核心能力仅能依赖安全保护所需的最小硬件集合以确保对硬件环境的广泛兼容性。为了确保 EasyAda-trust 开发者可以充分利用不同硬件环境上的资源，EasyAda-SM 的能力集合应当具备可拓

展性，开发者可以拓展或收缩其能力边界适应不同的硬件环境。

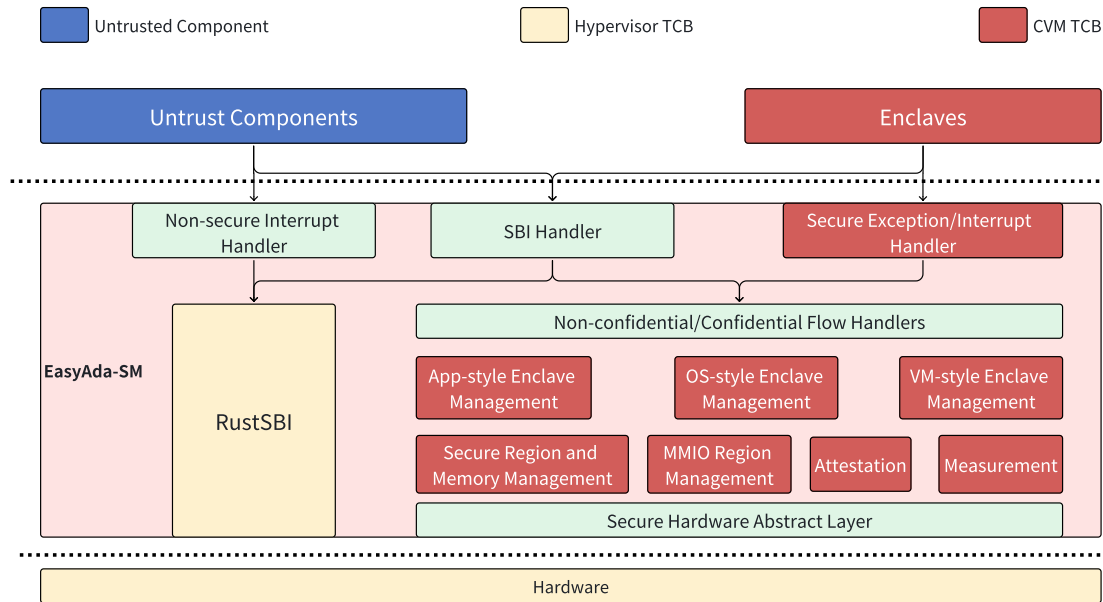


图 3-2 EasyAda-SM 总体架构

如图3-2所示，EasyAda-SM 是整个 EasyAda-trust 的核心安全组件，作为不可信组件和 enclave 间交互的桥梁，其需要承担以下几个核心职能：（1）对外提供 SBI 接口服务；（2）建立可信链，包括密钥存储和对 CVM 镜像和初始状态进行度量等功能；（3）隔离不可信组件与 enclave，包括 CPU 隔离，内存隔离，中断和 I/O 隔离等。

3.4.2.2 内存分类

EasyAda-SM 将内存划分为以下几类并据此设计内存隔离方案。

- （1）**通用区域和通用内存**：任何未被 SM 标记和保护的上/外内存都属于通用区域下的通用内存。为了提高系统整体的内存利用率，默认情况下所有内存都是由不可信的 host 负责管理的通用内存。
- （2）**安全区域**：被 SM 标记和保护连续内存区域被称为安全区域。安全区域由通用区域划分而来，SM 会为每个安全区域分配唯一的硬件单元用于实现内存隔离。
 - 1) **安全内存**：被 SM 标记和保护，但是尚未被 enclave 使用的安全内存，由 SM 负责分配和管理。
 - 2) **enclave 内存**：被 SM 标记和保护并且被 enclave 使用的安全内存。Enclave 内存由 SM 负责配置和保护，但是在 enclave 释放之前 SM 无权进行释放或再分配。

基于以上设计目标，本文给出如图3-3所示的 EasyAda-SM 的总体设计。EasyAda-SM 将通用内存视为初始状态（未初始化），enclave 内存视为最终状态（被使用），安全内存作为两者转化的桥梁，处于中间状态（初始化但未使用）。EasyAda-SM 引入安全内存的目的是平衡**缩小 SM 攻击面**和**提高系统整体内存利用率**两个需求。

Keystone 没有安全内存的中间状态，通用内存直接转化为 enclave 内存，优势是内存管理简单，攻击面较小，劣势是难以在 enclave 运行期间动态伸缩内存；ACE 则在系统启动时静态划分安全内存和通用内存区域，SM 负责从安全内存中为可信 VM 分配 enclave 内存，优势是可以在可信 VM 运行期间动态伸缩内存，劣势是安全区域在系统初始化时静态划分，容易降低系统整体内存利用率；而 RISC-V 的 CoVE 拓展则要求 host 以物理页为单位捐赠内存并由 SM 直接转化为 enclave 内存，优势是提高了系统整体内存利用率，劣势是 SM 需要实现多个 SBI 接口并且完全依赖 host 侧实现内存管理，显著增加了 SM 的攻击面。

因此 EasyAda-SM 延续了 Penglai 的设计思路，允许 SM 从 host 侧动态获取内存并转化为相互隔离的安全区域或归还安全区域到通用内存，然后由 SM 负责从安全内存中动态分配 enclave 启动和运行时需要的 enclave 内存。在提高系统内存利用率的同时减少对 host 侧内存管理的依赖，缩小 SM 的攻击面，实现两种需求的平衡。

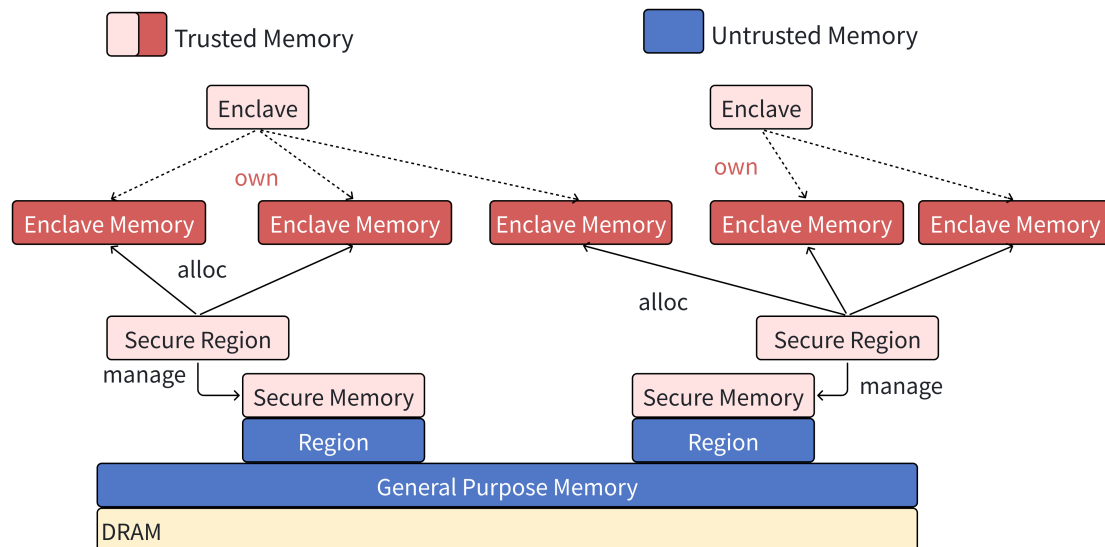


图 3-3 EasyAda-SM 的内存管理模式

3.4.2.3 模块化和分层设计

EasyAda-SM 将 TEE 所需的操作，如内存隔离，密钥管理，镜像度量和安全中断和异常处理等抽象为独立的子模块，每个子模块对外提供抽象的操作原语。如图3-2所示，EasyAda-trust 开发者可以针对不同的 TEE 方案开发独立的 enclave 管理模块，也可以对特定模块进行修改以适配专有指令集和硬件。根据最小特权原则（Principle of Least Privilege），每个模块仅被授予完成其特定功能所必需的最小权限集合，从而限制潜在安全漏洞的影响范围。

同时，通过对现有 RISC-V 主流 TEE 方案进行深入分析，本文发现不同 TEE 方案对安全内存管理，使用和隔离机制存在较大差异，在安全启动和中断，I/O 隔离等方面的机制和策略则较为共通，如 Penglai 的 SM 被设计为实现更灵活和细粒度的安全内存管理，因此需要 SM 实现安全内存分配能力；Keystone 的 SM 则被设计为一次性内存分配，SM 不进行动态安全内存分配，仅负责进行保护；ACE 中 TSM 驱动为了适应 CVM 二阶段页表的构建，实现了专用的物理页分配器（Page Allocator），充分利用 Rust 的所有权机制消除了重复分配的隐患。

EasyAda-SM 分层设计的核心是统一的安全区域和内存管理层（Secure Region and Memory Management, SRMM）。SRMM 将安全内存相关操作，如创建/销毁安全区域，申请/释放安全内存，隔离安全内存等进行封装，对上层的 enclave 管理模块等提供了统一的安全内存管理原语。同时，SRMM 将安全内存管理原语依赖的硬件操作划分为内存管理和内存保护两个部分，并且抽象出了统一的内存管理原语和内存保护原语，开发者可以根据硬件环境和业务需求提供具体的原语实现。通过提供统一的内存安全原语，SRMM 简化了开发 enclave 模块的难度；通过抽象统一的内存管理和保护原语，SRMM 实现了不同 enclave 管理模式间基础代码的复用和可拓展性，开发者可以在上层无感的条件下复用已有原语或根据场景需求和硬件环境实现新的原语。

3.4.3 安全特性设计

接下来将介绍 EasyAda-trust 如何针对 CVM 需求设计安全的虚拟化方案。

3.4.3.1 安全启动

安全启动的目标是构建可信链（Trusted Chain, TC），确保只有由授权用户开发的，未经篡改的可信组件可以加载到 TEE 中运行，并且最终的安全应用仅能在经过验证的平台上运行并获取敏感 I/O 数据。图3-4完整展示了 EasyAda-trust 开发生命周期中所有角色及其任务，其中红色/淡红色为可信组件以及可信的供应商；绿

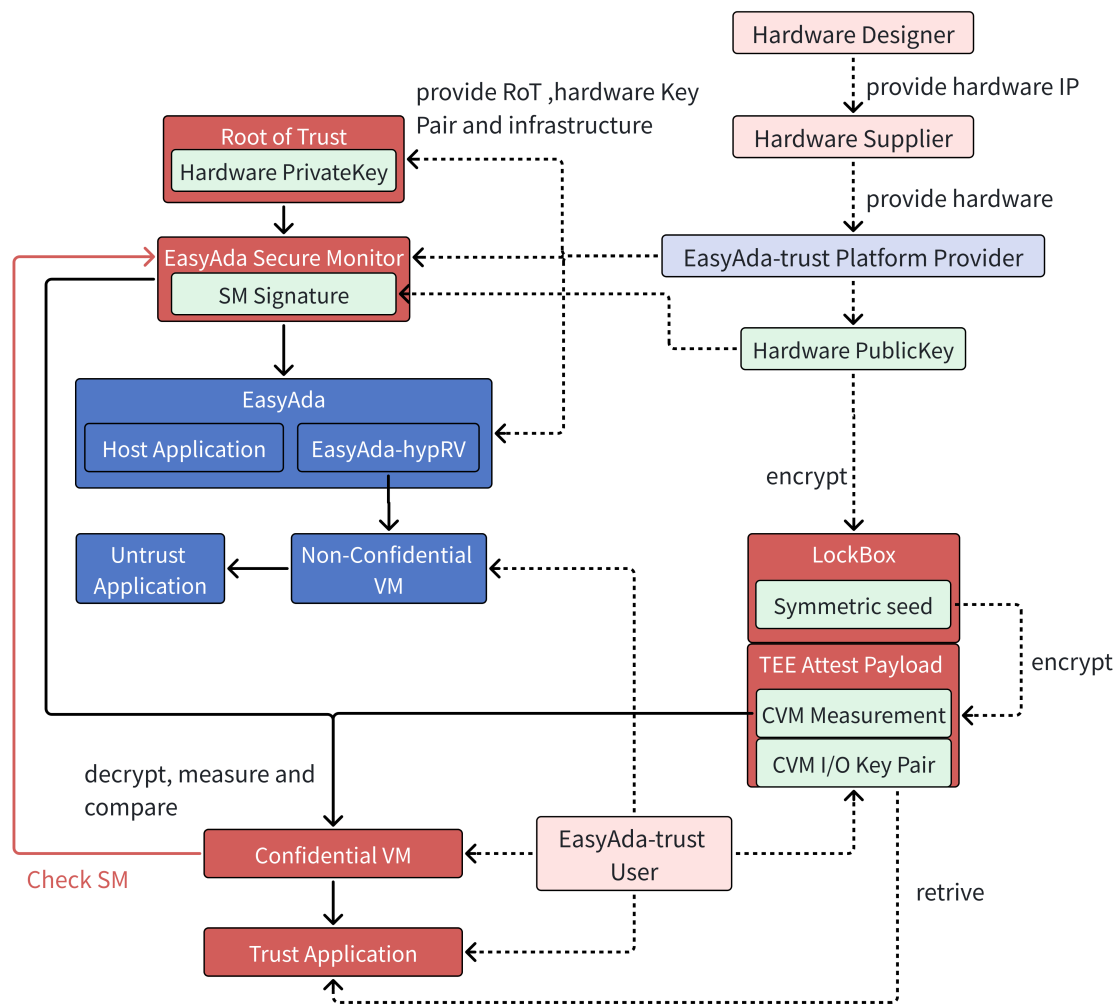


图 3-4 EasyAda-trust 可信启动流程

色组件为可信供应商提供的度量值，密钥对等；蓝色/淡蓝色组件则为不可信的供应商和组件。实线表示可信链建立的流程，虚线表示供应商与组件间的供应关系。

可信根: RoT 是可信链建立的起点。EasyAda-trust 将整个硬件和固件（存储在 eFUSE 和 ROM 等中的代码和数据）视为 RoT，即 EasyAda-trust 信任芯片和硬件平台供应商。为了减少对特定硬件如 TPM 的依赖，EasyAda-trust 平台供应商可以将硬件私钥存储在硬件 RoT 中作为根密钥，而硬件公钥提供给上层，用于生成数字签名和加密。

部署安全监视器：硬件平台供应商需要提供 RoT，硬件密钥对和 SM。在开发阶段，硬件平台供应商负责编译获得 SM 的二进制文件并对其计算生成预期度量值，然后使用平台供应商提供的硬件公钥生成数字签名。在系统启动时，RoT 会尝试对 SM 进行度量并结合硬件私钥对比数字签名，如果两者符合则证明 SM 是可信的，可以将 SM 加入到可信链中。此时 RoT 会将硬件私钥传递到 SM 中并进入

SM 初始化代码，SM 初始化流程会首先清空 RoT 副本并将 SM 使用的内存进行隔离，确保硬件私钥不会外泄。

环境初始化：SM 完成自身的内存隔离后，将继续进行两个部分的初始化：SBI 运行环境和 TSM 驱动初始化。该阶段初始化逻辑取决于 SM 设计：（1）如果将 TSM 驱动作为 SBI 固件的拓展，则首先进行 SBI 运行环境初始化，然后进行 TSM 驱动初始化，最后由 SBI 固件转移控制权。（2）如果将 SBI 固件作为 TSM 驱动的静态库则相反，由 TSM 驱动调用 SBI 固件实现的接口进行初始化，并最终由 TSM 驱动转移控制权。EasyAda-trust 采用方法（2）以确保 TSM 驱动可以在后续完全控制中断和陷入入口。TSM 驱动初始化主要需要完成安全内存初始化，防止 SM 在系统运行过程中被运行在更低特权级的不可信组件非法访问和篡改，并且确保低特权软件的执行在明确定义的入口点陷入到 TSM 驱动和 SM 中。当 TSM 驱动初始化完成时，SM 已经通过硬件实现了与不可信组件的强制隔离，可以安全进入更低特权级转移控制权到 hypervisor。

部署 CVM：部署 CVM 时需要进行双向验证（1）验证 CVM 是否可信且未被篡改。（2）验证硬件环境和 TSM 驱动是否可信。云服务厂商通常采取远程证明（Remote Attestation），通过远程的可信设备验证当前硬件环境和代码是否可信且未经篡改，只有经过验证的环境才能成功部署 CVM。然而嵌入式系统可能在无法访问网络的环境中长期运行。为了支持在离线环境中度量 CVM，EasyAda-trust 支持本地证明^[44]。在本地证明中，CVM 所有者使用自己的密钥对称加密当前 CVM 的 TEE 证明载荷（TEE Attest Payload, TAP）——用于存储 CVM 完整性度量值和特定密钥的文件，并且配合一个用于存储该对称密钥，使用可信的硬件公钥加密的锁盒（LockBox），同一 TAP 可以关联多个使用不同硬件公钥加密的锁盒以实现在多个硬件环境上部署。在部署 CVM 时，SM 会将 CVM 的初始化状态保存到安全内存中并对关键数据进行度量，然后使用 RoT 提供的硬件私钥解密锁盒，使用锁盒中的对称密钥解密 TAP，最终得到 CVM 的数字签名和特定密钥，如果对比成功则当前 CVM 可信，可以被部署。通过这种方法，本地证明可以实现 CVM 与硬件环境的双向证明，因为 TAP 仅能在持有对应硬件私钥的硬件环境上进行解密，即使 CVM 被强行拉起，不可信的硬件和固件仍然无法获取 CVM 开发者提供的特定秘密，无法从 CVM 中解密敏感代码或数据；另一方面，任何对 TAP，锁盒或者 CVM 镜像的篡改都会导致最终的度量失败，确保只有可信的，未经篡改的 CVM 可以被部署。同时，本地证明与远程证明并不冲突，在具备网络条件的环境下仍然可以使用远程证明验证可信环境。

获取密钥：当 CVM 通过度量后，TSM 驱动会将 TAP 中解密获得的特定秘密存储在安全内存中。CVM 可以通过 SBI 请求获取自己的秘密，秘密由 TSM 驱

动保护，其它 CVM 或不可信组件无法获取，用于进一步的代码和 I/O 数据加解密中。至此可信链建立完成。

3.4.3.2 安全 CPU 虚拟化

EasyAda-hypervisor 能够确保 VM 间 vhart 的隔离，VM 间不会泄漏上下文信息，但是无法实现 EasyAda-hypervisor 与 VM 间的上下文隔离，因此 EasyAda-trust 对 hart 引入了机密流和非机密流以实现不同执行环境间上下文的隔离。

RISC-V 的硬件虚拟化拓展引入了虚拟化专用特权级以实现 VM 间以及 VM 和 host 间 CPU 上下文的隔离。对于不可信 VM，SM 会将大多数系统级异常委派到 hypervisor 进行处理以监控 VM 的任何敏感行为。但是对于 CVM，将异常委派到 hypervisor 进行处理会导致 VM 运行上下文泄露给 hypervisor，因此需要 TSM 驱动拦截和监控 CVM 的敏感行为。

如图2-4所示，物理 CPU 的分配和 vhart 与物理 CPU 的绑定由 hypervisor 完成，与普通 VM 保持一致。但是为了避免不可信组件访问和篡改 CVM 上下文，在切换到 CVM 上下文后会配置其触发的异常直接陷入 M 态中，通过轻量上下文切换到 TSM 驱动进行处理。若异常需要由 hypervisor 处理，则由 TSM 驱动进行完整的安全上下文切换，清理所有上下文并切换到 hypervisor 上下文进行处理。

3.4.3.3 安全内存虚拟化

内存虚拟化主要涉及为 VM 分配运行内存，并建立和维护 VM 运行内存和虚拟设备地址的第二阶段地址映射表。前者用于容纳 VM 运行时的代码和数据，后者用于实现设备直通和 VM 间通信等。

不可信组件与 CVM 间内存隔离：为了实现 CVM 与不可信组件间的内存隔离，CVM 无法直接使用 hypervisor 分配的不可信内存作为运行时内存，因此 EasyAda-trust 引入了安全内存和安全区域的概念。安全内存指由 SM 配置，使用硬件实现强制内存隔离，不可信组件无法直接访问的内存区域，任何用于存储 CVM 敏感数据和代码的内存必须使用安全内存以确保其机密性和完整性。EasyAda-trust 以安全区域为单位管理安全内存，每个安全区域标记了一片物理连续的内存区域，由绑定的硬件和 SM 进行保护，安全区域内的内存分配由 SM 负责。安全内存由不可信内存转化而来，hypervisor 可以申请一块连续的不可信内存捐献给 SM，由 SM 进行保护，并将其标记为安全内存，之后任何不可信组件都无法直接访问这片内存区域。

CVM 间内存隔离：虽然 CVM 被视为 EasyAda-trust 软件 TCB 的一部分，但是 CVM 中的代码漏洞可能破坏 TEE 提供的安全保护，并且控制了 hypervisor 的恶

意攻击者可能运行恶意的 CVM 以尝试攻击受害 CVM，因此 CVM 间同样需要实现内存隔离。ACE^[36] 中所有 CVM 共享一个安全区域，通过硬件虚拟化提供的二阶段地址翻译实现 CVM 间内存隔离。虽然这提供了足够的安全防护，但是难以根据需求对安全内存进行动态收缩和扩容。EasyAda-trust 支持增加或回收安全区域，多个 CVM 可以共享一个安全区域以提高内存利用率。CVM 间的内存隔离则通过二级段地址翻译实现隔离。

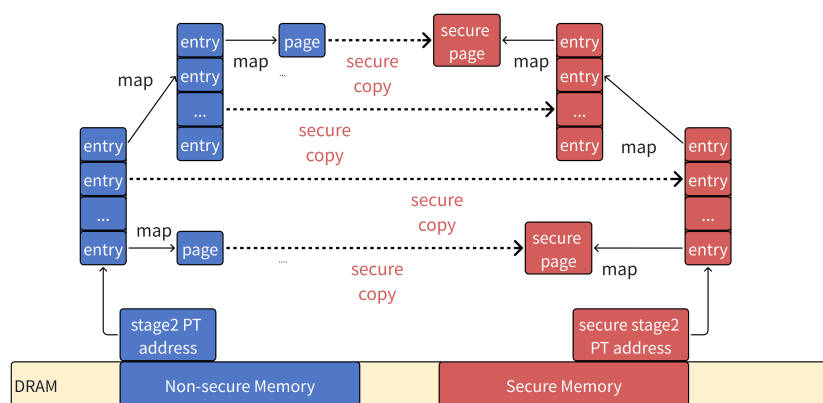


图 3-5 CVM 二阶段页表初始化

CVM 内存虚拟化：CVM 在创建 VM 实例阶段与不可信 VM 类似，由 hypervisor 完成内存申请和二阶段页表初始化等。当用户需要将 VM 转换为 CVM 时，hypervisor 驱动通过 TSM 驱动对外提供的 promote 接口，将二阶段页表和 FDT 的 IPA 作为参数交给 TSM。如图3-5TSM 首先会遍历由 hypervisor 为当前 VM 维护的二阶段页表，申请安全内存用于构建完全一致的页表作为 CVM 的初始二阶段页表，同时按序申请安全内存用以拷贝不可信内存中的数据，避免后续 hypervisor 篡改初始状态。

然后 TSM 驱动将按序遍历 CVM 的二阶段页表并度量 FDT，kernel 和 initrd 的初始度量值（具体 CVM 度量流程设计将在后续安全启动小节详细说明），并与 CVM 开发者提供的初始度量值进行对比，若度量值校验通过则说明当前 VM 符合预期，由可信的开发者开发并且没有被 hypervisor 篡改，允许被拉起并访问开发者提供的用于后续加解密 I/O 数据的密钥。

安全 MMIO 区域隔离：RISC-V 采用统一编址模式，设备的 MMIO 区域以内存区域的方式进行管理。因此 EasyAda-trust 基于安全区域与不可信组件的隔离实现了设备 MMIO 区域与不可信组件的隔离，并且通过为安全区域引入额外的标识符，标识当前安全区域类型是否为 MMIO 区域，并且可以进行进一步细分实现具体设备类型的标识。

不可信共享内存：除了存储数据和代码的安全内存，CVM 与不可信组件间需

要额外的共享内存以实现高效的数据交换，尤其针对磁盘等块设备的 I/O 虚拟化场景。EasyAda-trust 支持 CVM 向不可信 host 请求共享内存。

不可信共享内存由 host 分配和管理，主要用于 VM 间，VM 与 host 间，以及不可信组件与 CVM 间的数据交换。host 完成内存分配后，需要由 hypervisor 映射到 VM 地址空间内，或由 TSM 驱动映射到 CVM 地址空间内，确保组件可以在当前地址空间下直接访问共享内存。因为不可信共享内存由 host 管理，因此 CVM 在读写这类内存时，需要进行严格的安全检查并主动对敏感信息进行加密避免泄露。

EasyAda-trust 并不支持在 CVM 间分配可信共享内存。因为（1）根据角色划分，VM 和 CVM 均由同一角色——EasyAda-trust 用户负责开发，应用功能在开发时已经相对确定，不需要也不应该将需要大量数据交换的多个功能拆分到多个 CVM 中实现（2）CVM 和 TSM 驱动作为 TCB 应当尽可能保证精简，在 CVM 间支持可信共享内存会进一步增加 TCB 的大小和复杂度（3）少量的数据交换可以通过不可信共享内存和加密信道实现 CVM 间的数据交换。综上，EasyAda-trust 的设计并未考虑使用可信共享内存实现 CVM 间通信。

3.4.3.4 安全中断虚拟化

中断安全分为两个部分：中断发送的安全和中断接收的安全。两者都要求目标的中断不应被不可信组件获取和篡改。

对于发送中断，RISC-V 要求所有中断必须在 M 态下发送。对于不可信 VM，发送中断首先会陷入到 hypervisor 驱动中，由 hypervisor 驱动决定实际的接收 hart，再通过 SBI 调用陷入 SBI 固件进行中断发送。在这个流程中 hypervisor 对 VM 的中断有绝对的控制权。但是在 CVM 场景下，不可信组件无权控制 CVM 发送的中断，需要直接陷入到 TSM 驱动中发送，并且需要 TSM 驱动为 CVM 维护接收方到物理 hart 的映射关系。

对于接收中断，SM 会将除外部中断外的所有中断全部委派到 HS 态由 hypervisor 进行处理，因为 EasyAda-hypervisor 采用静态分区策略，hypervisor 可以进一步将时钟中断直接委派到 VS 态由 VM 直接处理，避免频繁陷入 hypervisor 带来的性能开销。其他中断如软件中断等需要陷入到 hypervisor 中进行处理，由 hypervisor 决定是否需要注入中断或切换到 vhart 服务线程上下文运行，无需对 CPU 当前上下文进行清理，相关中断配置由 hypervisor 负责。

但是对于 CVM，除时钟中断外的其他中断必须绕过 hypervisor 直接陷入到 TSM 驱动中进行处理并决定是否需要中断注入。如果中断涉及进一步的上下文切换，需要由 TSM 驱动负责清理 CVM 上下文避免泄露任何敏感数据。中断相关配置均由 TSM 驱动负责。

除了中断发送/接收安全外，还需要额外注意拒绝服务攻击（Deny of Service, DoS），包括 CVM 对 host 侧的 DoS 攻击，以及 host 对 CVM 侧的 DoS 攻击。前者因为 host 侧可以通过 TSM 驱动强制销毁 CVM 解决，而后者则需要进一步区分攻击方式，一种攻击模式是 host 生成大量伪造的中断请求，迫使 CVM 频繁陷入 TSM 驱动，导致 CVM 无法及时响应其他正常请求，这种攻击可以通过 TSM 驱动对来自 host 侧的中断进行监控和计数，进而对中断请求进行管控和屏蔽实现防御；另一种攻击模式则是 host 拒绝响应来自 CVM 的任何请求，因为 host 被设计为负责管理系统资源，所以不针对这种攻击进行防御。

3.4.3.5 安全 I/O 虚拟化

EasyAda-hypervisor 原有的安全 I/O 虚拟化方案能够实现：（1）VM 间数据流和控制流的隔离（2）确保同一时刻一个设备仅能划分给一个软件实体（VM 或 hypervisor）直接访问。

为了确保 CVM 使用的外部设备的安全性，EasyAda-trust 需要额外实现以下安全能力：

- （1）**平台和硬件的可信性验证**：根据当前设备的安全级别，在 CVM 使用安全设备之前对设备进行可信验证，防止 CVM 被恶意外设攻击导致敏感数据的泄漏和篡改。
- （2）**CVM 生命周期中 I/O 数据的安全性**
 - 1) **CPU 侧数据安全性**：保护 CVM 访问外设过程中输入输出数据的机密性和完整性，包括通过设备独占，数据加密和校验等方案实现。
 - 2) **设备侧数据安全性**：具备 DMA 能力的设备可以绕过 CPU 侧的内存隔离机制直接访问物理内存，恶意设备可以利用这一点直接读写安全区域，因此需要根据外设安全级别限制外设可访问的内存区域。

针对以上安全需求，EasyAda-trust 根据外设的使用特点将设备划分为两类：

- （1）**普通 I/O 设备**：这类 I/O 设备包括串口，网卡，磁盘等设备，这些设备的特点是设备对数据内容不敏感，无需或仅需少量计算。
- （2）**安全设备**：这类设备重视安全性，对输入数据内容敏感或需要进行大量计算，如指纹采集器，摄像头和硬件加速器等。

EasyAda-trust 根据设备类型采取了不同的可信性验证方案。对于由 host 管理的普通 I/O 设备，EasyAda-trust 不对其进行可信验证。而对于安全设备，EasyAda-trust 采用了 host 分配，TSM 驱动鉴权 and 管理的逻辑，并在安全启动流程中引入了可信设备注册，使用和回收的可信链。以系统开发时硬件供应商提供的可信硬件配置文件作为可信链起点，在 host 未将可信设备提升为安全设备时，仍然可以正

常控制该设备，同时 host 可以在系统运行过程中向 TSM 驱动请求提升设备为安全设备，由 TSM 驱动根据硬件配置文件对设备进行可信验证，只有通过验证的外设会被视为可信设备并提升为安全设备。因为 CVM 在启动时会对硬件环境进行验证，未通过验证的硬件环境无法对 CVM 核心的机密代码和数据进行加解密，因此由 TSM 驱动负责对外设进行可信验证是符合可信链建立流程的。

完成设备的可信验证后，EasyAda-trust 需要进一步保护机密 I/O 数据，对于 CPU 侧的保护，有三种思路可以用于保护 CVM 的输入输出数据：

1. 基于纯加密的数据保护：如图3-6(a)所示，因为系统设备主要用 host 负责管理，最简单的方案是 CVM 使用开发者提供的密钥对所有 I/O 数据进行加解密和度量，并通过不可信的共享内存和 VirtIO 协议栈实现与不可信组件的数据交换。最终由 host 侧的 vhart 服务线程调用实际外设驱动完成对外设的读写。

这种保护模式适用于纯粹的 I/O 请求保护，如网卡设备，串口，磁盘块设备等。由 CVM 作为数据发送者对数据进行度量和加密，用于加解密的密钥仅在安全内存中才会以明文形式存储和使用。而数据使用者通过配对的密钥进行解密和度量，阻止信道上任何潜在的攻击者进行窃听，可以有效保障数据的机密性。但是一方面，因为需要由不可信组件读写数据，需要额外的重传机制保障数据的完整性；另一方面，多数计算任务要求数据以明文形式提供，对数据进行加解密会引入额外的性能开销。

2. 基于临时设备授权的数据保护：如图3-6(b)所示，如果将设备管理和授权剥离，将设备授权提升到 SM 进行管理，由 SM 在不可信组件和 CVM 间实现设备的分时授权，确保同一时刻一个设备仅能授权给一个 CVM 或不可信组件使用，当一个实体获得授权之后，其他实体无权对设备进行任何读写和配置操作。

这种保护适用于全局唯一设备的 I/O 请求保护。在软件实体获得授权时可以获得设备的完全访问和配置权限，当 CVM 获取授权时，可以直接使用明文数据读写设备而不用担心不可信组件和其他 CVM 劫持设备或窃取和篡改数据；同样的，当不可信组件获取设备授权时，CVM 也无法劫持或访问设备；当解除权限时，由 SM 或软件实体自身清理设备状态，避免泄露敏感信息。这种模式可以在保护数据机密性和完整性的同时，避免加解密带来的开销。但是这种保护方式需要额外引入复杂的时分复用逻辑，尤其对于在内核态下直接进行管理的设备，需要对内核进行修改以确保 SM 可以正确监控系统对外设的使用，避免因调整设备授权导致当前实体对外设的访问异常中断。

3. 基于硬件分区的数据保护：如图3-6(c)所示，若当前硬件环境中存在冗余或专为安全应用设计的设备，参考静态分区的思想，可以对设备进行分区，将部分设备固定划分给可信组件使用的同时，避免影响不可信组件对外设的访问。如 ARM

下安全世界拥有独立的 UART, GPIO, flash 和 RAM 等设备专供安全世界使用。

这种保护模式适用于专用安全设备或冗余外设的 I/O 请求保护, 如指纹/声纹输入设备, 冗余串口/flash 等。因为 RISC-V 采用统一内存编制, 硬件分区可以基于内存隔离机制实现, CVM 可以以明文形式直接读写 TEE 外设, 无需额外的加解密工作; 同时 CVM 和 host 侧因为访问独立的外设, I/O 流程完全隔离, 对外设的访问互不干扰, 在保证数据完整性和机密性的同时具备更高的效率, 且无需在 host 和 CVM 间实现复杂的时分复用逻辑。但是一方面该方法对硬件环境有较高要求, 在硬件资源受限的环境中难以使用; 另一方面可拓展性不足, 如果存在多个 CVM 实例, CVM 间仍然需要针对设备实现时分复用逻辑。

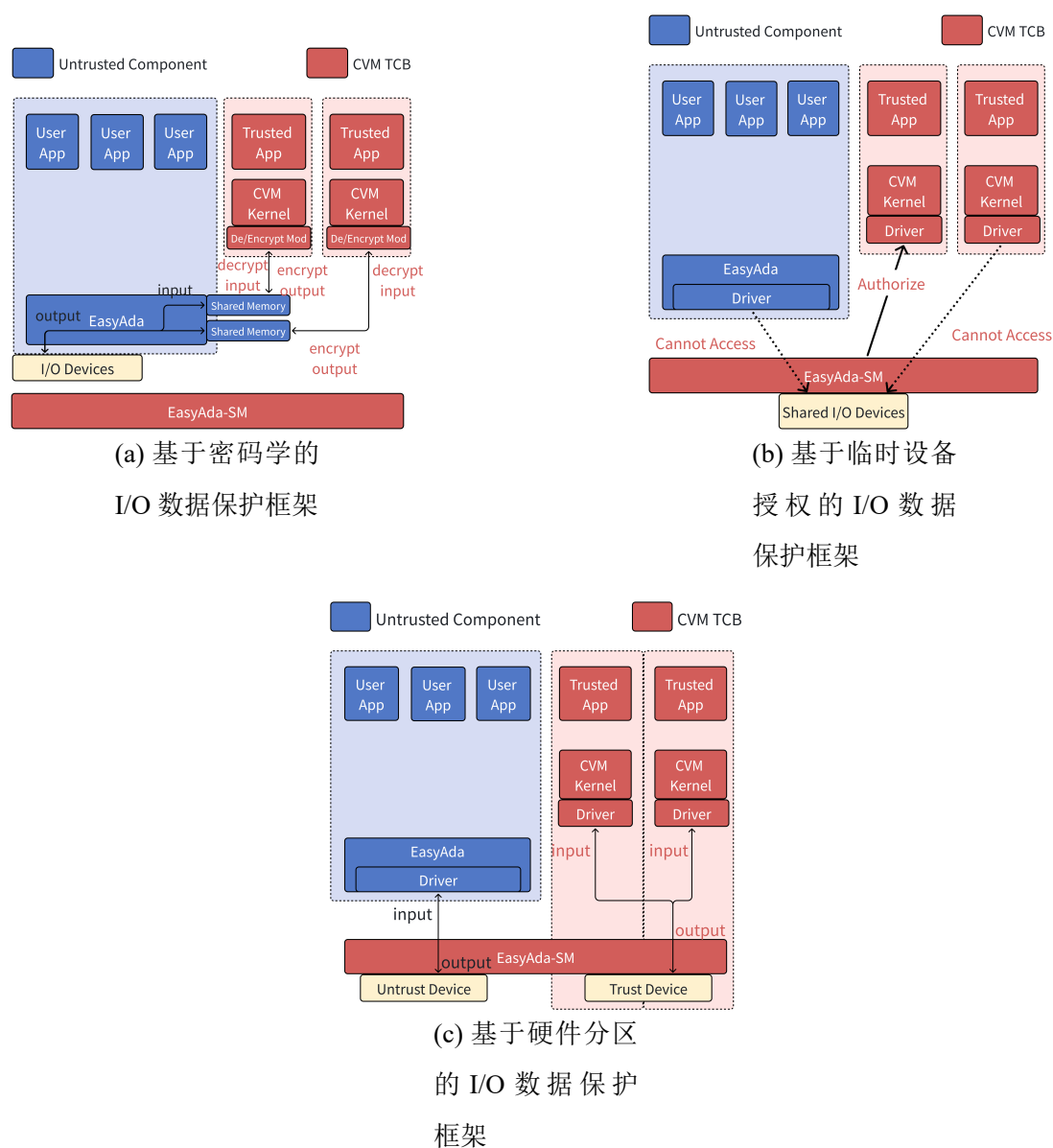


图 3-6 CVM 的 I/O 数据保护机制

对于普通 I/O 设备, EasyAda-trust 允许 CVM 不保护 I/O 数据 (如串口输出) 或参考3-6(a)方法保护 I/O 数据, 设备直接授权给 host 使用, CVM 和不可信 VM 需要通过不可信的共享内存与 host 通信, 由 host 完成对外设的实际操作。简化设备的管理逻辑, 同时确保设备可以拓展给更多组件使用。

而对于安全设备, EasyAda-trust 参考3-6(c)方法, 由 CVM 直接独占设备进行管理, SM 负责实现设备的数据缓冲区和 MMIO 区域与不可信组件间的隔离。CVM 仍然可以通过共享内存的方式与其他组件间进行交换数据。为了避免复杂的运行时设备发现, EasyAda-trust 要求开发者在开发阶段对完成可信设备的标记并将可信的硬件配置存储在硬件 RoT 中或直接固化在 TSM 驱动中, 用于对设备身份进行验证。

对于设备侧的 I/O 数据保护, EasyAda-trust 基于 RISC-V 的 IOPMP 拓展实现。为了避免 IOPMP 条目的快速耗尽和频繁的 IOPMP 切换, EasyAda-trust 对安全设备仅开发所有由 host 分配的安全区域的访问权限, 而非安全设备仅能访问不可信的内存区域, 具体 IOPMP 管理方案实现将在安全监视器章节详细介绍。

同时, 必须考虑到部分端侧硬件环境出于功耗和成本考虑, 仅实现了 RISC-V 硬件虚拟化拓展而未实现 IOMMU, 并且外部设备本身不具备地址管理单元 (Memory Management Unit, MMU)。因此 EasyAda-trust 在部分硬件环境中无法使用 IOMMU 实现 I/O 虚拟地址 (Input/Output Virtual Address), 使用 IPA 直接配置 DMA 会导致外设的内存访问触发未定义的行为, 必须要通过软件实现 IPA 到 HPA 的地址翻译。一种方案是在此类环境上整体回退到全虚拟化设备, CVM 仍然具备完整的设备驱动, 但是任何对 MMIO 设备的访问都需要陷入到 TSM 驱动中完成地址翻译和硬件操作, 对于性能敏感场景会引入无法接受的时延和性能开销。因此 EasyAda-trust 在保留了全虚拟化支持的基础上, 允许 CVM 显式向 TSM 驱动发起请求批量实现 IPA 到 HPA 的翻译, 仅需在配置 DMA 地址时触发一次下限, 并且可以通过合并 I/O 请求进一步降低地址翻译的开销。

为了实现 CVM 使用硬件加速器计算时数据的机密性和安全性, 下面将详细介绍基于 EasyAda-trust 的安全异构计算环境的设计。

3.4.3.6 安全异构计算环境设计

与其他 I/O 设备相比, 硬件加速器丰富的软硬件计算和存储资源增加了 I/O 数据保护的难度; 同时, 因为硬件加速器承担了大量的计算任务, 对性能非常敏感; 而且与其他设备类似, 需要考虑跨执行环境的设备复用。前文已经介绍了通过数据加密和独占设备的 I/O 数据保护, 但是这些措施在实现 CVM 数据保护时存在一定的局限性:

- (1) **纯密码学方案的吞吐开销：**对于资源稀缺的嵌入式场景，安全性并非硬件加速器的第一考量，因此通常并不具备硬件加解密能力，需要通过软件方案加解密，如果采用 3-7 (a) 的方法进行保护，对于 AI 训练中的大规模梯度同步，密码学操作会迅速占满显存控制器的带宽或 CPU 的计算周期，导致严重的计算延迟并显著降低带宽利用率。Lee^[45] 等人的研究显示这种开销导致 DDP 训练迭代时间平均增加了 8 倍，极端情况甚至达到 41.6 倍。同时，密码学基于块的计算方式与 GPU 非连续的内存访问间粒度不一致，会进一步放大加解密的负面影响。
- (2) **独占设备难以满足复杂场景：**复杂嵌入式场景下通常需要部署多个模型以应对不同的使用场景，如车载场景下座舱 AI 和自动驾驶模型通常位于不同的执行环境中，均需使用硬件加速器进行计算。因此 EasyAda-trust 被设计为支持在不可信组件和 CVM 间共享的硬件加速器。

根据硬件加速器与 CPU 的耦合程度，可以将其划分为以下三类：(1) 指令集拓展，将 AI 运算逻辑直接嵌入 CPU 流水线中，集成度最高；(2) 协处理器，与 CPU 共享物理内存并通过 MMIO 进行控制，集成度次之；(3) 片外加速器，拥有独立显存，自由度最高，通常在服务器等高性能场景下使用。端侧场景下为了控制功耗和成本，主流硬件平台中硬件加速器通常以协处理器的模式集成在 SoC 中，以统一内存架构 (Unified Memory Architecture, UMA) 的方式进行管理。因此除了隔离硬件加速器的 MMIO 区域外，还需要额外隔离从内存中动态分配的数据缓冲区。

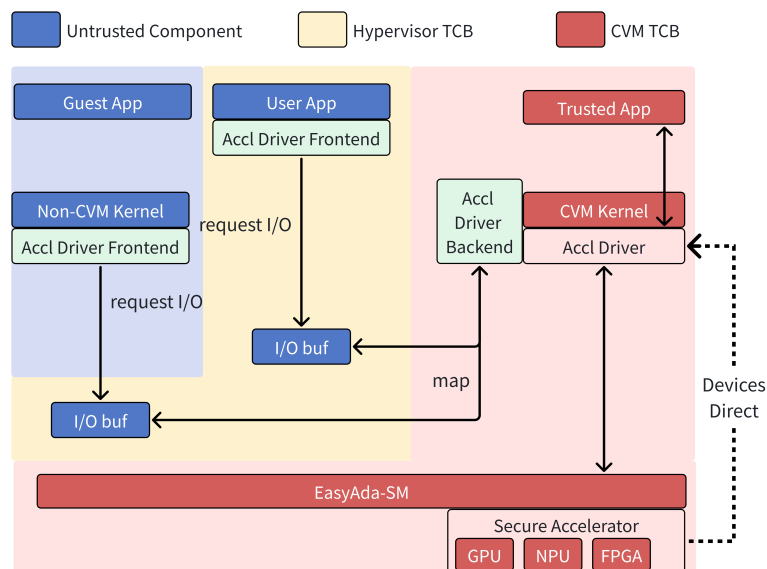


图 3-7 硬件加速器管理 CVM 设计

如图3-1所示，EasyAda-trust 为了降低用户安全管理硬件专用加速器的难度，实

现为不同的硬件加速器提供统一的保护方案，引入了专用加速器管理 CVM（Accelerator CVM，加速器管理 CVM）。得益于 CVM 的引入，开发者可以实现对现有硬件加速器驱动的复用而无需进行代价高昂的移植。

对于 CPU 侧，与其它安全设备类似，有 TSM 驱动通过安全硬件实现硬件加速器 MMIO 区域与不可信组件的隔离。而对于硬件加速器使用的动态数据缓冲区，因为数据缓冲区由加速器驱动从 CVM 的安全内存中进行分配，同样可以复用安全区域提供的内存隔离保障。对于外设侧，TSM 驱动会通过 IOPMP 授予且仅授予硬件加速器访问管理 CVM 的使用的安全区域的访问权限，由 TSM 驱动确保不会因为错误配置 DMA 导致硬件加速器 I/O 地址异常。得益于 I/O 数据保护，CVM 可以直接以明文数据形式与硬件加速器进行交互，避免了高昂的加解密开销。

为了实现硬件加速器在不同执行环境间的复用，加速器管理 CVM 可以作为半虚拟化设备后端，为其它不可信组件提供虚拟硬件加速器设备。不可信组件和加速器管理 CVM 间的虚拟设备发现同样在开发时通过配置硬件配置文件静态实现。因为 EasyAda-trust 的首要目标是确保 CVM 内机密应用和数据的安全性，对于不可信组件则不进行任何保障，因此不可信组件和加速器管理 CVM 间使用不可信的共享内存进行数据交换，并且不强制要求任何数据加密和校验。

3.5 本章小结

本章系统论述了基于 RISC-V 架构的微内核虚拟化方案 EasyAda-trust 的架构设计与核心机制。该方案以 EasyAda 和 EasyAda-hypervisor 微内核虚拟化方案为底层支撑，通过引入 Rust 编写的安全监视器 EasyAda-SM 及内存原语抽象，有效缩减了系统的 TCB。在安全机制方面，建立了涵盖机密虚拟机全生命周期的防护体系，利用本地证明、机密流隔离及二阶段页表等技术，保障了 CPU 状态与内存空间的强隔离性，并构建了抗攻击的安全中断处理路径。此外，针对端侧异构计算需求，方案结合 IOPMP 技术设计了专用加速器管理机制，在提升设备复用效率的同时确保了数据的机密性。

第四章 基于 Rust 的内存安全固件实现

4.1 内存安全固件

为了消除 SM 中不安全的内存访问，EasyAda-SM 使用适合嵌入式场景的内存安全语言 Rust 实现，通过静态的编译时内存检查消除代码中可能存在的不安全内存访问，避免了昂贵的运行时内存检查和回收。EasyAda-SM 基于 RustSBI^[46] 而非基于内存不安全语言 C 的 SBI 固件实现——OpenSBI 实现，进一步消除了 SBI 固件中不安全的内存访问。

4.2 安全区域和内存管理层 SRMM 实现

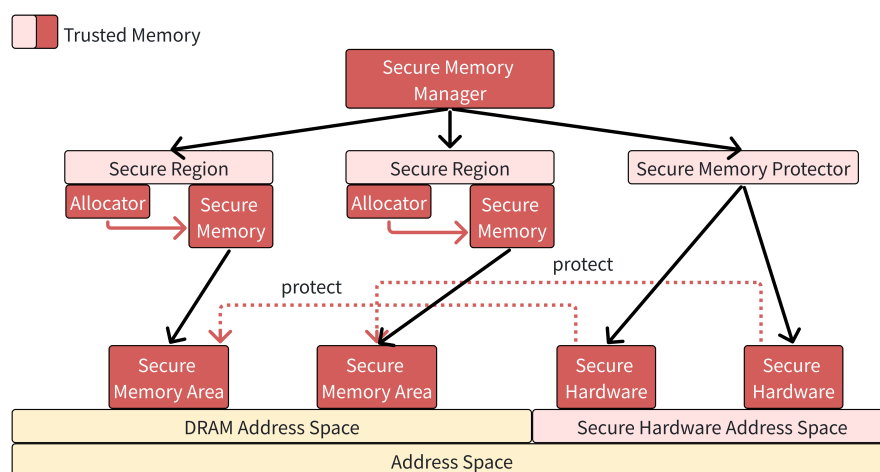


图 4-1 SRMM 管理单元实现

SRMM 的目标是实现不可信组件与安全内存间隔离的同时提高系统整体的内存利用率。如图4-1所示，SRMM 以安全内存管理器（Secure Memory Manager, SMM）作为安全内存的顶级管理单元，对外提供安全内存管理接口，每个 SRMM 包含多个安全区域（Secure Region, SR）和一个安全内存保护器（Secure Memory Protector, SMPt）。

每个 SR 关联一块连续内存区域和一个内存分配器（allocator），内存分配器对应一套内存管理算法实现。严格来说 SR 管理的内存存在被强制隔离之前并不属于安全内存，因为不同 TEE 方案对于安全内存的保护时机存在较大差异，因此 EasyAda-SM 将 SMPt 和 allocator 分离，由 SMM 自行决定内存的保护时机，为了方便叙述，后文以安全内存代指 SR 管理的内存区域。

SMPt 则负责管理用于隔离安全内存的安全硬件如 PMP, 并对外提供内存保护

接口。SMPt 的保护粒度是安全区域，主要目标是隔离不可信组件和安全内存，并辅助实现 enclave 间内存隔离。

为了在同一 enclave 管理模式提供统一的安全内存视图，每个 enclave 管理模块通常使用一个全局的 SMM 实现统一的安全内存管理，并使用每个 SR 独立的 allocator 实现精细化的内存管理。

根据上一章中介绍 EasyAda-SM 的分层设计思想，SMM 抽象了安全内存管理原语，allocator 抽象了内存管理原语，而 SMPt 则抽象了内存保护原语，三者共同实现对安全内存生命周期的管理。下面将按序说明每个原语的具体实现。

4.2.1 内存管理原语

表 4-1 EasyAda-SM 内存管理原语

原语	功能
new	创建一个空的 allocator（此时无安全内存需要管理）
init	使用指定连续内存区域初始化一个 allocator
alloc	从当前安全内存中申请内存
free	释放安全内存到当前安全区域中
avaliable	获取当前安全内存剩余可分配内存
total	获取当前安全内存总大小

如表4-1所示，内存管理原语抽象了安全内存管理所需的操作。包括初始化安全内存，申请/释放安全内存等。为了简化实现并避免重复安全检查引入的系统性能，内存管理原语不进行任何安全检查，如不在 free 原语中检查释放内存是否属于当前安全区域以避免破坏内存管理元数据等。

本文分析了不同 enclave 抽象的安全内存使用特点：

- （1）**基于应用的 enclave 抽象（App enclave）**：此类轻量化 enclave 通常直接运行在用户态，无权访问和控制任何内核态资源，如 MMU 和中断配置等。因此 TA 的虚拟地址空间和物理地址空间在 enclave 运行时不会进行任何修改。
- （2）**基于 OS 的 enclave 抽象（Rt enclave）**：此类 enclave 更加复杂，拥有完整的用户态和内核态资源，TA 的虚拟地址空间在运行时可能修改；但是因为直接运行在与 host 相同的特权级，为了保证高效的内存隔离，enclave 的物理地址空间在运行时不会进行修改。
- （3）**基于 VM 的 enclave 抽象（VM enclave）**：因为开启了硬件虚拟化，此类 enclave 兼具前两种 enclave 的特点：在虚拟化特权级下拥有完整的用户

态和内核态资源，又可以在运行时无感可控地调整 `enclave` 物理地址空间并且不破坏当前内存隔离。

EasyAda-SM 针对不同类型 `enclave` 使用安全内存的特点，提供了三种参考内存管理原语实现，下面将介绍其具体实现。

4.2.1.1 基于空分配算法的原语实现

该实现为安全区域的内存管理器的空实现，仅用于安全区域初始化，因为部分关键安全区域（如 SM 自身代码和数据占用内存区域）仅需要进行隔离，不允许进行任何分配，需要使用空分配算法作为分配器占位符。在空分配算法上尝试调用任何初始化或分配操作都会导致异常。

4.2.1.2 基于伙伴分配算法的原语实现

Buddy 分配器基于对 RustSBI 现有系统堆管理算法的复用，使用 buddy 算法管理安全内存，支持多个 `enclave` 共享相同的安全区域。该实现针对 App `enclave`，以 2 的幂次为粒度分配和管理安全内存，足以满足 App `enclave` 的内存需求，可以较好的兼顾分配效率和内存利用率，同时内存管理元数据直接存储在安全内存中，对 SM 本身的内存依赖较少。劣势是为了简化管理逻辑需要预留一部分安全内存用于存储内存管理元数据，在大片内存分配场景下容易产生外部碎片。

该算法可以确保内存分配行为的安全，即同一内存块同一时刻能且仅能属于一个 `enclave` 内存区域。但是对内存块间的隔离不提供任何保障，依赖 `enclave` 管理模块配置 MMU 实现隔离，因此不可用于 Rt `enclave` 的内存管理。

4.2.1.3 基于一次性分配器的原语实现

一次性分配器将当前安全区域内所有安全内存作为一个内存块进行分配和释放，同一安全区域仅供一个 `enclave` 使用。该实现专门针对 Rt `enclave` 和安全设备的内存管理。因为 Rt `enclave` 在内存管理上具备两个特点：（1）可以在 `enclave` 内部进行内存分配，利用预留的空闲内存区域（2）可以配置 MMU，因此不能通过 MMU 实现 `enclave` 间内存隔离。而一个安全设备的内存区域应当完全由一个实体进行管理，不应进行拆分。一次性分配器仅对 `enclave` 内存需求和类型进行简单判断，如果类型匹配且安全内存满足 `enclave` 需求，则直接完整分配所有安全内存为 `enclave` 内存，并返回内存信息，由 `enclave` 在内部进行二次内存管理；同时复用 SMPt 提供的安全区域间隔离实现 `enclave` 内存间的隔离（因为此时 `enclave` 独占安全区域）。

该算法可以确保任意类型 `enclave` 间的内存隔离，并且分配逻辑简单，开销较

低。但是因为没有对内存进行任何分配，对于不具备内存管理能力的 `enclave` 容易引入内部碎片降低内存利用率；同时，`SMPt` 可以使用的安全硬件数量较为有限（如 `RISC-V PMP` 条目数量多为 16，最多不超过 64 个），该方法会快速耗尽可用的安全硬件，因此不应大量使用。

4.2.1.4 基于页分配器的原语实现

页分配器（`Page Allocator`）是专为 `VM enclave` 设计实现的内存分配器。充分利用页表以特定粒度（如 4KB，2MB 和 1GB 等）建立映射的特点，结合 `Rust` 的所有权机制，避免了传统管理方式通过不安全的内存操作实现内存分配时可能导致内存管理元数据破坏的问题。**页分配器结构：**以图4-2为例，页分配器以类似页

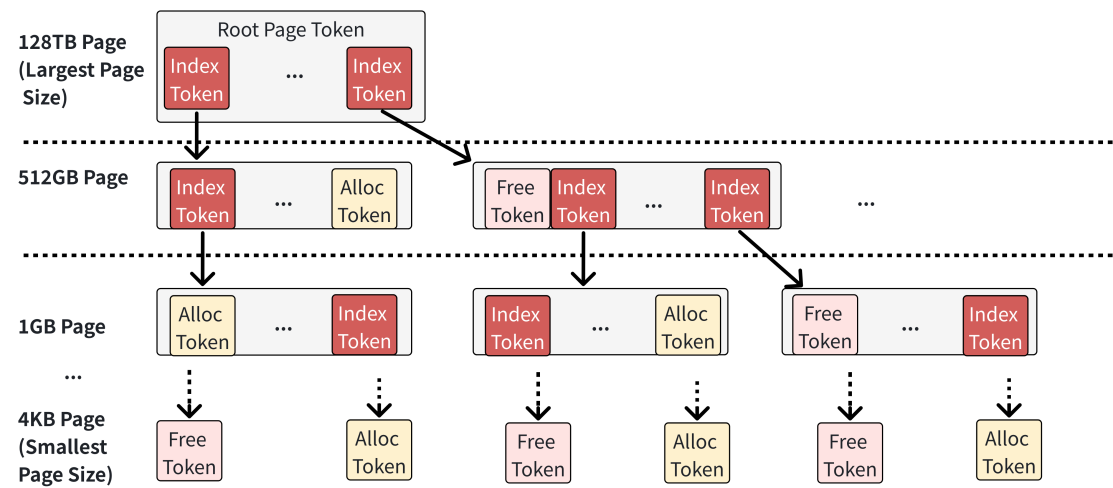


图 4-2 PageAllocator 结构

表和 `Btree` 的模式管理和索引内存块，以 `RISC-V` 硬件支持的物理页大小划分层级，其中每个节点称为 `page token (PTk)`，每个 `PTk` 包括：（1）物理页地址和范围；（2）子 `PTk` 数组：管理将当前物理页进一步细分为子物理页对应的 `PTk`。`PTk` 可以进一步细分为三类：

- （1）**索引 `PTk` (Index Page Token, IPTk)**：用于索引的 `PTk`，并不用于分配，仅用于标记当前物理页管辖的子物理页被分配和使用。`IPTk` 会记录当前区域被分割后可以支持的单次内存分配最大大小（即当前内存区域最大物理页大小）以进行快速分配时检查，并为子 `PTk` 数组实际分配内存以存储子 `PTk` 状态。
- （2）**空闲 `PTk` (Free Page Token, FPTk)**：用于实际分配物理页的 `PTk`，因为仅标识当前物理页未被使用，所以未对子 `PTk` 数组实际分配内存。
- （3）**被分配的 `PTk` (Allocated Page Token, APTk)**：`APTk` 标识当前物理页

已经被分配并不可被使用。严格来说 APTk 不占用内存并且不具备实体，根据 Rust 的所有权机制，当一个 PTk 被实际分配之后，其所有权会被转移和消耗，同时其使用的所有资源会被释放，后续该 PTk 不可被再次使用以避免不安全的二次分配。

分配器初始化：分配器初始化时会对需要管理的安全内存进行基本检查，其起始地址必须至少对齐 4KB 物理页。然后分配器会根据当前内存区域大小设置根 PTk 物理页大小，避免直接以 128TB 作为根 PTk 导致的内存浪费和递归深度增加。

接下来分配器将遍历安全内存并创建 PTk。为了尽可能减少 PTk 数量，分配器将计算当前地址对齐的最大物理页大小并创建 FPTk，同时递归创建或查找父 IPTk 以存储新的 FPTk。当安全内存遍历完成后，分配器初始化完成，可以使用。

PTk 生命周期管理：初始状态 PTk 仅包括用于实际分配的 FPTk 和用于索引的 IPTk。当申请分配物理页时，分配器按照以下逻辑处理：（1）从根 PTk 开始广度优先遍历查找第一个符合内存需求的 FPTk；（2）若得到的 FPTk 对应物理页刚好满足，则直接消耗当前 FPTk（转为 APTk）并返回物理页；（3）若大于所需物理页，则将当前 FPTk 拆分为多个更小的子 FPTk，将当前 FPTk 转变为 IPTk 并存储创建子 FPTk，然后继续对子 FPTk 重复（2）（3）操作。

当释放物理页时，（1）从根 PTk 开始，计算当前物理页在安全内存中的偏移并转为 IPTk 的子 PTk 索引；（2）索引到物理页偏移和大小均符合的子 PTk 位置，然后使用物理页创建新的 FPTk 并存入数组；（3）若当前 IPTk 下所有子 PTk 均为 FPTk，则消耗所有子 PTk 并执行合并，将当前 IPTk 转换为 FPTk。

4.2.2 内存保护原语

表 4-2 EasyAda-SM 内存保护原语

原语	功能
alloc	申请一个安全硬件用于隔离安全区域，返回 ID
free	根据 ID 释放一个安全硬件
disable	根据 ID 关闭指定安全硬件
enable	根据 ID 使能指定安全硬件
grant_access	在指定 CPU/hart 集合中使用指定安全硬件授权访问指定内存区域
retrive_access	在指定 CPU/hart 集合中使用指定安全硬件限制访问指定内存区域
is_protectable	检查指定内存区域是否可以被保护

如表4-2所示，内存保护原语抽象了隔离安全区域所需操作，包括安全硬件管

理和限制/授权指定内存区域访问。

因为不同的安全硬件和内存隔离机制间存在较大差异，如 PMP 机制下每个条目直接保护连续内存区域，而 MPT 条目则存储地址空间的权限表，两者的管理模式完全不同。因此 SMPt 不对安全设备管理进行约束，仅要求每种实现显式告知 SRMM 安全区域与硬件的关联关系（alloc/free 原语），以及根据关联关系实现对指定内存区域的访问权限控制（grant/retrieve）。

因为不同执行环境间通常拥有各自独立的 hart，因此 RISC-V 下硬件内存保护机制通常以 hart 为单位，每个 hart 拥有独立的硬件且仅负责配置当前 hart 内存保护视图。为了实现全局统一的内存保护视图，访存控制配置需要同时同步到所有相关 hart，以安全区域为例，SM 需要同时完成在不可信组件使用的 hart 集合中限制访问安全内存和在 enclave 使用的 hart 集合中授权访问安全内存，因此 grant 和 access 以 hart 集合为单位进行配置，不同 hart 间通过核间中断和原子操作确保配置的一致性。同时，具体的 hart 集合由更上层的 enclave 管理模块管理，内存保护原语仅负责同步配置到指定 hart 集合。

因为 EasyAda-trust 主要使用 RISC-V PMP 机制保护安全内存，因此下面主要介绍基于 PMP 的内存保护原语实现。

4.2.2.1 基于 PMP 的内存保护原语实现

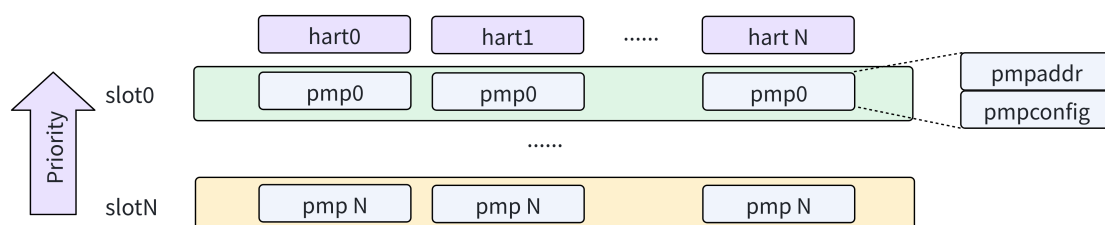


图 4-3 PMP 槽结构

PMP slot: RISC-V 下每个 hart 有独立的 PMP 条目和相关配置，仅控制当前 hart 内存访问。为了实现所有 hart 拥有统一的安全内存视图，如图4-3所示，PMP 管理器以 PMP slot 为单位管理 PMP 硬件，每个 PMP slot 是所有 hart 上索引相同的 PMP 条目的集合。

PMP 管理器: EasyAda-SM 定义的 PMP 管理器使用位图管理 PMP slot，为了提高灵活性，位图中包含三个掩码：（1）当前管理器管理的 PMP slot 索引的掩码，标记当前 PMP 管理器管理的所有 PMP slot 的索引，运行时不可变；（2）当前管理器可分配的 PMP slot 索引的掩码，标记当前 PMP 管理器总共可分配给安全区域的 PMP slot 索引，运行时不可变；（3）当前管理器已分配 PMP slot 索引的掩码，标

记当前 PMP 管理器中已分配 PMP slot 索引，运行时动态修改。

在初始化时，开发者初始化当前管理器可使用和管理的 PMP slot 索引掩码（两者的区别在于，可使用的 PMP slot 可以动态分配，用于隔离安全区域，而被管理的 PMP slot 同时包括可使用和开发者预留的 PMP slot，如最高权限的 PMP slot 被用于保护 SM，虽然该 slot 可以被配置，但是不能被使用）。在配置时，安全区域可以选择需要配置的 hart 的集合，并在这些 hart 上配置关联的 PMP slot 限制/授权访问安全内存。

授权/限制访问内存：上层模块可以自由决定本次授权/限制操作覆盖的 hart 范围。当前请求权限控制时，上层模块需要传入需要控制的内存区域，需要操作的 hart 集合以及被使用的安全硬件 ID。PMP 管理器会对安全硬件可用性进行基本检查，然后在当前 hart 上配置对应的安全硬件，如果涉及多个 hart，则需要通过 RISC-V 下软件中断向远端 hart 发起请求，确保配置的原子性和一致性。

4.2.3 安全内存管理原语

表 4-3 EasyAda-SM 安全内存管理原语

原语	功能
new	创建新的 SMM
init	初始化 SMM
deinit	重置软硬件状态并销毁 SMM
extend	创建一个新的安全区域
reclaim	回收一个空闲安全区域
alloc_em	从符合 enclave 类型的安全区域中申请 enclave 内存
free_em	释放 enclave 内存到对应安全区域中
grant_access	在指定 hart 集合上授权访问指定内存区域
retrive_access	在指定 hart 集合上限制访问指定内存区域

前文介绍了 SRMM 对内存管理和保护原语的抽象和实现，本节将介绍 SRMM 对安全内存管理原语的抽象和实现。如表4-3所示，安全内存管理原语总体上可以分为 SMM 生命周期管理，安全区域生命周期管理，enclave 内存管理和内存保护四类。

SMM 生命周期管理：SMM 作为 SRMM 的顶级管理单元，负责统筹安全内存资源和安全硬件资源，并使用安全硬件保护安全内存。在 init 阶段，SMM 预期完成安全硬件的初始化和关键安全区域的初始化（尤其是 SM 本身的代码和数据），此时 SMM 中不存在可以被用于分配 enclave 内存的安全区域，后续可以在 SMM 中动态拓展/回收安全区域。在 deinit 阶段，SMM 会检查所有安全区域是否可回收

(未使用)，为了确保内存回收不会异常中断 **enclave** 运行，SMM 要求 **enclave** 管理模块主动完成所有 **enclave** 的清理后才能发起 **deinit** 请求。同时，因为内存清理已经在 **enclave** 内存释放时完成，因此无需二次清理以避免敏感信息泄漏。然后 SMM 会消耗所有安全区域的所有权并记录安全内存信息归还到不可信 **host**。

安全区域生命周期管理：安全区域的初始化分为三个部分：内存分配，安全内存隔离，以及内存分配器初始化。内存分配通常由不可信的 **host** 完成；在拓展安全区域时，SMM 会首先检查当前安全硬件是否支持保护安全区域，以及是否与已有安全区域重叠，确保不会破坏 **enclave** 间内存隔离。然后 SMM 会尝试从 **SMPt** 中申请一个安全硬件，使用该安全硬件隔离安全内存和不可信组件，此时安全内存仅能被 SMM 访问，最后使用安全内存初始化安全区域，设置当期区域为未使用，完成安全区域初始化，但是内存分配器初始化时机由 SMM 自行决定，本文将在后文展示延迟初始化的实现；在回收安全区域时，SMM 会检查安全区域是否是未使用状态，**enclave** 管理模块必须主动销毁使用当前安全区域的 **enclave**，然后才能安全回收内存。

Enclave 内存生命周期管理：SMM 最终的目标是管理 **enclave** 分配，当 **enclave** 管理模块申请 **enclave** 内存时，需要告知 SMM 当前 **enclave** 类型以选择合适的 **安全区域** 及 **内存分配器**，SMM 会遍历所有安全区域查找符合类型和内存需求的安全区域，然后尝试申请 **enclave** 内存，如果申请成功则返回内存信息和安全区域编号，否则继续重复该操作直到遍历所有安全区域；当释放 **enclave** 内存时，**enclave** 管理模块需要提供 **enclave** 内存信息，由 SMM 负责遍历所有安全区域并查找 **enclave** 内存属于的安全区域，然后通过安全区域的内存分配器释放 **enclave** 内存。

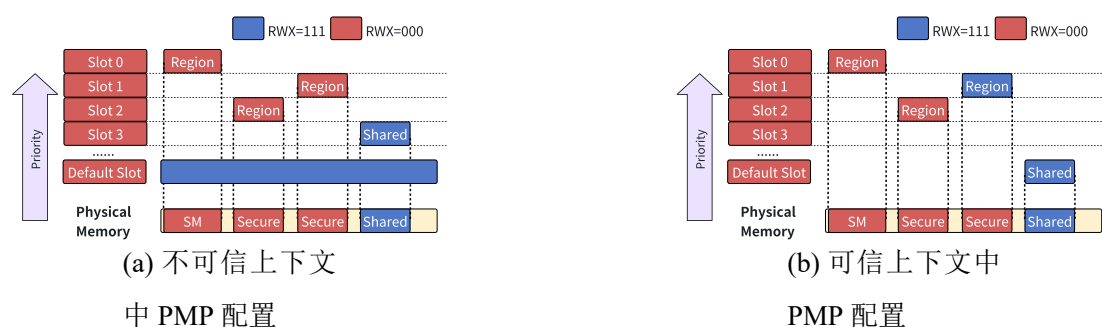


图 4-4 PMP 运行时配置

内存保护：如图4-4所示，本段以 PMP 为例介绍不安全上下文和 **enclave** 上下文间切换是内存保护配置。对于不可信的上下文，SM 会通过 **retrive** 限制 **hart** 访问任何安全区域，其它内存可以正常访问；而对于可信上下文，SM 会通过 **grant** 授予 **hart** 访问 **enclave** 内存所属的安全区域访问权限。因为复杂 **enclave** 可能使用

多个不连续的 enclave 内存和多个 hart，需要在每个 hart 上授权访问多个安全区域，并通过 hart 生命周期管理接口实现其它 enclave 使用的 hart 上下文的安全切换。

这里需要额外关注 default slot，该 slot 用于赋予任何 hart 默认访问所有内存区域的权限，避免因为保护安全内存导致 hart 无法访问不可信内存，图4-4(b)展示了 OS enclave 的管理方式，因为其具备配置 MMU 的能力且直接运行在内核态下，需要修改 default slot 禁止访问所有物理内存空间，而对于 App 和 VM enclave 则不强制要求，因为 enclave 与 MMU 相互隔离。

下面将详细介绍 EasyAda-SM 针对多种 enclave 抽象提供统一管理的安全内存管理原语实现。

4.2.3.1 面向通用 enclave 部署场景的安全内存管理原语实现

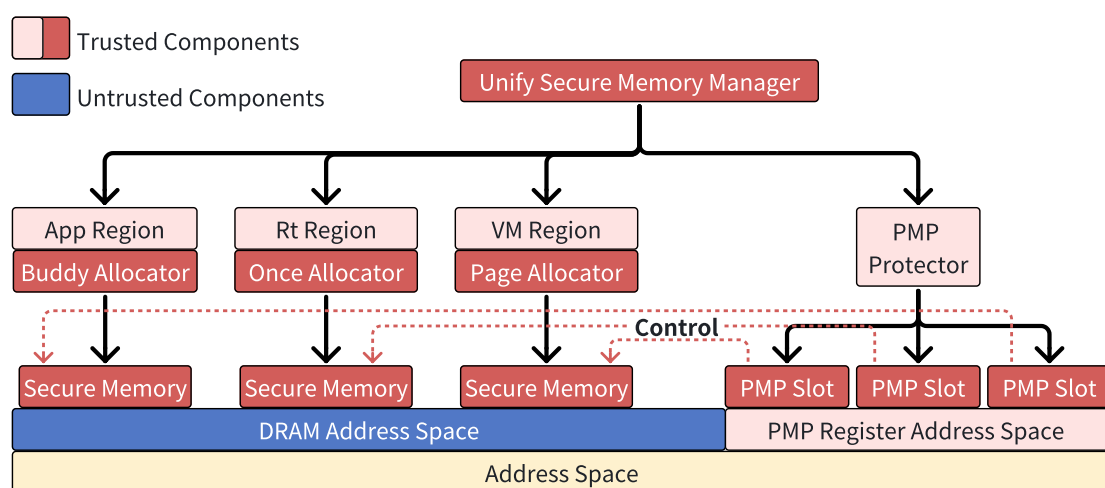


图 4-5 PageAllocator 结构

为了支持通用的 enclave 管理，降低用户开发难度，EasyAda-SM 实现了如图4-5所示的通用安全内存管理器（USMM），将 enclave 和安全区域划分为一一对应的五类：（1）Application （2）Runtime （3）VirtulaMachine （4）Devices（Devices 会进一步划分为串口，硬件加速器等设备类型）（5）None，并且如图??所示，为每类 enclave 绑定了特定的内存分配器。具体来说，不同 enclave 内存与不可信组件的隔离机制相同均基于 PMP 实现，区别在 enclave 内部内存管理和 PMP 权限配置不同。因此 EasyAda-SM 引入了通用安全区域表示多种安全区域状态，为了符合 Rust 的类型检查要求，EasyAda-SM 定义了包含所有分配器实现的枚举（类似 C/C++ 中的 union），安全区域通过在不同的分配器枚举间切换实现状态切换。并且状态切换按照以下状态机进行。

安全区域状态转换：初始状态下，所有安全区域均为未使用状态，使用空分配

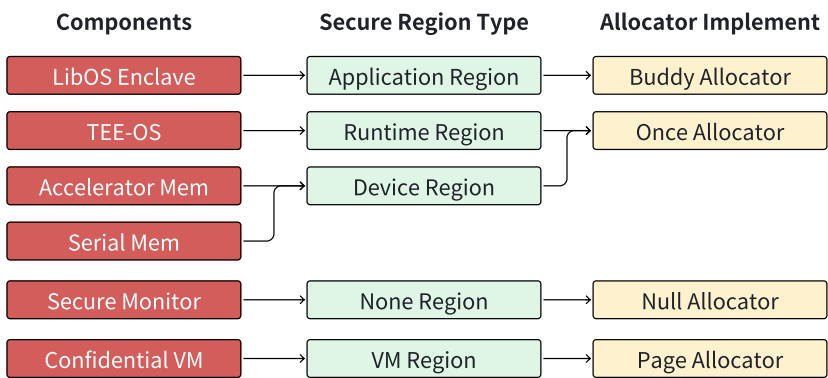


图 4-6 不同类型 Enclave 与区域类型和分配器的关联关系

器管理内存，仅实现内存隔离，无法进行任何内存分配；当申请 enclave 内存时需要显式指定 enclave 类型，USMM 将 enclave 类型划分为五类并为每一类绑定了一个 USMM 会遍历获取内存需求匹配的安全区域，若当前类型匹配则直接分配，或者当前区域未被使用则转化为目标 enclave 类型区域，初始化内存分配器并尝试分配内存，否则继续查找符合条件的区域；当释放 enclave 内存时，USMM 会遍历查找包含 enclave 内存的安全区域并尝试释放内存，若释放后安全区域无内存分配，则消耗当前安全区域所有权，转换为未使用安全区域。

除内存管理外，不同 enclave 抽象在安全区域保护逻辑上存在较大差异，但是该部分不属于 USMM 的职责范围，下一节将介绍如何基于 USMM 实现面向 CVM 的 enclave 管理模块。

4.3 面向 CVM 的 enclave 管理模块实现

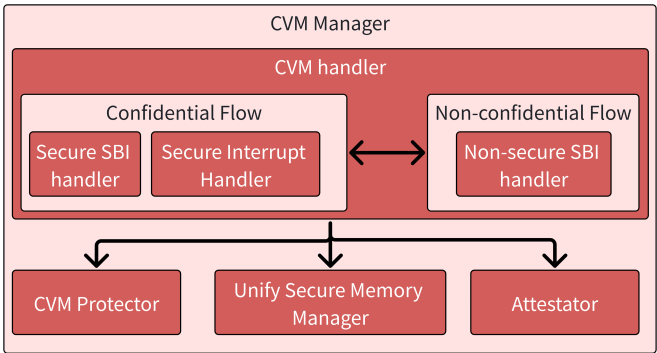


图 4-7 CVM 管理器结构

如图4-7所示，CVM 管理模块包括四个主要模块：

- (1) **通用安全内存管理器**：如前文所说，通用安全内存管理器目标是实现安全内存与不可信组件间的隔离，负责统一管理安全设备和安全内存区域。

- (2) **CVM 保护器**: CVM 保护器的目标是实现 CVM 间的地址空间隔离。因为 CVM 管理模块使用二阶段地址翻译机制在 CVM 间实现地址空间隔离的, 因此该组件实际负责 CVM 的二阶段页表管理。
- (3) **CVM 证明器**: CVM 证明器的目标是实现 CVM 的可信启动, 负责完成 CVM 初始状态度量, TAP 解密和密钥管理。
- (4) **CVM 处理程序**: CVM 处理程序是 CVM 管理模块的统一入口, 负责实现对不可信组件和 CVM 所有异常和中断的拦截和处理, 机密处理流和非机密流间敏感数据的隔离和安全上下文切换。

可以将该模块视为 TSM 驱动的一种实现, 后文直接以 TSM 驱动代指 CVM 管理器。

4.3.1 Enclave 间内存安全

Enclave 间内存安全的核心是隔离不同 enclave 的地址空间, 安全内存管理器实现了不可信组件和安全内存间的隔离, 如果直接基于其实现 CVM 间隔离会快速耗尽可用安全硬件, 减少可管理的 CVM 数量。因此 TSM 驱动利用 CVM 基于硬件虚拟化拓展运行的特点, 引入了 CVM 保护器基于二阶段页表管理和实现 enclave 间内存安全。

CVM 保护器的核心职责是维护当前 CVM 的二阶段页表, 包括初始化 CVM 二阶段页表, 映射/解映射安全内存和不可信内存以及使用二阶段页表翻译得到物理地址。为了避免 CVM 保护器耗尽 SM 的堆内存, CVM 保护器构建二阶段页表使用的内存主要来自于从安全内存管理器中申请的内存。为了提高系统内存利用率, CVM 保护器允许从多个安全区域中申请, 因为二阶段地址翻译和页分配器保证同一物理页仅能分配给一个 CVM 且 CVM 仅能访问指定的地址空间, 因此不会破坏 CVM 间内存隔离。

在 CVM 初始化时, CVM 保护器使用安全内存根据不可信 VM 的二阶段页表构建完全对等的 CVM 初始二阶段页表, 并将所有物理页中的内容拷贝到安全内存中, 该操作会耗费大量内存和 CPU 运行周期, 但是嵌入式场景下 CVM 部署之后通常会持续运行直到下一个电源周期, 而且安全性是 EasyAda-SM 的首要考虑目标, 因此 EasyAda-SM 认为在系统启动时该操作带来的延迟和性能降低是可以接受的。在系统运行时, CVM 保护器会负责二阶段页表的动态维护。

4.3.2 安全启动

安全启动的核心是确保只有被验证证明了可信且未被篡改的软件实体才能访问敏感资源。CVMM 引入了 attester 模块用于管理密钥并对 CVM 进行度量。

在系统开发阶段，EasyAda-trust 的硬件平台供应商会将硬件密钥固化在 RoT 中，并对上提供硬件公钥，可以进一步使用硬件公钥对 EasyAda-SM 的预期度量值进行签名。在系统初始化阶段，RoT 负责对 SM 进行度量并结合硬件密钥与数字签名进行对比，如果度量成功则转移控制权到 SM，并传递硬件密钥给 SM。该私钥后续会由 TSM 驱动用于 CVM 的安全启动。

在系统初始化时，CVMM 会初始化 attester 模块用于存储硬件私钥，并作为根密钥用于派生其他密钥。因为此时 SM 所在内存区域已经通过硬件实现隔离，因此硬件密钥无法被不可信组件窃取。

内存布局度量：当 hypervisor 请求提升 VM 为 CVM 并陷入 TSM 驱动时，TSM 驱动会尝试从安全内存中为 CVM 的 hart 分配元数据区域，用于存储 CVM 的 hart 上下文信息，并使用安全内存创建 VM 内存布局的安全副本，包括页表本身和物理页中的内容。然后 TSM 驱动将内存布局副本和 FDT 传递给 attester，由 attester 遍历内存布局副本并根据 FDT 对内存中的关键数据进行度量（包括 kernel，initrd 和 FDT 等）并生成运行时度量值，attester 并不关心非关键区域的内容，在度量过程中 attester 会负责清理所有非关键内存区域内容，避免不可信组件在其中存储恶意数据和代码。在内存布局度量完成后，attester 会将度量值存储到 CVM 在安全内存中的元数据中。

CVM 上下文度量：完成内存布局度量后，attester 会进一步度量 CVM 元数据区域中存储启动 hart 的初始上下文信息，包括通用寄存器和控制寄存器信息。因为除了启动 hart，其他 hart 的启动均由启动 hart 通过 hart 状态管理拓展（Hart State Management，HSM）拓展进行管理，初始上下文由 TSM 驱动进行设置和保护，所以无需进行度量。attester 同样会将初始上下文度量值存储在 CVM 对应的元数据中。

对比预期度量值：在系统初始化阶段，TSM 驱动从 RoT 中获取了硬件私钥。Attester 完成对 CVM 的度量之后，使用硬件密钥解密 CVM 开发者提供的锁盒，然后使用解密得到的对称密钥解密 TAP，得到 CVM 开发者提供的数字签名和密钥。如果使用硬件私钥签名的 CVM 上下文和内存布局度量值与数字签名匹配，则度量通过，当前 CVM 被纳入可信链中。

密钥管理：除了硬件私钥外，attester 还需要保护和管理 CVM 开发者提供的密钥。这些密钥通常会用于（1）CVM 解密机密应用或根文件系统等（2）作为根密钥直接或衍生密钥对网络，串口和磁盘块设备等 I/O 数据进行加解密。CVM 可以在运行时通过 SBI 请求从 TSM 驱动获取密钥。

4.3.3 中断和异常安全

表 4-4 CVM 管理模块在机密流支持的句柄类型

类别	功能描述
标准 SBI 拓展	包括（1）探测 SBI 基础扩展支持，获取硬件基本信息（2）发送处理器间中断，远程刷新指令缓存和虚拟内存地址等（3）Hart 状态管理，启动、停止、挂起机密 hart，查询 hart 运行状态等（4）系统控制，请求系统重置或关机
I/O 虚拟化拓展	（1）MMIO 服务请求，向不可信 host 发起 MMIO 读写请求（2）不可信的共享内存管理，向不可信的 host 请求共享内存，用于实现 I/O 半虚拟化（3）安全设备管理，请求直接映射/移除安全设备物理内存地址到 CVM，用于实现设备直通。
密钥管理	从 TSM 驱动中获取 CVM 的特定密钥等机密数据
机密异常处理	处理 CVM 二阶段地址访问异常，虚拟指令等

表 4-5 CVM 管理模块在机密流支持的句柄类型

类别	功能描述
标准 SBI 拓展	所有 RustSBI 实现的标准 SBI 拓展，由 CVM 处理程序调用 RustSBI 的接口实现。
CVM 生命周期管理	实现不可信 host 对 CVM 生命周期的管理，包括将普通 VM 提升为 CVM，运行 CVM 和销毁 CVM。
I/O 虚拟化管理拓展	（1）MMIO 请求回复，回复 CVM 的 MMIO 请求处理结果。（2）共享内存请求回复，回复 CVM 共享内存请求处理结果以及分配的共享内存信息。
安全区域管理拓展	管理安全区域的创建和回收，不可信 host 通过此拓展捐赠内存创建新的安全区域/回收空闲安全区域使用内存

中断和异常安全的核心是确保机密流和非机密流之间的敏感数据互不泄露，并且阻止不可信组件通过中断或异常攻击 CVM。

在标准流程中，M 态中断和异常的入口（trap handler）由 SBI 固件实现和初始化。而在 EasyAda-SM 中，CVMM 会修改 M 态 trap handler 以确保 CVMM 负责响应和监控所有异常和中断，再根据需要调用 RustSBI 固件提供的能力。这样的好处是无需在执行环境切换时重新配置 trap handler 以实现对 CVM 中断和 SBI 请求的接管和确保 CVMM 可以监控所有敏感操作。

对于非 TEE 环境，即标准的 hypervisor 和不可信 VM 流程，中断和异常默认

委派到 HS 态进行处理（**需要由 enclave 管理的设备的外部中断必须在 M 态下进行处理**），hypervisor 可以进一步委派中断到 VS 态进行处理。但是对于 TEE 环境，默认情况下 CVMM 会将所有 VS 态中断委派到 CVM 中，但是其它中断均需要陷入 M 态。而对于机密流下的异常，大部分会委派到 CVM 中进行处理，而 GUEST_PAGE_FAULT 等异常则直接陷入到 M 态进行处理。

不管是异常还是中断，最后都会进入 CVM handler 进行统一处理。CVM handler 实现了所有 CVMM 的对外提供的服务接口，如表4-4和4-5所示，根据 hart 当前状态划分为机密流和非机密流两类，当前 hart 如果运行在 enclave 中，则会进入可信流处理，反之则进入不可信流处理。

4.3.3.1 机密流和非机密流

因此，CVM handler 的核心工作之一是实现不可信组件和 CVM 上下文间的安全切换。EasyAda-SM 将其 hart 的上下文切换分为**普通上下文切换**和**安全上下文切换**两个阶段。CVM/hypervisor 进入 SM 时首先会执行普通上下文切换，保存当前执行必须的通用寄存器和配置寄存器，然后进行处理。如果需要切换执行环境，即跨处理流处理时，CVMM 会进行安全上下文切换，将 hart 当前所有执行状态保存到安全内存中，同时加载目标上下文的执行状态，并且，CVMM 还会通过 USMM 执行关键的**内存访问权限调整**和清理操作（如 TLB），避免当前软件实体的敏感信息泄漏给将要执行的一方，并确保正确配置将要执行的软件实体相关内存资源的权限。

4.3.4 IOPMP 管理

为了防止支持 DMA 的恶意外设绕过 CPU 的内存隔离直接访问安全内存，EasyAda-trust 使用 IOPMP 拓展限制了每个硬件可以访问的内存区域。在开发时，硬件供应商需要为可信设备静态分配 SID 作为唯一标识符号。

在系统启动时，TSM 驱动创建安全区域是会同时完成对最高特权级的 IOPMP 条目（SM 条目）的配置，限制任何设备对 SM 内存区域的读写，同时配置最低优先级的条目（default 条目）赋予设备默认访问所有不可信内存区域的权限。此时设备可以正常使用，但是无法读写 SM 中的代码和数据。同时，TSM 驱动会为所有设备的 SID 创建，绑定并启用一个用于限制访问的 MD，初始状态下每个 MD 中仅包含 SM 和 default 条目。

图4-8展示了 EasyAda-trust 使用 IOPMP 控制设备访问安全区域的流程。在系统运行时，EasyAda-trust 以 **IOPMP 条目对**的形式管理 IOPMP 硬件，每当一个安全区域创建时，TSM 驱动会按序申请第一对空闲的 $2n-1$ 和 $2n$ 表项， $2n-1$ 用于授

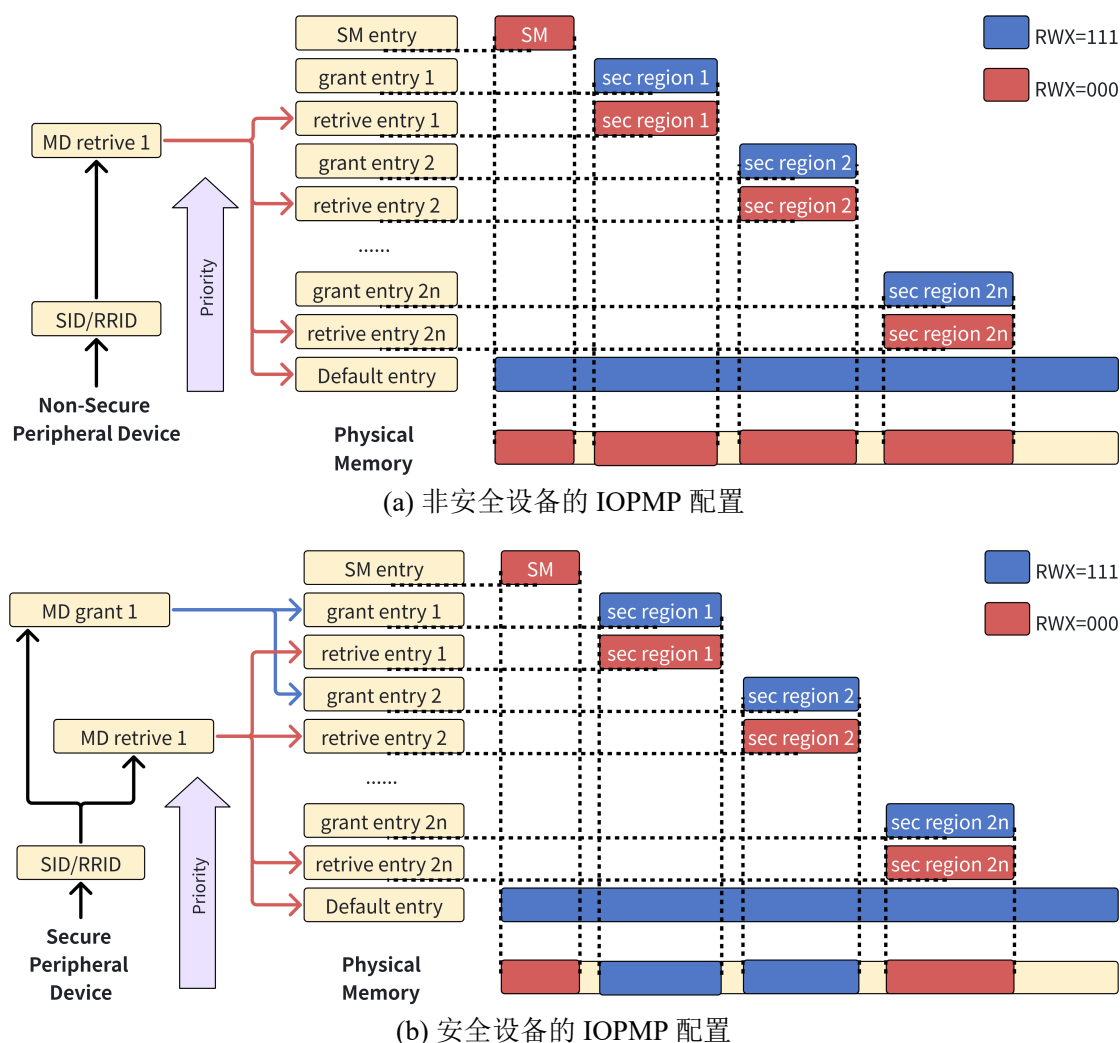


图 4-8 EasyAda -SM 中非安全/安全设备的 IOPMP 管理

权访问安全区域的，2n 则用于限制访问安全区域。如图4-8(a)所示，对于不安全的设备，每当 TSM 驱动创建新的安全区域和 IOPMP 表项时，会同步当前配置到所有设备的限制 MD 中，默认限制所有设备对安全区域的读写。

同时，hypervisor 可以在任何时刻请求将可信设备提升为安全设备，TSM 驱动会通过可信的硬件配置文件验证设备可信，并创建安全区域隔离安全设备的 MMIO 区域，以及为安全设备创建未初始化的，用于授权访问的 MD。当安全设备实际分配给 CVM 时，TSM 驱动会将 CVM 关联的所有安全区域的授权条目加入设备的授权 MD 中，并切换安全设备的 MD 为。如图4-8(b)所示，此时安全设备仅能访问 CVM 关联的安全区域，杜绝了安全设备将敏感数据写入不可信内存的隐患。

TSM 驱动仅在设备提升为安全设备时，或从安全设备回退为不安全设备时进行重新配置，避免频繁切换 IOPMP 配置时上下文切换和缓存一致性带来的性能开销。

4.4 本章小结

本章详细论述了基于 Rust 语言的内存安全固件 EasyAda-SM 的实现。通过充分利用 Rust 的内存安全特性，方案从根源上消除了安全监视器中的内存破坏漏洞，并借助模块化设计的安全区域与内存管理层，实现了物理内存的细粒度原语抽象。在虚拟机管理层面，本章介绍了面向 CVM 的 enclave 管理模块的实现，集成了基于 PMP 的强隔离原语、本地证明安全启动及 TSM 驱动下的安全中断异常处理路径。此外，针对异构计算场景，方案实现了基于 IOPMP 的动态管理机制，通过对 I/O 访问路径的统一监控与隔离，在保障硬件加速器高效复用的同时，构建了防御恶意 DMA 攻击的安全底座。

第五章 基于微内核的可信虚拟化方案实现

5.1 安全 CPU 虚拟化实现

EasyAda-hypervisor 将 vhart 设计为微内核操作系统中的线程进行调度,其 TCB 中除了标准的通用寄存器信息外,额外记录了当前 VM 的控制寄存器信息用以准确恢复 VM 的运行上下文。同时, EasyAda-hypervisor 为每个 vhart 实现了两个独立的上下文: vhart 运行线程和 vhart 服务线程,运行线程负责存储/调度 VM 运行时上下文,服务线程则负责实现 VM 的异步 I/O 请求。

为了提升系统整体性能, EasyAda-hypervisor 将物理 hart 静态划分为三类功能区: (1) vhart 运行区, 主要承担 VM 及其应用的计算任务。(2) vhart 服务区, 主要用于运行 vhart 服务线程, 用于异步处理 VM 的 I/O 请求, 确保 VM 的 I/O 请求能尽快得到响应并减少对 VM 计算任务的影响。(3) 系统服务区, 主要用于运行微内核关键的系统服务, 如内存和进程管理, 文件系统服务等, 避免因调度抖动导致系统整体性能下降。

EasyAda-trust 拓展了 EasyAda-hypervisor 的 CPU 虚拟化实现, 引入了额外的安全机制实现 CVM 和 hypervisor 间上下文的隔离。

5.1.1 安全上下文管理

EasyAda-trust 中, TSM 驱动不介入不可信 VM 的 vhart 管理, 仍由 hypervisor 负责, 而 CVM 的 vhart 调度虽然由 hypervisor 负责, 但是其上下文必须由 TSM 驱动安全管理并对 hypervisor 不可见, hypervisor 仅能通过 TSM 驱动对外提供的 CVM 生命周期管理接口实现对 CVM 的 vhart 的调度。

如图5-1所示, EasyAda-trust 在原有 hart 分区的基础上将 vhart 运行区进一步划分为 CVM vhart 运行区和不可信 VM 运行区, 两者主要的差异在与执行环境间上下文切换流程。

5.1.1.1 安全上下文切换

为了避免不必要的上下文保护和恢复开销, EasyAda-hypervisor 将 vhart 上下文保护划分为两个阶段: 执行环境内的轻量上下文切换和执行环境间安全上下文切换。对于不可信 VM, hypervisor 在逻辑上将其上下文划分为 VM 侧的 VM 上下文, 以及 hypervisor 侧的 vhart 运行上下文, 两者各自拥有完整的 (1) 最小架构状态寄存器集合 (如通用寄存器和 TSM 驱动运行时可能修改的控制寄存器), (2) 虚

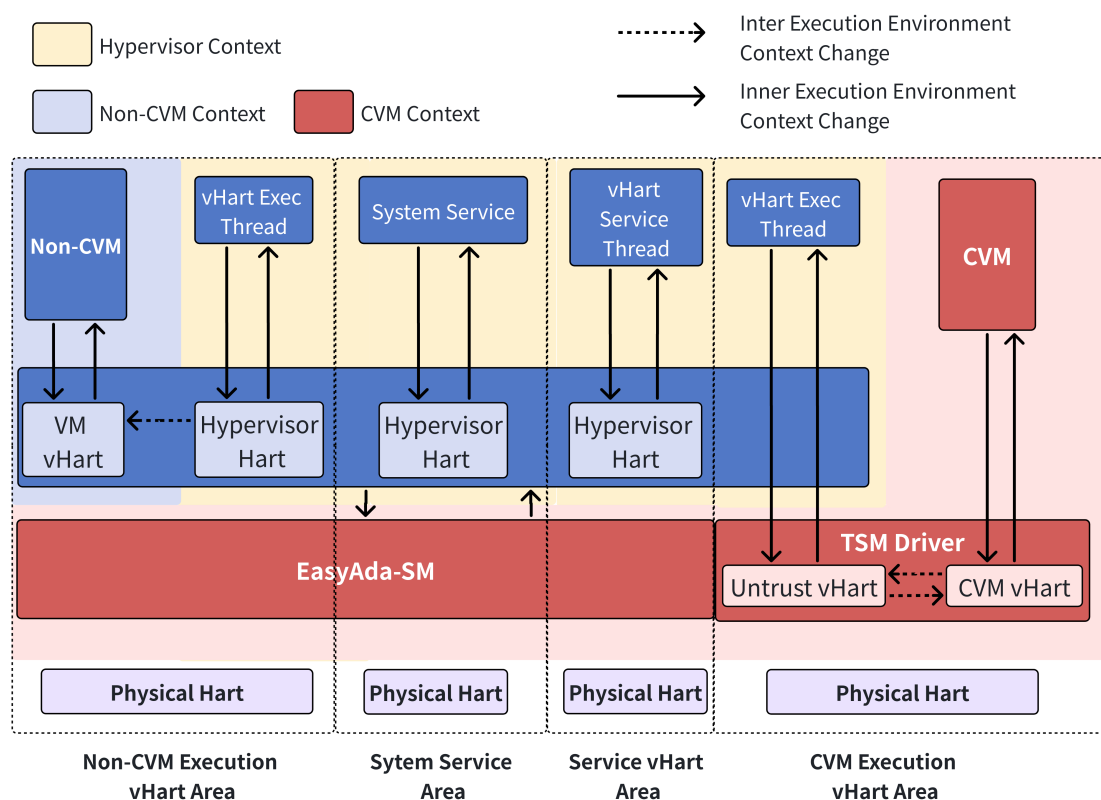


图 5-1 EasyAda-trust 下 hart 分区结构图

虚拟化控制寄存器信息，（3）虚拟化功能寄存器信息。但是因为 hypervisor 运行时不会修改（2）和（3）中的信息，所以 VM 与 hypervisor 间上下文切换仅需保存（1）中的信息即可，而 VM 间切换时需要完整保存并清理（1）（2）（3）在内的所有上下文信息。因此 hypervisor 使用 vhart 运行线程同时管理两类逻辑上下文的信息。

CVM 的上下文切换具备以下特征：

- （1）**更多的上下文信息：**为了保护 CVM 上下文信息，CVM 不信任任何来自 hypervisor 的上下文信息，因此除了前文提到的三类上下文信息，TSM 驱动需要额外切换 M 态和 HS 态的控制寄存器信息（如中断和异常委派状态信息等）。
- （2）**隔离的物理内存视图：**CVM 和 hypervisor 使用相互隔离的物理内存视图。在上下文切换时 TSM 驱动还需配置安全硬件实现安全内存与不可信组件的隔离。
- （3）**安全的执行流切换：**CVM 和 hypervisor 的执行流相互隔离，在特定请求场景下如 MMIO 异常时会触发执行流切换。与不可信 VM 不同，hypervisor 在执行流切换不能获取任何 CVM 和 TSM 驱动上下文信息。

针对 CVM 上下文切换的特点，EasyAda-trust 将 CVM 上下文划分为不可信

vhart 和机密 vhart 两类,前者负责存储 hypervisor 侧 vhart 运行线程所有上下文信息,后者负责存储 CVM 所有的上下文信息,两者均包括所有通用寄存器和 VS/HS/M 态系统控制寄存器信息,如果当前环境支持浮点或向量计算指令拓展,则还需要额外保存浮点/向量处理单元的上下文信息。

为了减少 CVM 和 hypervisor 间上下文切换带来的性能开销, CVM 的上下文切换仍然分为两个阶段进行。参考上一章的 CVM 管理模块章节,若当前 hart 执行流不跨运行环境切换,则仅进行轻量上下文切换,否则执行完整的安全上下文切换流程。

轻量上下文切换:当前 hart 从 hypervisor/CVM 陷入 TSM 驱动中执行,由 TSM 驱动直接完成请求处理, hart 仅需保存当前环境的通用寄存器信息,由 TSM 驱动完成处理后恢复。因为不涉及执行环境切换,无需修改当前物理内存保护视图,

安全上下文切换:当前 hart 需要在 hypervisor 和 CVM 间转移控制权,则需遵照 CVM 管理模块的有限状态机,完成参数的安全传递后,由 TSM 驱动保存全部的上下文信息到对应的 vhart 中,清理当前环境,然后完成下一个执行环境的上下文恢复,最终完成执行流切换。在实际切换前,还需重新配置安全硬件以切换物理内存视图。由不可信 vhart 切换到机密 vhart 时, TSM 驱动会授权当前 hart 访问除 SM 自身外所有的安全区域和不可信内存,同时因为二阶段页表的存在, TSM 驱动可以完全控制 CVM 的地址访问;相对的,当机密 vhart 切换到不可信 vhart 时, TSM 驱动会限制当前 hart 对所有安全区域的访问。

5.1.2 安全异常处理

根据4.3.3小节的叙述, EasyAda-SM 通过 Rust 静态内存检查消除了不安全内存访问可能导致的敏感数据泄漏问题,并通过构造->处理->析构的流程严格约束了每个处理器可以访问的能力和资源范围。同时, TSM 驱动没有对外提供写入不可信组件内存的能力,确保其无法泄漏数据到不可信组件中。

但是必须注意的是,虽然 TSM 驱动可以确保不同处理器和执行流之间的安全,但是 CVM 负责正确的使用 TSM 驱动提供的处理器,如 TSM 驱动支持为 CVM 分配不可信的共享内存以实现 I/O 半虚拟化,但是不能也无法对数据提供机密性和完整性的保证, CVM 需要自行对数据进行加解密和度量以确保数据使用符合预期。

5.2 安全内存虚拟化实现

内存虚拟化主要涉及虚拟机运行时内存,设备 MMIO 区域和用于 I/O 半虚拟化的共享内存的二阶段地址映射等,二阶段页表的根页表地址作为虚拟机配置信

息的一部分保存在 vhart 运行线程的上下文中。EasyAda-Hypervisor 需按照以下流程建立和维护二阶段页表：（1）用户态 VMM 根据当前 VM 的配置文件如 FDT 计算 VM 所需运行时内存（除 MMIO 区域外的内存），通过系统服务获取足够的物理内存。（2）根据配置文件将 VM 的 kernel, FDT 等加载到符合对齐要求的地址中。（3）根据配置文件计算 GPA 到 HPA 的映射关系，通过 hypervisor 驱动创建初始初始二阶段页表并存储在 vhart 的上下文中。

MMIO 区域映射主要用于直通设备的管理。由内核态下 hypervisor 驱动在创建 VM 时，根据配置文件中解析得到的物理设备信息，并依据 IPA 和 HPA 的映射关系将物理设备直接映射到 VM 的地址空间中，并且将该设备的中断号注册至虚拟机，以确保后续的虚拟中断注入能够正确触发，从而使虚拟机能够正常访问直通设备并处理其中断事件。

共享内存映射主要用于基于半虚拟化技术的虚拟设备管理。在 VM 在系统运行过程中访问虚拟设备 MMIO 区域的 GPA 触发异常时进行初始化。内核态下的 hypervisor 驱动拦截到 VM 的异常后与虚拟设备信息进行匹配，若属于虚拟设备 MMIO 区域，则切换到 vhart 运行上下文并在用户态下向系统服务申请对应的物理共享内存，最终由 hypervisor 驱动在内核态下完成共享内存到 VM 地址空间的映射，完成异常处理并切换到 VM 上下文继续运行。EasyAda 对共享内存以 UNIX 风格的文件进行管理，对于 VM 则文件的所有者为 vhart 运行线程，而通常会映射到 vhart 服务线程中以实现异步 I/O。

EasyAda-trust 未对不可信 VM 的内存虚拟化流程进行修改，仅将 CVM 的内存虚拟化流程与 hypervisor 分离，hypervisor 仅负责创建安全区域和分配共享内存，主要的安全内存/设备分配和二阶段映射工作则由 TSM 驱动负责。

5.2.1 安全区域生命周期管理

如第三章所述，EasyAda-trust 强制要求 CVM 除用于半虚拟化 I/O 的共享内存外的所有内存必须使用安全内存，而安全内存全部从安全区域中分配获得。为了提高系统内存的利用率，EasyAda-trust 要求 hypervisor 通过 TSM 驱动提供的安全区域管理拓展实现运行时动态创建和回收安全区域。

5.2.1.1 安全区域创建

系统早期初始化时仅存在一个不可读写的安全区域负责保护 SM 自身。系统运行时，hypervisor 需要主动从系统服务中申请连续的物理内存（因为 PMP 仅支持保护连续的物理内存）或物理设备的 MMIO 区域并翻译为物理地址，然后通过 TSM 驱动完成安全区域创建和初始化。在创建过程中，TSM 驱动会根据当前硬件

环境的 FDT 检查传入内存区域合法性，只有在 FDT 中存在且合法的内存/设备区域能够用于创建安全区域，TSM 驱动会根据传入内存区域类型设置安全区域的类型。一旦安全区域创建成功，不可信组件无法再访问安全内存，hypervisor 负责确保内存保护不会中断当前正在执行任务。

为了避免 hypervisor 转换恶意设备/内存，TSM 驱动会存储可信的硬件配置文件，并且对 hypervisor 提供的内存地址进行匹配，只有由硬件配置文件标记可信的内存/MMIO 区域可以转换为安全区域。

5.2.1.2 安全区域分配

安全内存主要分为两类：安全内存区域和安全设备区域。安全内存区域可根据分配器类型细分使用场景，而安全设备区域可根据设备类型进行进一步细分。当 hypervisor 请求创建 CVM 时，TSM 驱动会从符合当前场景的多个安全区域中顺序申请足够的安全内存，用于容纳 CVM 镜像和二阶段页表等 CVM 元数据，并根据 CVM 的 FDT 尝试预分配安全设备区域，当 CVM 被证明可信后再进行实际分配和映射。

5.2.1.3 安全区域回收

如果 hypervisor 中内存或设备资源紧张，可以尝试从 TSM 驱动中回收空闲的安全内存/设备区域。TSM 驱动会查找未被使用的安全内存区域并回收当前安全区域资源，然后返回物理内存信息到 hypervisor。安全设备区域回收则需要 hypervisor 传入设备的 MMIO 地址，由 TSM 驱动尝试释放并归还设备到 hypervisor。

5.2.2 不可信共享内存映射

和不可信 VM 类似，CVM 使用的共享内存由 hypervisor 分配和管理（为了缩小 TEE 攻击面 EasyAda-trust 中安全内存不支持在多个 CVM 间共享，因此后文以共享内存代称不可信的共享内存）。与首要目标是安全性的安全内存相比，共享内存的首要目标是实现不可信组件与 CVM 间高效的数据交换，因此 TSM 驱动不对共享内存提供任何机密性和完整性的保障，仅确保映射过程的安全性，共享内存不可映射到任何已有映射范围，避免 CVM 泄漏敏感数据。如图5-2所示，共享内存主要用于软件实体间跨执行环境的数据交换。共享内存的请求和映射是两个异步的操作，在虚拟化场景下，共享内存的请求方通常是 CVM，通过虚拟机无感的内存访问异常或主动的 SBI 请求下限到 TSM 驱动中，由 TSM 驱动转发请求到 vhart 运行线程，后者作为共享内存文件的所有者完成共享内存文件的创建，并在 hypervisor 内部向 vhart 服务线程发起映射请求，要求对方完成对共享内存文件的

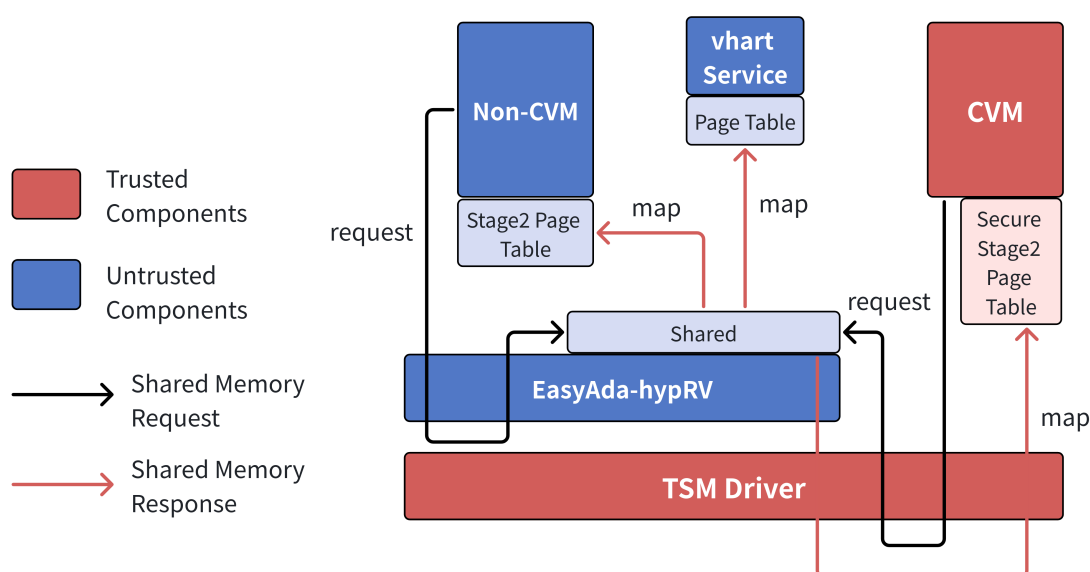


图 5-2 EasyAda-trust 共享内存创建流程

映射。然后由 `vhart` 运行线程发起 SBI 请求，告知 TSM 驱动当前请求已完成，最后由 TSM 驱动安全地映射共享内存到 CVM 的地址空间中，并切换到机密 `vhart` 继续运行。TSM 驱动仅介入共享内存控制流，即建立映射的过程，不感知上层对共享内存的读写操作（数据流），因此通信双方需要主动基于共享内存进行通信协议和信道的初始化，并且 CVM 需要主动确保共享内存使用过程中的安全性，如进行必要的加密和校验。

5.3 安全 I/O 虚拟化实现

EasyAda-hypervisor 支持三种 I/O 虚拟化技术：（1）全虚拟化，用于模拟简单或安全性要求较高的设备，如串口和中断控制器等；（2）半虚拟化，用于模拟需要大量 I/O 数据的设备，如网卡，磁盘设备等；（3）设备直通，用于管理驱动较复杂或有特殊需求的设备，如硬件加速器等。

EasyAda-hypervisor 中 VM 设备初始化流程是 VM 无感的被动流程，目的是减少对 VM 代码的修改，并且采用静态分区和动态分配相结合的硬件发现机制以兼顾降低系统复杂度和提高硬件利用率两个需求，完整的设备发现和初始化分为三个步骤：（1）在系统开发时，硬件供应商需要提供可信的硬件配置信息如 FDT，而应用开发者需要根据硬件配置划分出 VM 直通设备并通过 VM 硬件配置文件实现 VM 与设备的静态绑定，而全虚拟化和半虚拟化设备可以由软件实现时分复用，因此在运行时完成设备发现；（2）在 VM 初始化时，hypervisor 会对 VM 硬件配置文件进行检查，如果需要并且存在匹配的直通设备，则将设备分配给当前 VM，

否则会回退为全虚拟化设备；（3）在 VM 初始化时，虽然直通设备已经分配给当前 VM，但是首次使用仍然需要陷入到 hypervisor 中完成设备到 VM 地址空间的映射，再由 VM 完成初始化，全虚拟化设备则在每次访问 MMIO 区域时陷入到 hypervisor 中，由 hypervisor 负责处理，半虚拟化设备则在第一次访问 MMIO 区域时陷入到 hypervisor 中完成共享内存的创建和到通信双方的映射，之后数据流无需 hypervisor 介入，控制流则按照 VM 为主，vhart 服务线程为从端的主从通信模式进行通知，VM 主动访问特定 MMIO 区域，陷入 hypervisor 并发送事件通知 vhart 服务线程进行处理，完成通知流程，当 vhart 服务线程完成处理后，则由 hypervisor 执行中断注入，通知 VM 当前 I/O 请求完成。

EasyAda-trust 在 CVM 中基本复用了 VM 全虚拟化/半虚拟化设备的管理和使用逻辑，重点拓展了 EasyAda-hypervisor 管理安全设备的能力。

安全设备管理基于安全区域管理实现，安全设备创建的时机取决于 hypervisor。当 hypervisor 需要将特定设备转换为安全设备时，仍然通过安全区域管理请求要求 TSM 驱动创建安全区域以隔离安全设备的 MMIO 区域，hypervisor 负责确保安全设备隔离不会异常中断当前业务。因此不再进行赘述，本节主要描述 EasyAda-trust 如何确保 CVM 使用设备过程中的安全性。

5.3.1 安全设备管理

EasyAda-trust 中安全设备遵循主机分配，TSM 驱动鉴权的逻辑。在系统开发时，硬件供应商需要对上层提供硬件配置文件，而 EasyAda-trust 的应用开发者则需要提供 CVM 专用的虚拟硬件配置文件，这个过程实现了 CVM 与 TSM 驱动静态设备发现流程，避免复杂的运行时设备发现逻辑。

在 VM 创建时，hypervisor 仍然按照相同的逻辑进行设备初始化，如果当前 VM 使用安全设备，虽然可以正常建立映射，但是 VM 运行时无法对设备进行任何读写操作。当 hypervisor 请求将 VM 提升为 CVM 时，TSM 驱动会对 CVM 的硬件配置文件进行检查，如果存在匹配的安全设备，则 TSM 驱动将为当前 CVM 分配安全设备对应的安全区域，否则将对应 MMIO 区域回退为全虚拟化设备。

当 CVM 运行时访问 MMIO 区域下陷到 TSM 驱动中时，TSM 驱动会根据当前 MMIO 区域类型（全虚拟化/半虚拟化/设备直通）进行分类处理，全虚拟化设备会转换为 MMIO 请求，而半虚拟化设备则会转换为共享内存请求，然后将请求转发到 hypervisor 侧进行处理并返回。而直通的安全设备则由 TSM 驱动完成物理 MMIO 区域到 CVM 地址空间的映射，由 CVM 完成设备初始化。

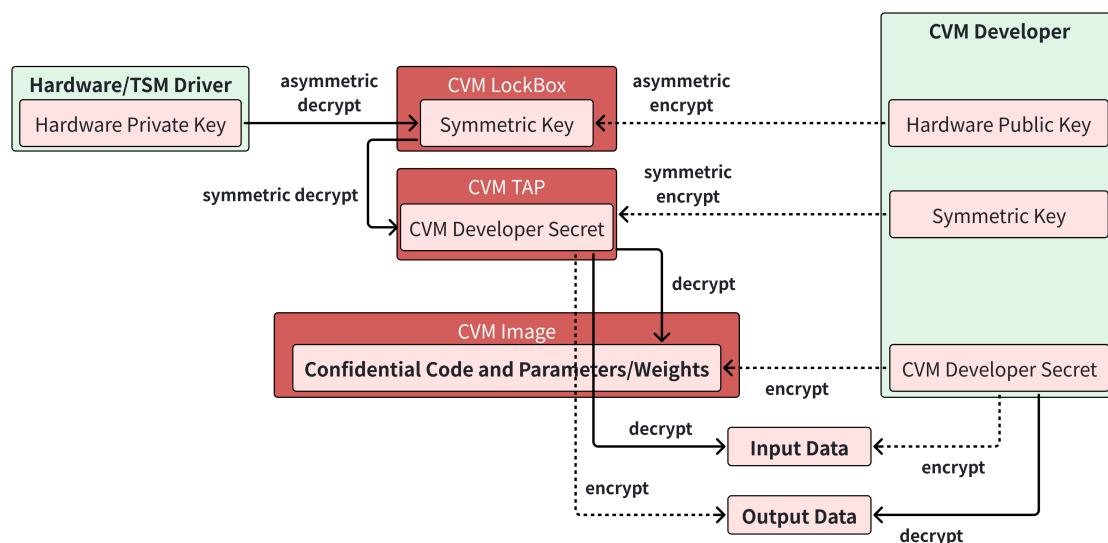


图 5-3 EasyAda-trust 下 CVM 的 I/O 密钥管理

5.3.2 数据使用环境保护

安全数据流需要建立在安全可信的执行环境上。机密应用是 I/O 数据的主要使用者，包括用户的 AI 模型和参数等。

可信硬件配置文件会用于运行时的安全设备检查，只有被 TSM 驱动标记为可信的设备可以转化为安全设备，防范 hypervisor 通过向 CVM 提供恶意设备实现窃取和篡改机密数据。

为了确保硬件平台和 TSM 驱动本身是可信的，在 TSM 驱动验证 CVM 可信的同时，CVM 也会对硬件平台进行验证，这个步骤通过密钥交换完成。图5-3展示了 CVM 使用的 I/O 密钥管理流程，根据 EasyAda-trust 的安全启动流程，CVM 在启动时会与硬件环境进行双向验证，通过双向验证后的 CVM 和硬件环境都是可信的。然后 TSM 会使用硬件私钥对锁盒进行解锁，并使用其中的对称密钥解密 TAP 获取开发者提供的秘密并存储在 TSM 驱动。当 CVM 启动时，会通过密钥管理请求从 TSM 驱动获取开发者密钥并使用密钥对核心的机密应用代码和数据进行解密，确保不会在不可信的硬件环境中泄漏用户代码和数据。

5.3.3 CPU 侧 I/O 数据安全

安全设备管理可以确保不可信组件无法以任何形式访问安全设备的 MMIO 区域，因此机密应用可以以明文形式向安全设备传输数据并且不会被篡改或窃取。

但是对基于不可信的共享内存实现的半虚拟化设备，CVM 需要对其中的数据进行额外的加密以保护数据机密性。CVM 和数据供应方（网络数据发送方或写入磁盘数据方）需要使用 CVM 开发者预先提供的对称/非对称密钥及其衍生的会话

密钥对数据进行加解密和校验，以并且不会使用篡改后的数据。

5.3.4 设备侧 I/O 数据安全

为了防止支持 DMA 的恶意外设绕过 CPU 的内存隔离直接访问安全内存，EasyAda-trust 使用 IOPMP 拓展限制了每个硬件可以访问的内存区域。在开发时，硬件供应商需要为可信设备静态分配 SID 作为唯一标识符号。hypervisor 可以在任何时刻请求将设备提升为安全设备。TSM 驱动会通过可信的硬件配置文件检查当前设备是否可信，如果设备可信则为设备创建未初始化的可信 MD，只有当安全设备实际分配给 CVM 时，TSM 驱动会将 CVM 关联的所有安全区域的授权条目加入设备的可信 MD 中，并将设备的 SID 与可信 MD 关联。此时安全设备仅能访问安全内存，杜绝了安全设备将敏感数据写入不可信内存的问题。

相对的，当 CVM 销毁并释放当前安全设备时，TSM 驱动会重置设备的可信 MD，hypervisor 可以向 TSM 驱动请求回收当前设备，TSM 驱动会重新为当前设备的 SID 绑定不可信 MD。

5.4 机密异构计算环境实现

EasyAda-trust 一个重要的目标是保障用户 AI 模型和敏感数据的机密性和完整性。该目标可以进一步细分为两个目标：

- (1) **保障用户运行环境的可信和隔离：**为了保护用户 AI 模型，CVM 仅能在可信的硬件环境上对用户模型进行解密和部署，并且由 EasyAda-trust 确保 CVM 运行环境与不可信组件的隔离。
- (2) **保障用户 I/O 数据的机密性和完整性：**为了确保用户敏感数据不被窃取或篡改，EasyAda-trust 需要对用户 I/O 数据采取必要的加密和校验，并确保用户模型使用硬件加速器过程中的安全。

同时，EasyAda-trust 为了增强系统的可拓展性，需要实现硬件加速器在不同执行环境间的复用，为了实现以上目标，EasyAda-trust 实现了专用的硬件加速器管理 CVM，并通过 I/O 半虚拟化技术实现硬件加速器在不同执行环境间的复用。下面将详细介绍 EasyAda-trust 如何实现机密异构计算环境。

5.4.1 硬件加速器的保护

为了确保 CVM 使用硬件加速器过程中的安全性，hypervisor 必须显式将硬件加速器提升为安全设备，TSM 驱动在加速器管理 CVM 启动时，会检查并尝试分配安全设备，只有经过验证的安全硬件加速器可以映射到 CVM 的地址空间中进行管理。同时，硬件加速器使用的内存缓冲区由加速器管理 CVM 内的驱动从当前

CVM 的安全内存中进行分配，不可信组件无法访问安全的内存缓冲区和 MMIO 区域。

5.4.2 加速器管理 CVM 初始化

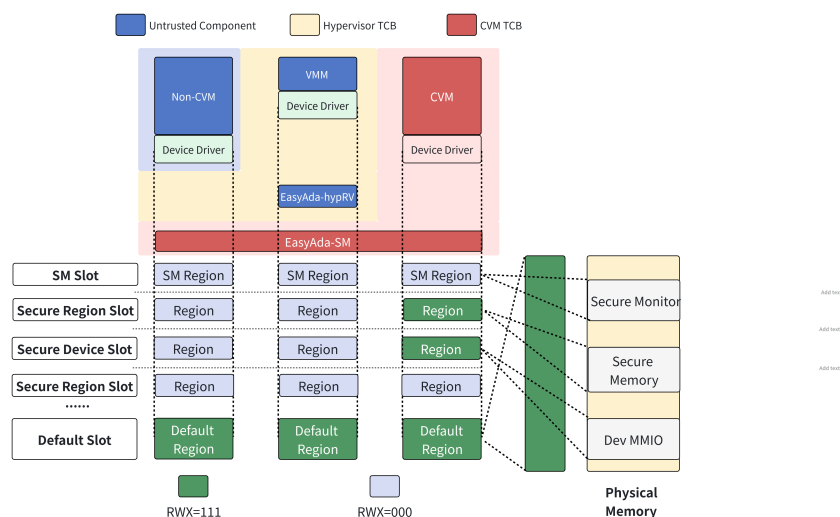


图 5-4 引入安全设备后的 CVM 安全内存视图

在未引入 CVM 的架构下，EasyAda-hypervisor 为了实现对现有驱动的复用，避免增大微内核操作系统的攻击面，会在启动阶段将硬件加速器的 MMIO 地址透传到硬件加速器管理 VM 中，由该 VM 加载驱动并完成硬件加速器初始化，后续通过 I/O 半虚拟化技术（如 VirtIO 协议）实现硬件加速器在 VM 间以及 host 间的时分复用。

加速器管理 CVM 的创建与其他 CVM 类似，在创建加速器管理 CVM 前，hypervisor 需要主动将可信的硬件加速器提升为安全硬件，因为提升时机由 EasyAda-hypervisor 决定，应当由其自身确保无任何组件正在访问硬件加速器，避免访问流程的异常中断。当硬件加速器被提升为安全设备后，不可信组件无法访问硬件加速器的 MMIO 区域，

接下来由 hypervisor 负责创建 CVM，在 TSM 驱动为 CVM 分配安全设备时，会同时为硬件加速器创建授权 MD 用于授予当前硬件加速器访问安全区域的能力，因为端侧硬件加速器多以协处理器的形式集成在 SoC 中，因此其数据缓冲区需要动态进行分配，为了降低 IOPMP 管理的开销，TSM 驱动配置 IOPMP 授权硬件加速器访问加速器管理 CVM 关联的所有安全区域以确保可以正常访问动态数据缓冲区，因为配置 DMA 时会通过 IOMMU 或 TSM 驱动软件翻译 IPA 为物理地址，避免了硬件加速器因为 DMA 配置错误写入其它安全内存中。隔离完成后，不可信组件无法访问硬件加速器的 MMIO 区域，硬件加速器也无法写入不可信的内存区

域。

加速器管理 CVM 创建成功后,如图5-4所示,此时加速器管理 CVM 是除 TSM 驱动外唯一可以访问硬件加速器软件实体。当加速器管理 CVM 实际运行时,需要按照安全启动流程, CVM 从 TSM 驱动获取开发者密钥并对开发者提供的 AI 模型和参数进行解密和部署,并对不可信的半虚拟化和全虚拟化设备进行初始化。

加速器管理 CVM 创建成功之后,可以直接对其上的机密应用提供服务。如果需要与不可信环境间进行复用,则需要与请求使用硬件加速器的不可信组件建立信道,下面将介绍信道建立的过程。

5.4.3 建立信道

加速器管理 CVM 其它组件均通过 I/O 半虚拟化技术——VirtI/O 协议实现硬件加速器的时分复用。

虚拟设备的发现在开发是通过静态的硬件配置文件如 FDT 文件静态完成, EasyAda-trust 应用开发者需要根据硬件供应商提供的可信物理硬件配置完成设备静态分区,并在加速器管理 CVM 的硬件配置文件中预留内存区域用于建立映射共享内存。

加速器管理 CVM 会对设备进行初始化,当访问到预留的内存区域时会陷入 TSM 驱动,由 TSM 驱动检查 MMIO 区域类型后,将共享内存申请请求转发到 EasyAda-hypervisor 处理。EasyAda-hypervisor 负责完成共享内存分配并返回地址给 TSM 驱动,由 TSM 驱动完成共享内存到 CVM 地址空间的映射以及 CVM 上下文的恢复,并准备进行驱动后端初始化。同时, EasyAda-hypervisor 会标记该共享内存为虚拟硬件加速器设备区域,等待映射到驱动前端中。

当不可信 VM 尝试初始化虚拟硬件加速器时,会陷入到 EasyAda-hypervisor 中,由 EasyAda-hypervisor 将会获取之前加速器管理 CVM 申请的共享内存,并将其映射到请求方 VM 对应的虚拟设备地址中。之后 VM 和加速器管理 CVM 可以通过该共享内存完成数据交换,由 TSM 驱动和 EasyAda-hypervisor 转发和注入中断进行 I/O 请求通知和回复。EasyAda-trust 并不要求加速器管理 CVM 和请求方 VM 的初始化时序,双方可以由任意一方进行初始化并完成共享内存的申请和标记, EasyAda-trust 不感知实际的进一步的协议交互和信道建立流程,通信双方可以基于共享内存,通过轮询或中断完成通信协议的初始化。

5.5 本章小结

本章详细论述了基于微内核的可信虚拟化方案的工程实现，重点阐述了如何适配 EasyAda-hypervisor 以实现 CVM 的安全管理与资源虚拟化。在 CPU 虚拟化方面，方案通过扩展 EasyAda-hypervisor 的 hart 分区架构，引入了机密与非机密 vhart 运行区，并利用 TSM 驱动接管 CVM 的两阶段安全上下文切换流程，确保了 CVM 的 CPU 状态对不可信 hypervisor 完全隐匿。在内存与 I/O 虚拟化层面，方案将安全资源的直接控制权从 EasyAda-hypervisor 中剥离，确立了由 hypervisor 负责物理内存分配、设备发现与宏观生命周期调度，进而委托 TSM 驱动完成安全区域动态构建、二阶段页表隔离映射及安全设备鉴权配置的职责解耦机制。此外，针对端侧异构计算需求，方案利用 EasyAda-hypervisor 辅助建立不可信共享内存通信信道，成功实现了专用硬件加速器管理 CVM 的安全部署与跨环境复用。综上所述，本章通过对 EasyAda-hypervisor 实施职责解耦与非侵入式安全拓展，在充分复用微内核统一资源管理优势的同时，为 CVM 提供了一个高安全、高可靠的可信虚拟化运行底座。

第六章 系统测试与分析

第七章 总结和展望

参考文献

- [1] OpenAMP. Welcome to the openamp project documentations[EB/OL]. 2026-01-11, <https://openamp.readthedocs.io/en/latest/index.html>.
- [2] ARM. Arm security technology building a secure system using trustzone technology[EB/OL]. 2009-4-1, <https://developer.arm.com/documentation/PRD29-GENC-009492/latest>.
- [3] ARM. Intel® architecture instruction set extensions programming reference[EB/OL]. 2025-11-12, <https://www.intel.com/content/www/us/en/content-details/869288/intel-architecture-instruction-set-extensions-programming-reference.html>.
- [4] AMD. Using sev with amd epyc™ processors[EB/OL]. 2023-10-03, <https://docs.amd.com/v/en-US/58207-using-sev-with-amd-epyc-processors>.
- [5] Feng E, Lu X, Du D, et al. Scalable memory protection in the PENG LAI enclave[C]. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), 2021: 275–294.
- [6] Lee D, Kohlbrenner D, Shinde S, et al. Keystone: an open framework for architecting trusted execution environments[C]. Proceedings of the Fifteenth European Conference on Computer Systems, 2020: 1-16.
- [7] GlobalPlatform. Tee system architecture v1.3[EB/OL]. 2022-05-01, <https://globalplatform.org/specs-library/tee-system-architecture/#>.
- [8] Limited A. Learn the architecture - introducing arm confidential compute architecture version 4.0[EB/OL]. 2025-03-19, <https://developer.arm.com/documentation/den0125/latest>.
- [9] Cerdeira D, Santos N, Fonseca P, et al. Sok: Understanding the prevailing security vulnerabilities in trustzone-assisted tee systems[C]. 2020 IEEE Symposium on Security and Privacy (SP), 2020: 1416-1432.
- [10] Matsakis N D, Klock F S. The rust language[C]. Association for Computing Machinery, 2014: 103-104.
- [11] Wang H, Wang P, Ding Y, et al. Towards memory safe enclave programming with rust-sgx[C]. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019: 2333-2350.
- [12] Wan S, Sun M, Sun K, et al. Rustee: Developing memory-safe arm trustzone applications[C]. Proceedings of the 36th Annual Computer Security Applications Conference, 2020: 442-453.
- [13] Popek G J, Goldberg R P. Formal requirements for virtualizable third generation architectures[J]. Commun. ACM, 1974, 17(7): 412-421.

-
- [14] Sahoo J, Mohapatra S, Lath R. Virtualization: A survey on concepts, taxonomy and associated security issues[C]. 2010 Second International Conference on Computer and Network Technology, 2010: 222-226.
- [15] Qumranet A, Qumranet Y, Qumranet D, et al. Kvm: The linux virtual machine monitor[J]. Proceedings Linux Symposium, 2007, 15.
- [16] Barham P, Dragovic B, Fraser K, et al. Xen and the art of virtualization[J]. SIGOPS Oper. Syst. Rev., 2003, 37(5): 164-177.
- [17] Dall C, Li S W, Nieh J. Optimizing the design and implementation of the linux ARM hypervisor[C]. 2017 USENIX Annual Technical Conference (USENIX ATC 17), 2017: 221-233.
- [18] Hwang J Y, Suh S B, Heo S K, et al. Xen on arm: System virtualization using xen hypervisor for arm-based secure mobile phones[C]. 2008 5th IEEE Consumer Communications and Networking Conference, 2008: 257-261.
- [19] Martins J, Tavares A, Solieri M, et al. Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems[C]. Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020), 2020: 3:1-3:14.
- [20] Li H, Xu X, Ren J, et al. Acrn: a big little hypervisor for iot development[C]. Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, 2019: 31-44.
- [21] Trujillo S, Crespo A, Alonso A. Multipartes: Multicore virtualization for mixed-criticality systems[C]. 2013 Euromicro Conference on Digital System Design, 2013: 260-265.
- [22] Matos E d, Ahvenjärvi M. sel4 microkernel for virtualization use-cases: Potential directions towards a standard vmm[J]. Electronics, 2022, (24).
- [23] Limited B. Qnx hypervisor 8.0 empowering embedded systems with trusted virtualization and mixed-criticality support[EB/OL]. 2025-10-01, <https://qnx.software/content/dam/qnx-xwalk/pdf/product-briefs/qnx-hypervisor-8-product-brief.pdf>.
- [24] McCune J M, Li Y, Qu N, et al. Trustvisor: Efficient tcb reduction and attestation[C]. 2010 IEEE Symposium on Security and Privacy, 2010: 143-158.
- [25] Steinberg U, Kauer B. Nova: a microhypervisor-based secure virtualization architecture[C]. Proceedings of the 5th European Conference on Computer Systems, 2010: 209-222.
- [26] Jia Y, Liu S, Wang W, et al. HyperEnclave: An open and cross-platform trusted execution environment[C]. 2022 USENIX Annual Technical Conference (USENIX ATC 22), 2022: 437-454.
- [27] Pereira S, Sousa J, Pinto S, et al. Bao-enclave: Virtualization-based enclaves for arm[C]. 2022 IEEE 8th World Forum on Internet of Things (WF-IoT), 2022: 1-6.

- [28] AMD. Amd sev-snp: Strengthening vm isolation with integrity protection and more[EB/OL]. 2020-01-01, <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more>.
- [29] AMD. Intel® trust domain extensions[EB/OL]. 2025-06-01, <https://cdrdv2.intel.com/v1/dl/getContent/690419>.
- [30] Wang J, Li A, Li H, et al. Rt-tee: Real-time system availability for cyber-physical systems using arm trustzone[C]. 2022 IEEE Symposium on Security and Privacy (SP), 2022: 352-369.
- [31] Brasser F, Gens D, Jauernig P, et al. Sanctuary: Arming trustzone with user-space enclaves[C]. 2019: .
- [32] Zhao S, Zhang Q, Qin Y, et al. Sectee: A software-based approach to secure enclave architecture using tee[C]. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019: 1723-1740.
- [33] Hua Z, Gu J, Xia Y, et al. vTZ: Virtualizing ARM TrustZone[C]. 26th USENIX Security Symposium (USENIX Security 17), 2017: 541-556.
- [34] Feng E, Feng D, Du D, et al. siopmp: Scalable and efficient i/o protection for tees[C]. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, 2024: 1061-1076.
- [35] Du D, Yang B, Xia Y, et al. Accelerating extra dimensional page walks for confidential computing[C]. Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, 2023: 654-669.
- [36] Ozga W. Towards a formally verified security monitor for vm-based confidential computing[C]. Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy, 2023: 73-81.
- [37] Wu X, Tian D J, Kim C H. Building gpu tees using cpu secure enclaves with gevisor[C]. Proceedings of the 2023 ACM Symposium on Cloud Computing, 2023: 249-264.
- [38] Deng Y, Wang C, Yu S, et al. Strongbox: A gpu tee on arm endpoints[C]. Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, 2022: 769-783.
- [39] Fan S, Hua Z, Xia Y, et al. Xputee: A high-performance and practical heterogeneous trusted execution environment for gpus[J]. ACM Trans. Comput. Syst., 2025, 43(1-2).
- [40] Wang C, Zhang F, Deng Y, et al. Cage: Complementing arm cca with gpu extensions[C]. Proceedings of the 31st Annual Network and Distributed System Security Symposium.
- [41] Wang C, Tang D, Ci C, et al. ccAI: A Compatible and Confidential System for AI Computing[M], Association for Computing Machinery, 2025, 340-353.

- [42] Lu H, Deng Y, Mertoguno S, et al. Mole: Breaking gpu tee with gpu-embedded mcu[C]. Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, 2025: 693–707.
- [43] International R V. The risc-v instruction set manual: Volume ii[EB/OL]. 2025-05-08, https://docs.riscv.org/reference/isa/_attachments/riscv-privileged.pdf.
- [44] Hunt G D H, Pai R, Le M V, et al. Confidential computing for openpower[C]. Proceedings of the Sixteenth European Conference on Computer Systems, 2021: 294–310.
- [45] Lee J, Wang Y, Rajat R, et al. Characterization of gpu tee overheads in distributed data parallel ml training[J]. arXiv preprint arXiv:2501.11771, 2025.
- [46] RustSBI. rustsbi[EB/OL]. 2026-02-10, <https://github.com/rustsbi/rustsbi>.

攻读硕士学位期间取得的成果